

Mining and Structuring Logging Guidelines for Improving Log Analysis

Dilsan Mahadeva

RUHR-UNIVERSITÄT BOCHUM

Master's Thesis – September 30, 2026
Chair of Software Engineering.

Supervisor: Thorsten Berger
Advisor: Mukelabai Mukelabai



Abstract

Software systems generate large amounts of log data for debugging, monitoring, anomaly detection and root cause analysis. Effective log analysis depends on the quality and consistency of these logs. Because logging practices are applied inconsistently and guidelines are unclear, individual developers' results show poor logging formats, structures, and information.

This thesis investigates software logging from both developer and guideline perspectives. First, it examines the logging practices and challenges software developers encounter during software development and maintenance. Second, existing logging guidelines are collected from academic literature and open-source software repositories. The identified guidelines are organised in a structured catalogue and analysed and sorted into themes and characteristics, including what to log, where to log, how to log and log-level verbosity. Finally, the identified guidelines are compared with the practices and challenges reported by software developers to align the practical logging needs with the guidelines in the catalogue.

The thesis provides an overview of existing logging guidance and software development practice. The contribution is uploading a logging guideline catalogue for software developers to refer to.

Official Declaration

Hereby I declare, that I have not submitted this thesis in this or similar form to any other examination at the Ruhr-Universität Bochum or any other institution or university.

I officially ensure, that this paper has been written solely on my own. I hereby officially ensure, that I have not used any other sources but those stated by me. Any and every parts of the text which constitute quotes in original wording or in its essence have been explicitly referred by me by using official marking and proper quotation. This is also valid for used drafts, pictures and similar formats.

I also officially ensure that the printed version as submitted by me fully confirms with my digital version. I agree that the digital version will be used to subject the paper to plagiarism examination.

Not this English translation, but only the official version in German is legally binding

Eidesstattliche Erklärung

Ich erkläre, dass ich keine Arbeit in gleicher oder ähnlicher Fassung bereits für eine andere Prüfung an der Ruhr-Universität Bochum oder einer anderen Hochschule eingereicht habe.

Ich versichere, dass ich diese Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen benutzt habe. Die Stellen, die anderen Quellen dem Wortlaut oder dem Sinn nach entnommen sind, habe ich unter Angabe der Quellen kenntlich gemacht. Dies gilt sinngemäß auch für verwendete Zeichnungen, Skizzen, bildliche Darstellungen und dergleichen.

Ich versichere auch, dass die von mir eingereichte schriftliche Version mit der digitalen Version übereinstimmt. Ich erkläre mich damit einverstanden, dass die digitale Version dieser Arbeit zwecks Plagiatsprüfung verwendet wird.

DATE

DILSAN MAHADEVA

Erklärung

Als öffentlich finanzierte Forscher wollen wir durch die Veröffentlichung dieser Arbeit einen Beitrag zur Forschungsgemeinschaft leisten, damit andere Forscher dieses Wissen nutzen und weiterentwickeln können. Ich bin damit einverstanden, dass meine Dissertation dauerhaft und öffentlich gespeichert wird. Sie kann dann z. B. von Google Scholar indexiert werden, so dass sie von anderen Forschern gefunden und referenziert werden kann.

Consent

As publicly funded researchers, we aim to contribute to the research community by publishing this work, so other researchers can use and further develop this knowledge. I agree that my thesis will be permanently and publicly stored. It can then be indexed by, for instance, Google Scholar so that it can be found and referenced by other researchers.

DATE

DILSAN MAHADEVA

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Goals and Research Questions	3
1.4	Organisation of this Thesis	5
2	Background	7
2.1	Software Logging	7
2.1.1	Purpose of Logging	8
2.1.2	Log Messages and Log Statements	8
2.1.3	Log Levels and Verbosity	9
2.1.4	Structured and Unstructured Logging	9
2.2	Logging in Software Development and Maintenance	10
2.2.1	Logging Implementation	10
2.2.2	Log Analysis	11
2.2.3	Fault Localisation and Root Cause Analysis	11
2.2.4	Challenges of Software Logging	12
2.3	Logging Guidelines and Practices	12
2.3.1	What to Log	13
2.3.2	Where to Log	14
2.3.3	How to Log	14
2.3.4	Guidelines for Log Levels and Verbosity	14
2.4	Log Quality and Its Impact on Analysis	15
2.4.1	Log Quality and Consistency	15
2.4.2	Impact of Logging Decisions on Log Analysis	16
2.5	Related Work	17
2.5.1	Studies on Developer Logging Practices	17
2.5.2	Research on Logging Guidelines	18
2.5.3	Research on Logging Quality and Analysis	18
2.6	Research Gap	19
3	Methodology	21
3.1	Research Design	21
3.2	Developer Study	23
3.2.1	Study Design	23
3.2.2	Participants and Recruitment	23

3.2.3	Survey and Data Collection	24
3.2.4	Data Analysis	24
3.3	Collection of Logging Guidelines from Academic Literature	25
3.3.1	Search Strategy	25
3.3.2	Selection Criteria	26
3.3.3	Guideline Extraction	27
3.4	Collection of Logging Guidance from Open-Source Repositories	28
3.4.1	Repository Search Strategy	28
3.4.2	Repository Retrieval	30
3.4.3	Repository Selection	30
3.4.4	Repository Characteristics	31
3.4.5	Data Cleaning and Exclusion	32
3.4.6	Relevant Logging Content Identification and Extraction	33
3.5	Logging Guideline Catalogue Construction and Classification	36
3.5.1	Contextual Content Tagging	36
3.5.2	Guideline Formulation and Normalization	37
3.5.3	Manual Review of Normalised Guidelines	38
3.5.4	Relationship Between Academic and Open-Source Guidelines	38
3.5.5	Initial Literature-Based Codebook	39
3.5.6	Inductive Codebook Extension and Final Classification	40
3.5.7	Guideline Consolidation and Traceability	41
3.5.8	Validation of the Guideline Catalogue	43
3.6	Thematic Analysis	45
3.6.1	Theme Identification	46
3.6.2	Theme Distribution	46
3.6.3	Coverage of Academic Recommendations	47
4	Results	49
4.1	Developer Study Results	49
4.1.1	Survey Response Overview	49
4.1.2	Logging Implementation Practices	50
4.1.3	Log Analysis Practices	54
4.1.4	Fault Localization Practices	57
4.1.5	Logging Challenges	60
4.1.6	Answer to RQ1	60
4.2	Logging Guidelines from Academic Literature	61
4.2.1	Literature Search Results	61
4.2.2	Selected Literature	62
4.2.3	Extracted Logging Guidelines	63
4.2.4	Coverage in the Open-Source Catalogue	63
4.2.5	Answer to RQ2	64
4.3	Logging Guidelines from Open-Source Repositories	65
4.3.1	Repository Collection Results	65

4.3.2	Repository Characteristics	66
4.3.2.1	Project Types	67
4.3.2.2	Programming Languages	68
4.3.2.3	Application Fields	68
4.3.3	Extracted Logging Guidelines	68
4.3.4	Validation and Refinement of the Guideline Catalogue	69
4.3.5	Distribution Across Logging Categories	70
4.3.6	Answer to RQ3	71
4.4	Themes in the Logging Guideline Catalogue	73
4.4.1	Thematic Catalogue Overview	73
4.4.2	What to Log	74
4.4.3	Where to Log	75
4.4.4	How to Log	77
4.4.5	Log Levels and Verbosity	77
4.4.6	Final Thematic Codebook	78
4.4.7	Answer to RQ4	80
5	Discussion and Conclusion	81
5.1	Discussion of Findings	81
5.1.1	Developer Practices and Challenges	81
5.1.2	Logging Guidance in Academic Literature	83
5.1.3	Open-Source Logging Guidelines	84
5.1.4	Themes within the Logging Guideline Catalogue	85
5.2	Contribution and Implications	87
5.3	Threats to Validity	88
5.3.1	Internal Validity	89
5.3.2	External Validity	89
5.3.3	Construct Validity	90
5.3.4	Conclusion Validity	91
5.4	Future Work	92
5.5	Conclusion	93
	List of Figures	95
	List of Tables	97
	List of Prompts	98
	Bibliography	99
A	Survey Results	104
A.0.1	Logging Implementation	104
A.0.1.1	Open-Ended Logging Implementation Responses . .	106
A.0.2	Log Analysis	107
A.0.2.1	Open-Ended Log Analysis Responses	108

A.0.3	Fault Localization	111
A.0.3.1	Open-Ended Fault Localization Responses	112

1 Introduction

The first chapter provides an overview of the critical role software logging plays in modern software development and maintenance, highlighting how effective logging supports system reliability, debugging, and monitoring. It also addresses the significant challenges posed by inconsistent logging practices and heterogeneous log data, which can hinder both manual and automated analysis. By establishing the practical and research relevance of these issues, the chapter sets the stage for the subsequent examination of logging guidelines and the need for systematic approaches to improve log quality. In response to these challenges, this chapter first outlines the motivation for addressing inconsistencies in software logging, and then articulates the specific research problem that arises from these issues. Then, the next chapter describes the goals and research questions on logging practices and guidelines. The structure of this thesis is outlined in the following sections.

1.1 Motivation

Modern software systems generate large amounts of log data, providing valuable insights into runtime behaviour. Logs are widely used for debugging, system monitoring, anomaly detection, failure diagnosis and root cause analysis. In large-scale, distributed software systems, the volume of generated log data can become too large for efficient manual inspection. As a result, automated log analysis techniques have become very important for supporting developers and operators in analysing system behaviour and identifying relevant events [1, 2]. The effectiveness of automated log analysis does not only depend on the analysis technique. It also depends on the quality and consistency of the log data that is provided as input. The latest studies show that such inconsistencies can reduce accuracy and reliability [3, 4, 5]. Automated techniques generally require log messages to contain useful information and to follow structures to process reliably. Inconsistent message formats, missing contextual information, unsuitable log levels or poorly placed log statements can make the analysis harder and may reduce the quality of the generated log data.

This creates an important problem because individual developers already influence log quality. Developers have to decide what information should be logged, where logging statements should be placed, how log messages should be created and which log level should be used [6, 7]. Even in industry, logs remain inconsistent [8]. Developers often make these decisions during software development, and they may differ across

developers, teams, projects, programming languages, and application domains. This makes investigating logging guidelines an interesting research problem. It remains unclear which logging recommendations are provided, how specific and detailed these recommendations are, and what extent different sources offer similar guidance, and which aspects of logging receive the greatest attention. Moreover, it is also relevant to investigate whether guidelines exist for the different decisions developers face when implementing logging and whether these recommendations could support generating logs better suited to automated analysis.

Consequently, even software systems that generate large amounts of log data may not generate logs well suited for later automated analysis. Existing research has investigated different approaches for improving automated log analysis, for example, through more effective parsing and analysis techniques. However, improving the analysis method alone does not solve problems that already exist in the input data. If important information is missing or logging statements are implemented inconsistently, an analysis technique can only process the information originally recorded. Logging practices and recommendations are documented in open-source projects [9] and in grey literature [10, 11]. However, this guidance is distributed across different sources, formats, and software contexts, making it difficult to obtain a structured overview of the available recommendations. Therefore, it is also relevant to investigate how logging practices and recommendations used during software development can improve log data at its source.

Observations from industrial software projects show that logging structures and conventions can be inconsistent or only partially documented, supporting this thesis's motivation. Especially in larger projects involving several developers and software components, incomplete or unclear logging guidance can result in different logging styles within the same system. A more structured understanding of existing logging guidelines could therefore support more consistent logging practices and provide a foundation for improving the quality of generated log data.

Motivated by this, this thesis investigates existing logging guidelines and practices and structures them into a catalogue based on relevant logging decisions.

1.2 Problem Statement

Automated log analysis techniques are widely used to help developers understand and maintain software systems. These techniques are commonly applied for tasks such as debugging, system monitoring, anomaly detection and root cause analysis. Most automated log analysis methods assume that log data are structured, consistent, and contain enough information to support analysis. In practice, this assumption often does not hold.

Developers usually write log messages manually, often without clear or standardised logging guidelines. As a result, log formats and message contents vary across software projects, development teams and programming languages. While some log

messages contain detailed and structured information, others lack important context or follow inconsistent formats. This inconsistency complicates automated tools and reduces the reliability of log analysis. Although many existing studies focus on improving log analysis techniques, such as log parsing or anomaly detection, less attention is given to how log data is created in the first place. The quality of the input logs is often taken for granted. When log data are inconsistent, incomplete or poorly structured, even advanced automated analysis techniques may fail to produce accurate and meaningful results.

Logging guidelines and practices can improve log quality by specifying what to log, when to log, and how to structure log messages. In practice, however, developers often rely on personal experience or make logging decisions on the spot without a systematic and reusable approach. Logging rules are rarely documented in a clearly and consistently and are often spread across documentation and source code. This makes it difficult to identify common practices or apply them consistently across different projects and programming languages.

As a result, developers can not refer to a single structured reference that brings together the different logging recommendations documented across academic and practical sources. Without a systematic approach to collecting and organising logging guidance, it is difficult to identify recurring recommendations, differences between sources, and areas where guidance is limited.

A structured catalogue of logging guidelines can therefore provide a consolidated reference for investigating existing logging recommendations and support more consistent consideration of logging decisions during software development.

1.3 Goals and Research Questions

The goal of this thesis is to develop a structured and traceable catalogue of logging guidelines and to investigate the logging decisions represented within practical software-development guidance. The catalogue covers four central logging dimensions: what to log, where to log, how to log, and which log levels or verbosity to use. To achieve this goal, the study combines three complementary perspectives. First, the study investigates developer practices and challenges through an industry survey. Second, the study analyses academic literature to identify existing logging recommendations and establish an initial literature-derived classification. Third, the study collects logging guidance from open-source repositories and transforms it into a consolidated guideline catalogue.

The academic recommendation set is then compared with the consolidated open-source catalogue to examine how research recommendations are represented in the collected practical guidance. The catalogue's thematic structure is analysed using a deductive-inductive classification in which literature-derived themes provide the initial structure, and additional themes are introduced where required by the open-source guideline content.

Therefore, this thesis formulates the research questions, presented in Table 1.1.

RQ1	What logging practices and challenges do software developers encounter in software development and maintenance?
RQ2	What logging guidelines are documented in academic literature?
RQ3	What logging guidelines are documented in open-source repositories?
RQ4	What themes can be identified within the logging guideline catalogue?

Table 1.1: Research Questions

RQ1 investigates the logging practices and challenges encountered by software developers during software development and maintenance. Insights into logging practices and challenges show how software developers struggle to implement consistent logging statements.

This study investigates industry practices through a survey of software developers. The survey focuses on log implementation, log analysis, and fault localisation practices. Understanding these practices and challenges from real-world industry provides a reference for existing and missing logging guidelines.

RQ2 focuses on identifying logging guidelines from academic literature. Examining research and academic literature shows how far researchers have advanced in analysing logging.

Most research has investigated different aspects of software logging, including log placement, log content, log levels, log quality and log analysis. The literature includes both logging recommendations and studies that discuss empirical, conceptual, or tool-oriented evidence about logging. This gives the catalogue a research-based perspective and shows which recommendations have already been discussed and validated in academic work.

RQ3 investigates logging guidelines in open-source software repositories. Guidelines from open-source repositories provide a practical source for logging guidance. This can be extracted from developers' documentation, configuration files, Markdown files and other repository artefacts. Characteristics such as the number of commits, contributors, programming language and application field provide contextual information about these repositories.

Guidelines are often distributed across repositories and may not be explicitly described as logging guidelines. Therefore, different search queries and collection strategies are used to identify relevant results while maintaining sufficient coverage

and relevance. High development activity and community participation can indicate that a project is actively maintained and used. At the same time, language and field information help determine the software contexts in which particular logging guidelines occur. This resulting collection forms the empirical basis for developing a structured catalogue of logging guidelines from open-source software repositories.

RQ4 examines the themes and characteristics that result from the identified logging guidelines. Classifying the collected guidelines by theme, pattern, and characteristics supports creating a structured, reusable guideline catalogue. This includes aspects such as what information should be logged, where logging should occur, how log messages should be structured and which log levels or verbosity should be used. The open-source guidelines are classified within these dimensions using the literature-derived codebook as the deductive starting point. When the initial themes do not sufficiently capture the collected guidance, additional themes are developed inductively from the guideline content. This classification forms the thematic structure of the reusable logging guideline catalogue and supports analysis of the concerns represented within the collected practical guidance.

1.4 Organisation of this Thesis

Following the introduction, Chapter 2, *Background*, explains logging concepts to support the subsequent chapters. This chapter includes related work to provide context for current research.

Chapter 3, *Methodology*, explains how the research is investigated. Therefore, this chapter explains in detail how logging guidelines are collected for the literature search and open-source project. This includes analysing academic literature, collecting and processing open-source repository guidance, constructing and validating the logging guideline catalogue, and classifying it thematically and comparing its coverage.

Chapter 4, *Results*, contains the results derived from the methodology. It provides a detailed analysis of the collected data and the developed logging guidelines catalogue. Finally, the *Discussion and Conclusion* present the interpretation and potential use of the logging guideline catalogue. It also addresses limitations and possible challenges that may arise in future work.

2 Background

This chapter introduces the concepts required to understand the role of logging in software development and maintenance. It first provides an overview of software logging, including its purpose, the distinction between log statements and log messages, the use of log levels and verbosity, and structured and unstructured representations of log data.

The chapter then discusses logging in software development and maintenance. It focuses on logging implementation, log analysis, fault localisation, root cause analysis, and the challenges developers encounter when producing and using log information. These concepts provide the background for the developer study conducted in this thesis.

Subsequently, logging guidelines and practices are introduced according to four principal aspects: *what to log*, *where to log*, *how to log*, and *log levels and verbosity*. These aspects provide the conceptual basis for analysing logging recommendations identified in academic literature and for organising and classifying the logging guidelines collected from academic literature and open-source repositories.

Finally, the chapter discusses logging quality and the impact on logging analysis to establish the relationship between decisions made during logging implementation and the later use of generated log data. The chapter then presents related research, followed by the research gap this thesis addresses.

2.1 Software Logging

Software logging refers to the recording and monitoring of information about events and states that occur during the execution of a software system. Developers introduce logging statements into source code to make relevant runtime information observable. When a logging statement is executed, it generates a log message that can subsequently be stored, inspected, searched or processed [5].

Logging is used throughout different phases of the software lifecycle. During development, log information can help developers understand program behaviour, inspect execution states and identify unexpected behaviour. In deployed systems, logs can support monitoring, troubleshooting, failure diagnosis, auditing, security analysis and other operational activities [5, 12].

The usefulness of logging depends on several implementation decisions. Developers must determine which information should be recorded, where logging statements

should be placed, how generated messages should be represented and how much information should be produced. These decisions determine which runtime information will later be available to developers and analysis tools [5, 13].

The following subsections introduce the main concepts of software logging that are required for the remainder of this thesis.

2.1.1 Purpose of Logging

The primary purpose of software logging is to provide information about a software system's behaviour and state during execution. Unlike information that exists only temporarily during runtime, log data preserves event information after it occurs. Recorded information can therefore be inspected and analysed at a later time, enabling developers and operators to investigate system behaviour without necessarily reproducing the original execution [5].

During software development, logs can observe program execution, inspect relevant program states, trace operations, and support debugging. After deployment, logs can additionally support monitoring, troubleshooting, failure diagnosis, auditing, security-related analysis, and operational maintenance [5, 12].

Different purposes may require different types and amounts of information. Detailed diagnostic information can be useful during development because developers may require insight into individual execution steps and internal program states. In production environments, recording the same amount of information can introduce additional storage and processing costs or make relevant events more difficult to identify [14].

Logging therefore requires balancing sufficient runtime information for its intended purpose with avoiding unnecessary or excessive output to save resources and maintain system performance.

2.1.2 Log Messages and Log Statements

An important distinction exists between a *log statement* and a *log message*. A log statement is an instruction embedded in the source code that specifies recording information when the program reaches a particular point. A log message is the runtime output generated when the corresponding statement is executed [5, 9].

For example, a developer may introduce a logging statement such as:

```
logger.error("Failed to process order {}", orderId);
```

If the variable `orderId` contains the value 4817, execution of this statement may result in a log message such as:

```
2026-01-01 10:00:00 ERROR Failed to process order 4817
```

The logging statement remains part of the source code, whereas the log message represents information produced during a particular program execution. A logging statement may contain both static and dynamic information. Static information describes the event, whereas dynamic information is obtained from the current execution. Depending on the implementation and logging framework, a resulting log message may contain a timestamp, severity level, component or class name, event description, identifier, error information or relevant program values.

The placement and content of logging statements therefore determine which information becomes observable during later analysis [13].

2.1.3 Log Levels and Verbosity

Log levels commonly indicate the severity or purpose of a log message. Typical levels include *DEBUG*, *INFO*, *WARN* and *ERROR*, although the available levels and their exact interpretation depend on the logging utility and conventions used within a software project [15].

Lower-severity levels are commonly used for detailed diagnostic or execution information, while higher-severity levels generally indicate events that require greater attention. Log levels can additionally control which messages are emitted or retained in a particular execution environment.

Selecting an appropriate log level is itself a logging decision. Previous research has investigated the automatic recommendation and adaptation of logging levels [16, 17, 18].

Closely related to log levels is the concept of *log verbosity*. Verbosity describes the overall amount and level of detail contained in generated log output. High verbosity can provide detailed information for debugging and diagnosis, while simultaneously increasing the volume and cost of logging. Research on distributed systems has shown that logging volume can result in measurable costs [14].

Low verbosity reduces generated information but may leave insufficient detail when investigating unexpected behaviour. Selecting appropriate verbosity therefore requires consideration of both the intended purpose and operating environment of the software.

2.1.4 Structured and Unstructured Logging

Log messages may be represented as free-text or in more structured formats. Software logs are frequently considered semi-structured because individual messages can contain both recurring textual structures and variable runtime information [6, 19].

In practice, developers commonly use (custom) logging frameworks or logging utilities to generate and manage log messages [15]. Depending on the framework and its configuration, log information can also be produced in a structured format, for

example, by representing individual attributes as separate fields rather than embedding all information within a free-text message.

Structured or consistently formatted representations can simplify filtering, querying, correlation, and automated processing. Log parsing research similarly attempts to identify recurring structures and variable parameters in semi-structured log messages [1, 2].

2.2 Logging in Software Development and Maintenance

Logging is not limited to generating runtime information. Developers interact with logging throughout software development and maintenance by introducing and modifying logging statements, examining generated log output and using logs to investigate unexpected software behaviour [9, 20]

For this thesis, several activities are particularly relevant: logging implementation, log analysis, fault localisation and root cause analysis. These activities describe different stages in producing and using logging information and provide the conceptual basis for investigating developers' practical experiences with logging.

2.2.1 Logging Implementation

Logging implementation, also called log instrumentation, describes the process of adding, configuring and maintaining logging code within a software system. Chen and Jiang distinguish log instrumentation from log management and identify logging approach, logging utility integration and logging code composition within log instrumentation [5].

During implementation, developers make decisions concerning which events and states should be recorded, where logging statements should be located, how messages should be formulated and which log level should be assigned [13, 16].

The intended use of the generated information influences decisions made during logging implementation. Logging introduced during development may primarily support debugging and understanding program execution. Logging in deployed software may additionally support monitoring, maintenance, troubleshooting, security analysis and failure investigation.

Logging implementation is not limited to adding new statements. Logging code is actively maintained and modified during software development [9, 20]. Components change, new functionality is introduced, and existing logging statements can become outdated, unnecessary or insufficient.

The quality of the resulting log information therefore depends substantially on implementation decisions made by developers [5].

2.2.2 Log Analysis

Log analysis refers to examining recorded log data to understand the behaviour of a software system. Developers and operators may analyse logs to identify relevant events, investigate errors, monitor system behaviour or reconstruct sequences of actions that occurred before a failure.

Automated log analysis has received considerable research attention, including techniques for parsing, anomaly detection, failure prediction and diagnosis [6]. However, practical log analysis can also consist of manual or semi-manual activities.

An empirical study at Microsoft involving questionnaire responses from 105 employees and interviews with 12 employees investigated the scenarios, methods, tools and pain points associated with log analysis in practice [8]. Developers may manually inspect files, search for particular messages or identifiers, filter messages by severity or component, compare entries across time, or use dedicated analysis and visualisation tools.

Analysis complexity can increase when information is distributed across different components and services. Furthermore, missing contextual information, inconsistent messages, and large volumes of irrelevant output can make relevant events harder to identify. Log analysis therefore constitutes an important activity during both software development and maintenance, and provides information that may support subsequent failure investigation.

2.2.3 Fault Localisation and Root Cause Analysis

Fault localisation involves identifying the part of a software system associated with an observed failure or incorrect behaviour. Although a failure may be visible, identifying its origin can require information about execution and internal system state. Logs can support this process by preserving execution information that would otherwise be unavailable after a failure. Research on software diagnosability has demonstrated that improving logging information can reduce the set of potential failure causes that developers must investigate [21].

Log messages can also contribute contextual development knowledge when developers attempt to understand software behaviour during maintenance [22].

Fault localisation can be distinguished conceptually from root cause analysis. Fault localisation primarily concerns narrowing down the location or component associated with an observed problem. Root cause analysis goes further by attempting to determine why the failure occurred and which underlying condition or sequence of events produced it.

Log information can contribute to both activities. A sequence of messages can indicate which component first reported abnormal behaviour, while contextual information can help developers investigate the conditions that caused the event.

The effectiveness of logs for fault localisation and failure diagnosis depends on whether the necessary information was recorded during the original execution. If

relevant runtime information is absent, developers may need to reproduce the failure, introduce additional instrumentation or rely on other debugging techniques [21].

2.2.4 Challenges of Software Logging

Implementing useful logging is a non-trivial software engineering activity. A recent literature review of software logging practices identifies a broad range of challenges spanning instrumentation, storage and preparation of logs for subsequent analysis [23]. Industrial studies likewise report differences between developers' logging intentions and the information that is finally captured [24].

One central challenge concerns the *selection of information*. Insufficient logging can leave relevant system behaviour unobservable. At the same time, excessive logging can create large volumes of output without providing equivalent additional value.

A related challenge concerns the *placement of logging statements*. Logging too little can miss out on execution information required for maintenance, while logging too much can hide relevant information and introduce additional information noise [25].

The *quality and consistency of messages* can also affect their usefulness. Unclear wording, inconsistent terminology, and missing contextual information or different representations of equivalent events can complicate later interpretation.

Developers must additionally determine appropriate *log levels and verbosity*. Incorrect levels can cause significant events to be overlooked or result in many messages being treated as equally important.

Logging may also introduce *performance and storage costs*. Generating, transmitting and retaining large amounts of runtime information requires system resources [14].

Another concern involves *privacy and security*. Runtime information can potentially contain sensitive values if developers do not consider which information should or should not be recorded. Therefore, selecting data for logging involves not only deciding what is useful, but also what is appropriate to retain.

Finally, logging changes as software evolves. Empirical studies show that logging code changes throughout software evolution rather than remaining static after its initial introduction [9, 20].

These challenges show that logging involves a series of software engineering decisions, not simply adding output statements. Logging guidelines can help developers make these decisions more consistently and systematically.

2.3 Logging Guidelines and Practices

Logging guidelines recommend on how to design and implement logging in software systems. Such recommendations can support developers in deciding which information should be recorded, where logging statements should be introduced, how the

generated messages should be constructed and which log level or degree of verbosity should be used.

Previous empirical and survey research demonstrates that logging involves several distinct implementation decisions rather than a single activity [5, 12, 13].

In this thesis, logging guidelines are initially considered according to these four aspects:

- *what to log*,
- *where to log*,
- *how to log*, and
- *log levels and verbosity*.

These aspects provide a common conceptual structure for describing recommendations. In this thesis, classifications identified in academic literature provide the initial basis for organising the practical guidance collected from open-source repositories.

A particular guideline may address more than one aspect. For example, a recommendation can specify that an exception should be logged while also prescribing the contextual information to include in the corresponding message. Nevertheless, the four aspects provide a useful starting point for organising the principal decisions involved in logging implementation.

2.3.1 What to Log

The aspect *what to log* concerns selecting the information that is recorded during software execution. This can include events, errors, state changes, execution progress, system interactions, requests, and runtime values.

Empirical logging research has directly examined what information developers record. Fu et al. emphasise that developers must make informed decisions about both where and what to log [13]. Pecchia et al. investigated event-logging practices in industry and identified different forms of runtime information captured through logging [26]. The usefulness of log data depends on whether the recorded information provides sufficient context for its intended purpose. Information useful for debugging may differ from information required for monitoring, security analysis or operational maintenance.

For example, recording only that an operation failed may provide little diagnostic value. Information identifying the operation, affected entity, relevant runtime state and error condition can make the event considerably easier to investigate.

Decisions concerning what to log also include information that should *not* be recorded. This can increase log volume and introduce privacy and security risks.

Selecting log content therefore requires weighing the need for useful contextual information against unnecessary or inappropriate data.

2.3.2 Where to Log

The aspect *where to log* concerns where logging statements appear in source code and the software architecture.

Fu et al. conducted an industrial empirical study on logging locations and identified recurring categories of locations where developers introduce logging statements [13]. Subsequent work has further investigated and automatically suggested logging locations at the code-block level [25].

A logging statement can capture only the information available at its execution point. Potential locations can therefore include error-handling paths, important control-flow decisions, component boundaries, external interactions and state transitions.

Too few logging statements can leave important execution paths unobservable, while excessive placement can create unnecessary output [25].

The placement of logging statements matters because it determines which portions of program execution become observable during later analysis.

2.3.3 How to Log

The aspect *how to log* concerns how logging is implemented and how recorded information is represented.

Chen and Jiang organise log instrumentation research around three steps: the logging approach, logging utility integration and logging code composition [5]. These aspects show that implementation involves more than deciding which event to record. Logging utilities provide functionality to produce and manage log messages. An empirical study of Java projects found diverse and sometimes complex patterns in the use of logging utilities, including projects that combine multiple utilities or implement project-specific logging abstractions [15].

At the message level, how information is logged can include decisions concerning wording, formatting, identifiers, timestamps, error information, contextual values and the representation of dynamic data.

Consistent message structures and terminology can improve readability and can additionally support searching, filtering, correlation and automated processing.

The dimension *how to log* therefore covers both the technical implementation of logging and the way in which the resulting information is represented.

2.3.4 Guidelines for Log Levels and Verbosity

Logging guidelines may also address log-level selection and how much information to generate.

Selecting appropriate log levels is an established logging decision and has been addressed by both recommendation systems and automated logging approaches [16,

18]. Other approaches investigate dynamically adapting logging levels according to system behaviour [17].

An unsuitable severity assignment can reduce the usefulness of generated log data. Important failures can receive insufficient attention if assigned an inappropriate level, whereas misuse of high-severity levels can reduce the ability to distinguish critical events.

Logging verbosity describes the overall amount and detail of generated output. Higher verbosity can increase the amount of information available for diagnosis, but also increases the volume and cost of logging [14].

Guidelines concerning log levels and verbosity therefore aim to provide an appropriate level of information for the intended purpose and operational context.

2.4 Log Quality and Its Impact on Analysis

The usefulness of logging depends not only on the volume of generated log messages, but also on the quality and relevance of the recorded information. Decisions made when logging statements are implemented influence how easily the resulting logs can later be understood, searched, compared and processed. This is particularly important because the information available during debugging or failure investigation is limited to what the program recorded during execution [5].

This section therefore considers two related aspects. First, it discusses characteristics that influence the quality and consistency of log messages. Second, it explains how decisions made during logging implementation can affect later log analysis.

2.4.1 Log Quality and Consistency

Useful log messages should contain sufficient information to clearly understand the recorded event. Important aspects include the completeness of the recorded information, the clarity of the message and the consistent use of terminology, identifiers, formats and severity levels.

Incomplete messages make failure investigation harder when important context is missing. For example, a message that only records that an operation failed is of limited use if it does not identify the affected operation, request, component or error condition.

The wording of log messages is also relevant. Unclear descriptions or inconsistent terminology can make it harder for developers to search for related events or understand a message's meaning. Similar events therefore are represented in a reasonably consistent way across the software system.

Consistency also concerns technical aspects of logging. Messages describing comparable events may use different formats, identifiers or log levels across components. Such differences can make manual inspection more difficult and can also complicate

automated processing. Empirical research on logging maintenance has shown that developers frequently modify logging statements to improve characteristics such as formatting, style and message quality [20].

Research has also identified recurring anti-patterns in logging code, showing that poorly designed logging statements can reduce the usefulness of the information [7]. At the same time, improving log quality does not simply mean recording more information. Large amounts of repetitive or unnecessary output can make relevant events harder to identify. Log quality therefore depends on providing information that is sufficiently clear and complete without producing unnecessary data.

2.4.2 Impact of Logging Decisions on Log Analysis

Earlier logging decisions determine the information available during log analysis. Information such as events, identifiers, system states, or contextual values can only support later investigation if the relevant information was recorded during execution. In such cases, developers may need to reproduce the problem or introduce additional logging before further analysis is possible [5].

How information is represented can also affect later analysis. Consistent message structures and identifiers make it easier to search for related events and to compare information across components. In contrast, inconsistent or highly variable log messages may require further processing before systematic analysis is possible.

This is also relevant for automated log analysis. Many automated approaches first transform raw log messages into a more structured form [6]. One common preprocessing step is log parsing, which identifies recurring message structures and separates constant parts of a message from variable runtime values [1, 2].

For example, the messages

```
User 193 failed login from 192.168.1.20
User 417 failed login from 192.168.1.35
```

share the same underlying event structure even though the user identifiers and IP addresses differ. A parser may represent both messages using a common template such as:

```
User <*> failed login from <*>
```

This allows similar events to be grouped and processed together. More recent approaches also address heterogeneous log sources in which different systems or components use different message structures [19].

The quality of this preprocessing can influence later analysis tasks. For example, previous work has investigated how the choice of log parser affects the performance

of log-based anomaly detection [27].

For this thesis, however, the main relevance of automated log analysis is the connection to logging implementation. Decisions about *what to log*, *where to log*, *how to log* and *log verbosity* determine the information that is available afterwards. These decisions therefore influence both developers who inspect logs manually and tools that process logs automatically.

2.5 Related Work

Previous research has investigated software logging from several perspectives. These include empirical studies of developer logging practices, approaches for assisting logging implementation, research on individual logging decisions, and work examining the use and analysis of generated logs.

A systematic mapping study by Gu et al. provides an overview of research on logging practices in software engineering [12]. More recently, Batoun et al. reviewed research on software logging practices and investigated remaining challenges across instrumentation, storage and log preprocessing [23].

The following subsections focus on the areas most closely related to the objectives of this thesis.

2.5.1 Studies on Developer Logging Practices

Yuan et al. conducted an early empirical investigation of logging practices in open-source software and examined how logging statements are used and modified during software evolution [9].

Chen and Jiang later replicated and extended this type of analysis using Java projects from the Apache Software Foundation. Their results similarly demonstrate that logging code is actively maintained throughout software evolution [20].

Researchers have also studied industrial logging practices. Rong et al. examined whether the information developers intend to capture is actually recorded and identified differences between developers' intentions, concerns, and the logging implemented in practice [24]. Further research has examined how developers' backgrounds and experiences influence their logging practices [28].

Logging practices have additionally been investigated within more specific contexts, including mobile analytics [29] and test code [30].

These studies provide important empirical knowledge concerning how developers implement and maintain logging. The developer study in this thesis complements this research by examining developers' experiences across logging implementation, log analysis, fault localisation, root cause analysis and associated challenges.

2.5.2 Research on Logging Guidelines

Previous work has investigated several decisions for which developers may require guidance.

Fu et al. studied where developers introduce logging statements in industrial software and highlighted the importance of informed decisions regarding both where and what to log [13].

Pecchia et al. investigated event-logging practices in an industrial context and examined the runtime information recorded through logging [26].

Other research has attempted to support logging decisions automatically. Approaches that derive logging patterns from existing software have been proposed to help developers make logging decisions [16]. Subsequent studies have examined logging locations in greater detail and proposed more specific recommendations for placing logging statements [25].

Recent work has also investigated generating complete logging statements. For example, LANCE addresses logging location, logging level and message generation [18]. More recent studies have explored generating logging statements for code blocks [31] and using broader program context to improve logging statement generation [32]. These approaches focus on particular logging decisions or on automatically supporting logging instrumentation and demonstrate continued research interest in this area. However, this thesis has a different objective. Rather than proposing another automated recommendation technique, it identifies logging recommendations in academic literature, uses literature-derived classifications as the initial analytical framework, and constructs a consolidated catalogue from logging guidance documented in open-source repositories. The academic recommendations are subsequently compared with the consolidated open-source catalogue.

2.5.3 Research on Logging Quality and Analysis

Researchers then use the generated logs for a range of analysis activities. Automated log-analysis research includes areas such as parsing, anomaly detection, failure prediction and diagnosis [6].

Log parsing has received particularly extensive attention. Zhu et al. evaluated tools and benchmarks for automated log parsing [2], while Logram uses n -gram dictionaries for efficient parsing [1]. Other approaches, such as UniParser, address heterogeneous log data [19].

Research has also shown that preprocessing decisions can affect downstream analysis. Fu et al. empirically examined the effect of different log parsers on anomaly-detection performance [27].

From the perspective of this thesis, this literature is relevant because the usefulness of later analysis partly depends on earlier logging decisions. Chen and Jiang explicitly emphasise that the effectiveness of log-management and analysis techniques depends strongly on the quality of instrumented logging code [5].

Consequently, logging guidelines can influence not only the process of implementing logging but also the usefulness and analysability of the generated log information.

2.6 Research Gap

Existing research provides substantial knowledge concerning software logging. Empirical studies describe how developers introduce and maintain logging statements, while other research investigates individual logging decisions and automated techniques for supporting log instrumentation [5, 12].

Research has also documented a range of remaining practical challenges related to logging instrumentation, storage, configuration, and log preparation for further analysis [23].

However, logging knowledge comes from different sources. Academic publications contain empirical findings, recommendations and proposed techniques, while practical logging guidance may also be documented directly within software projects, technical documentation, development instructions and other practical sources.

These sources can differ in terminology, scope, level of detail and intended context. In particular, recommendations contained within open-source repositories can be distributed across README files, development documentation, coding instructions, configuration material and source files rather than appearing in dedicated guideline documents.

Within the literature and sources examined in this thesis, this creates an opportunity to identify academic logging recommendations, organise practical open-source logging guidance in a structured, and traceable catalogue, and examine how the recommendations identified in academic research are represented in the collected open-source guidance.

This thesis therefore addresses the research problem from four complementary perspectives.

First, a developer study investigates current practices and challenges associated with logging implementation, log analysis and fault localisation.

Second, the thesis analyses academic literature to identify actionable logging recommendations and literature-derived classifications relevant to the principal logging decisions.

Third, logging guidance documented in open-source repositories is identified, extracted, normalised, and consolidated to construct a structured, traceable catalogue of practical logging recommendations.

Finally, the resulting open-source guideline catalogue is classified and analysed to identify recurring themes. At the same time, the academic recommendation set is compared with the consolidated catalogue to examine how research-based recommendations are represented within the collected practical guidance.

Together, these activities connect developers' reported experiences, academic log-

ging research and practical open-source guidance while preserving their distinct roles within the study.

3 Methodology

The methodology is based on data-driven empirical research to investigate logging practices and challenges and guidelines from different perspectives. It collects and combines data from software developers, academic literature and open-source software repositories. First, logging practices and challenges encountered during software development and maintenance are investigated through a developer study. Second, academic literature is analysed to identify actionable logging recommendations, literature-derived classifications and supporting empirical, conceptual or tool-oriented evidence. Third, logging guidance identified in open-source repositories is extracted, classified and analysed according to recurring themes and characteristics, including what to log, where to log, how to log and log-level verbosity. Finally, the logging catalogue is constructed from the open-source repository guidance and organised by themes into a consolidated set of logging guidelines.

3.1 Research Design

The research design of this thesis investigates software logging from three perspectives: developer practices and challenges, logging guidelines documented in academic literature and logging guidance documented in open-source software repositories.

The study is structured according to four research questions. First, **RQ1** investigates logging practices and challenges encountered by software developers during software development and maintenance. This research question is addressed through a developer survey.

Second, **RQ2** investigates logging guidelines and practices documented in academic literature. Relevant publications are identified, selected and examined for actionable recommendations concerning software logging.

Third, **RQ3** investigates logging guidance and practices documented in open-source software repositories. Relevant repository material is collected, prepared and analysed to identify logging recommendations and construct an open-source guideline catalogue. Publications that provided relevant empirical, conceptual or tool-oriented evidence but did not contain a recommendation were retained as supporting literature but were not treated as independent recommendation targets.

The logging guidelines identified for **RQ2** and **RQ3** remain traceable to their respective source sets. The open-source recommendations are normalised, classified,

consolidated and validated to construct the logging guideline catalogue. The academic recommendations establish the literature-based foundation of the deductive codebook and are subsequently compared with the consolidated catalogue to assess the extent of academic coverage.

Finally, **RQ4** investigates the themes represented within the resulting logging guideline catalogue. A deductive classification is first applied using the predefined logging themes from the literature. The classified guideline content is then examined to identify recurring concepts and additional themes across the collected guidelines.

The developer study used for **RQ1** remains analytically separate from the construction of the guideline catalogue. Relationships between developer practices and challenges and the identified guideline themes are considered later during the discussion of the findings rather than being used to construct the themes for **RQ4**.

An overview of the research design is shown in Figure 3.1. The individual methodological stages are described in the following sections.

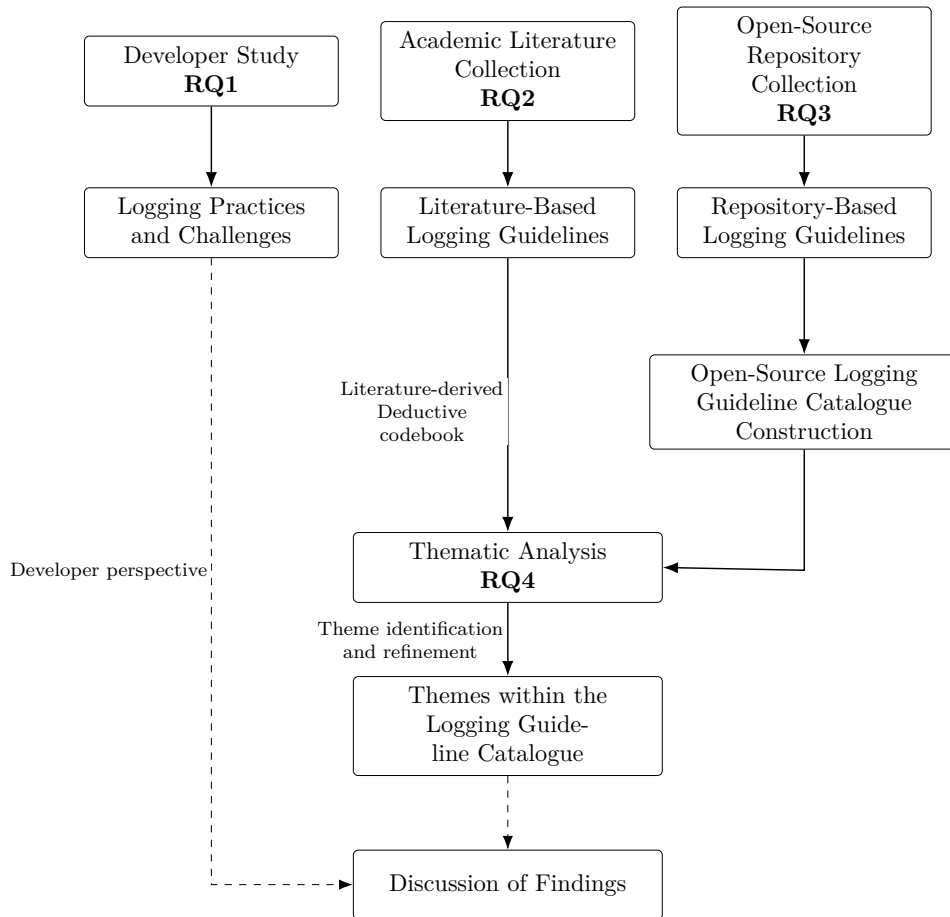


Figure 3.1: Overview of the research design.

3.2 Developer Study

The developer study investigates logging practices and challenges encountered during software development and maintenance. It aims to provide practical insights from industry experience on logging to answer **RQ1**. The study focuses on how developers implement logging, analyse logs and use them for fault localisation. For this purpose, a survey was conducted with software developers at an industry company. The company operates in the aviation field. The following subsections describe the study design, participant recruitment, survey structure, data collection procedure, and analysis of the collected responses in more detail.

3.2.1 Study Design

A survey was designed to collect information about developers' experiences with logging in real-world software projects. The survey includes structured and free-text questions to collect quantitative and qualitative information about logging practices and challenges. Structured questions are used to identify common patterns and frequently reported practices, while free-text questions allow participants to explain their experiences in more detail. This combination provides both comparable responses across participants and gives additional context. The collected responses are used to identify common logging practices and recurring challenges encountered in practical software development.

3.2.2 Participants and Recruitment

The target group of the developer study consisted of software developers with practical experience in software development and maintenance. Participants were recruited from an industry company operating in the aviation field to obtain insights from an industry development environment.

Participation in the survey was voluntary. The questionnaire focused primarily on participants' experience with software logging and the difficulties encountered when implementing, analysing and using logs for fault localisation. Detailed information about individual projects or personally identifying information was not required for the analysis.

The survey dataset contained 26 recorded responses. Since not all participants completed the entire questionnaire, the dataset contained both complete and partial responses. Ten participants reached the final section of the survey, while other participants responded only to earlier parts of the questionnaire.

Partial responses were retained where a valid answer had been provided. Questions that were not answered were excluded from the corresponding analysis and were not interpreted as negative responses. This approach preserved useful information

from partially completed questionnaires while accounting for the number of valid responses across individual survey questions.

3.2.3 Survey and Data Collection

The questionnaire was structured into three main parts covering logging implementation, log analysis and fault localisation. Each part addressed different activities and challenges that developers may encounter when working with logs.

The first part focused on logging implementation. Participants were asked about the amount of development effort spent on logging, the data to log, the reasons for adding log statements and the tools or workarounds that have been used. Further questions addressed how the relevance of logged information is assessed, how log levels are selected, which log formats are used, and what difficulties arise when deciding what, where, and how to log.

The second part focused on log analysis and examined how developers use logs during software development and maintenance. Participants were asked for which purposes logs are analysed, how relevant log entries are identified and which methods or tools are used to access and inspect log data. Additional questions addressed working with unfamiliar logs, like structured, semi-structured and unstructured logs, collaboration with other developers and difficulties that may occur during the analysis process.

The third part investigated fault localisation and the role of logs in identifying the origin of software failures. Participants were asked which additional information is required alongside log data and how important knowledge about the software architecture, project context, development history and technical environment is for locating faults. The questionnaire also considered the effort required for fault localisation, situations in which available log information is insufficient and challenges caused by failures involving several system components.

Different question formats were used throughout the questionnaire, including predefined response options, multiple-selection questions, agreement scales, rankings, percentage estimates, and free-text responses. The structured questions support comparison across participants, whereas free-text responses provide additional context and allow participants to describe experiences that may not be covered by predefined answer options, such as best practices and effective strategies for logging implementation, log analysis and fault localisation for each survey section.

3.2.4 Data Analysis

The survey responses were analysed using quantitative and qualitative methods. Structured questions were evaluated using frequencies, percentages, agreement levels and rankings. For multiple-selection questions, the number of selections for each response option was counted. Agreement-scaled questions were used to identify

general tendencies among the participants, while percentage-based responses were used to estimate the effort required for different logging activities. Because some questionnaires were only partially completed, the number of valid responses was considered separately for each question. Missing answers were excluded from the corresponding analysis and were not treated as negative responses. The free-text responses were lightly paraphrased for readability and conciseness while preserving their original meaning. The paraphrased responses were manually checked against the original answers. No additional thematic coding or grouping was applied. The free-text responses complement the structured responses by providing explanations and observations that cannot be represented sufficiently through predefined answer options. The findings from both forms of analysis are used to answer **RQ1** by identifying commonly reported logging practices and recurring challenges encountered during software development and maintenance.

3.3 Collection of Logging Guidelines from Academic Literature

To answer **RQ2**, academic literature related to software logging was collected and analysed. The primary aim was to identify actionable recommendations for implementation and configuring logging. Relevant publications that provided empirical, conceptual or tool-oriented evidence were also retained where they contributed to understanding logging practices, instrumentation or the analytical framework used in this thesis.

Only identifiable actionable recommendations were treated as recommendation targets for the later coverage comparison. Supporting observations, tool descriptions, evaluation results and other non-prescriptive evidence were retained as literature context but were not treated as independent logging guidelines.

The actionable academic recommendations provide the reference set used for the literature-based classification and the later comparison with the consolidated open-source guideline catalogue.

The following subsections describe the search strategy used to identify relevant publications, the criteria applied to select the literature and the procedure used to identify and extract actionable logging recommendations.

3.3.1 Search Strategy

Google Scholar was the primary platform for identifying relevant academic publications. Several search terms were selected to cover different terminology used in research on software logging. The following search terms were used:

- *Logging Guideline*
- *Logging Practice*
- *Logging Survey*
- *What to Log*
- *Where to Log*
- *How to Log*

Using multiple search terms reduced the dependency on a single terminology, since logging recommendations may be described as guidelines, practices, decisions or recommendations in different publications. The search terms were intended to provide broad coverage of logging research rather than to correspond one to one with the analytical dimensions used later in the study. Therefore, more specific topics such as log levels and verbosity could also be identified through broader terms such as Logging Guideline or Logging Practice.

The initial search focused on publications from 2020 to 2026. Relevant publications identified through the search were additionally examined for references to earlier work on software logging. When a cited publication provided relevant original research on logging practices or guidelines, it was added to the literature set. This backward snowballing process was used to identify publications that were not directly retrieved by the initial Google Scholar searches. Regardless of the publication year, all relevant references identified through backward snowballing were considered for inclusion, because older publications may contain important research that is still relevant to current logging practices.

3.3.2 Selection Criteria

Publications were selected based on their relevance to software logging implementation, logging practices or the subsequent use of generated log data. The literature set therefore contained both publications providing actionable logging recommendations and publications providing relevant empirical, conceptual or tool-oriented evidence.

Publications containing actionable guidance were particularly relevant when they addressed at least one of the four logging aspects considered in this thesis: what information should be logged, where logging should occur, how logging should be implemented or represented, or how log levels and verbosity should be selected.

Empirical studies were also retained when they provided relevant evidence about logging practices, instrumentation decisions or recurring logging problems. Similarly, publications describing or evaluating logging tools, automated instrumentation approaches, log parsers or related techniques could be retained as supporting literature when they contributed to the conceptual background or interpretation of software logging.

A distinction was subsequently made between inclusion in the literature set and inclusion in the academic recommendation set. A publication was treated as a source of an actionable recommendation only when an identifiable statement prescribed, recommended, or clearly demonstrated an action concerning the creation, configuration or implementation of logging. Descriptive observations, tool descriptions, evaluation results, model-performance measures and empirical findings without a corresponding recommendation were not treated as independent coverage targets. Publications were excluded when their relationship to logging was incidental and they did not provide relevant evidence concerning logging practices, instrumentation, log quality or the analytical context of the study. Duplicate publications were also excluded.

3.3.3 Guideline Extraction

The selected publications were examined for statements containing recommendations or guidelines concerning software logging. A statement was considered a logging guideline when it explicitly described an action, recommendation or practice concerning the creation or configuration of log information.

Each identified guideline was extracted while preserving a reference to its original publication. When a publication contained several distinct recommendations, these were manually assigned to the corresponding logging category and recorded together in the catalogue for each publication. The original context and source information were retained so that every guideline could be traced back to the publication from which it originated.

The extracted recommendations were manually summarised rather than copied word for word. The purpose of this summarisation was to create concise and comparable guideline statements while preserving the meaning of the original recommendation. No additional recommendation, restriction or opposite recommendation was introduced during this process.

At this stage, semantically similar recommendations identified in different publications were retained as separate source-level observations. This preserved information about the independent occurrence of similar logging recommendations across academic sources. The literature-derived recommendations were retained as a separate reference set for construction of the initial deductive codebook and for the later academic coverage comparison described in Section 3.5.

Each extracted guideline retained its original publication reference and was prepared for subsequent use according to the main logging aspects *what to log*, *where to log*, *how to log*, and *log levels and verbosity*.

3.4 Collection of Logging Guidance from Open-Source Repositories

To answer **RQ3**, logging guidelines were collected from open-source software repositories hosted on GitHub. Unlike academic literature, open-source projects do not necessarily document logging guidance in dedicated guideline documents. Relevant recommendations may instead appear across different repository artefacts, such as README files, contribution guidelines, project documentation, source files or configuration files. Therefore, a multi-stage repository collection process was applied to identify, prepare and analyse relevant repository content. The collected material was subsequently used to extract logging guidelines while preserving their connection to the source.

The following subsections describe the repository search strategy, repository selection, collected repository characteristics and the procedure used to identify and extract logging guidelines.

3.4.1 Repository Search Strategy

The repository search aimed to achieve broad coverage of logging guidance in open-source software projects. Since logging recommendations may be expressed using different terminology, several complementary GitHub search queries were used instead of relying on a single search term.

The initial query set was developed with the assistance of a large language model (GPT-5.4). GPT-5.4 was used to suggest more precise terminology and combinations of search terms related to different aspects of software logging for more precise results. The proposed queries were subsequently reviewed, adapted and selected before being applied to GitHub. Therefore, GPT-5.4 supported the formulation of candidate search queries, while the final search strategy and selection of queries remained part of the research process.

The final query set covered explicit logging guidelines as well as related topics such as log content, structured logging, log levels, sensitive information, observability and logging anti-patterns. Table 3.1 summarises the queries used during the repository search.

ID	Focus	Search Query
Q1	General logging guidelines	("logging guidelines" OR "logging best practices" OR "logging standards" OR "logging conventions" OR "logging policy") (path:/docs/ OR path:/.github/ OR filename:README.md OR path:**/CONTRIBUTING.md)
Q2	What and when to log	("what to log" OR "log levels" OR "when to log" OR "structured logging" OR "sensitive information") ("logging" OR "application logs") (path:/docs/ OR path:**/README.md)
Q3	Engineering standards and conventions	(logging observability tracing telemetry) ("engineering standards" OR handbook OR conventions OR practices OR guidelines)
Q4	Structured logging	("structured logging" OR "correlation id" OR "trace id") ("best practices" OR guidelines OR standards)
Q5	Security and sensitive information	(logging OR logs) ("sensitive data" OR PII OR GDPR OR credentials) ("guidelines" OR "best practices")
Q6	Log levels and severity	("log level" OR ERROR OR WARN OR INFO OR DEBUG) ("logging guidelines" OR "logging conventions")
Q7	Repository popularity and programming language	("logging guidelines" OR "logging best practices") OR ("logging guidelines") language:Java OR ("structured logging") language:Go
Q8	Markdown-based documentation	(logging OR observability) path:*.md (guidelines OR practices OR standards)
Q9	Observability, telemetry and diagnostics	(observability OR telemetry OR diagnostics OR tracing) (logging guidelines OR standards OR practices)
Q10	Logging anti-patterns	("logging smells" OR "bad logging practices" OR "logging anti-patterns")

Table 3.1: GitHub search queries used for repository collection.

The queries were intended to complement each other by covering different terminology and forms of logging guidance. Specific search operators were used to focus on repositories and file types in which guidelines are commonly documented.

The results returned by the individual queries were combined into an initial search dataset. Since the same repository or file could be returned by several queries, duplicate handling and further preparation of the retrieved repositories were performed

in the subsequent repository selection step.

3.4.2 Repository Retrieval

After defining the search queries, an initial attempt was made to retrieve the search results through the GitHub API. However, some of the predefined queries could not be executed in their intended form because of restrictions affecting long and complex search expressions containing several logical operators.

Simplifying the queries would have changed the previously defined search strategy. Therefore, the original query expressions were retained and executed through the GitHub search interface instead.

The searches were executed on 8 June 2026. For each query, the first 10 search-result pages were collected, corresponding to a maximum of approximately 100 returned results per query. This boundary also reflected the GitHub search interface's limitations during data collection.

The returned search-result pages were collected and processed using a Python script. The script extracted the available repository names, file references and source URLs from the search-result pages and stored these records for further processing. This procedure was repeated for each of the defined search queries.

The retrieved results from all queries were subsequently combined into an initial dataset. At this stage, the complete set of retrieved records within the defined search boundary was retained without removing duplicate repositories or files. Cleaning and exclusion were performed only after the initial repository selection and collection of repository characteristics.

The use of a fixed number of search-result pages means that the collected dataset represents the results available within the defined search boundary rather than all repositories available on GitHub. This limitation is considered further in the discussion of threats to validity.

3.4.3 Repository Selection

The results retrieved through the different search queries formed the initial repository selection. The purpose of this stage was to preserve the complete set of potentially relevant repositories and files identified through the predefined search strategy. A repository was retained in the initial selection when the corresponding search result contained potentially relevant logging-related material. No minimum threshold based on the number of stars, commits, contributors or repository activity was applied at this stage. Even repository creation date was considered later during the selection process. This allowed both large and smaller open-source projects to be considered, since relevant logging guidance may also occur in projects with lower levels of development activity.

Results returned by several different search queries were not removed at this point.

Similarly, repositories with overlapping characteristics or common authors were initially retained. This ensured that metadata could first be collected for the complete search result set before exclusion and duplicate handling were performed.

The resulting initial selection therefore represents the complete repository dataset obtained from the defined GitHub searches before data cleaning.

3.4.4 Repository Characteristics

Repository metadata were collected for the initial selection before duplicate handling and other exclusion steps. The purpose of collecting these characteristics was to provide contextual information about the projects in which potentially relevant logging guidance occurred and to support later descriptive analysis of the repository dataset.

The collected information included, where available:

- repository and source reference,
- programming language,
- project type,
- application field or domain,
- number of commits,
- number of contributors or developers,
- repository activity,
- repository owner or author information.

Repository information directly available through GitHub, such as repository reference, programming language, owner information, activity, commits, and contributors were obtained from the corresponding repository information where available.

Project type and application field were determined from information available in the repository documentation and description. Where AI-assisted (GPT-5.6 Sol) classification was used to support the assignment of these characteristics, the assigned values were reviewed against the corresponding repository information before being retained.

Programming language, project type and application field were collected to describe the technical and application context in which the logging guidance occurred. These characteristics also support later descriptive analysis of whether particular guidelines or themes occur across different types of projects or are concentrated within particular parts of the collected dataset.

The number of commits and contributors was interpreted as contextual information about development activity and community involvement rather than as a measure

of repository or guideline quality. Repositories were therefore not excluded because of few commits, contributors or other popularity indicators.

Collecting the metadata before data cleaning additionally supported the comparison of retrieved records and the identification of duplicate or overlapping repository entries during the subsequent cleaning procedure.

3.4.5 Data Cleaning and Exclusion

After the initial repository selection and metadata collection, the dataset was cleaned before guideline extraction.

Since the same repository or file could be returned by several search queries, duplicate search-result entries were identified and removed. Multiple occurrences representing the same underlying repository or source material were consolidated into a single entry.

Repository owner and author information was additionally considered when identifying overlapping results. However, repositories were not excluded because of the same author or owner. Separate repositories were retained when projects or relevant material are represented. Entries were consolidated only when duplicate or identical source material are referenced.

Forked repositories were also examined during the cleaning process. Where a fork contained the same relevant material as the corresponding original repository, the original repository was retained, and the duplicated fork was excluded. When GitHub search results displayed both an original repository and its forks, preference was therefore given to the original repository to retain the source material closest to its original project context.

Additional duplicate and similarity detection was performed on the retrieved files using cosine similarity. The similarity scores were used to identify files with identical or highly similar textual content and to support the comparison of potential duplicate material. A cosine similarity value of 1.0 indicated identical representations of the compared file content. Files with almost identical content, with a similarity value of around 0.9, were manually examined and duplicate copies were removed before guideline extraction.

Files with lower similarity values were not automatically removed based on their similarity score. Instead, similarity information was used to distinguish between identical source material and files that contained related but potentially different content. This prevented independently documented logging guidance from being removed simply because its wording or structure was similar to another file.

Repositories or referenced files that were no longer accessible on GitHub at the time of data preparation were excluded. Information about excluded and unavailable repositories was retained to keep the preparation procedure traceable.

These cleaning steps concerned duplicate or overlapping source material within the repository collection. Similar logging recommendations occurring independently in different repositories were not removed at this stage. Such occurrences were retained

as independent source-level evidence and were considered separately during the later construction and semantic consolidation of the logging guideline catalogue.

After completion of the cleaning procedure, all remaining repositories and files containing potentially relevant logging-related material were prepared for guideline identification and extraction.

3.4.6 Relevant Logging Content Identification and Extraction

After data cleaning, the remaining relevant repository files were downloaded and stored locally. The files were retained in their original file formats whenever possible to preserve the source material and maintain traceability to the repository and file from which the content originated.

The collected files were analysed for logging recommendations concerning four pre-defined logging dimensions:

- *what to log,*
- *where to log,*
- *how to log,*
- *log levels and verbosity.*

These four dimensions were used as high-level categories for identifying relevant logging content within the repository files. At this stage, the purpose of the classification was to locate potentially relevant source passages rather than to assign the more detailed guideline codes introduced later during catalogue construction.

A passage was considered relevant when it contained an identifiable recommendation, instruction or practice concerning the implementation or configuration of software logging. This included both positive recommendations describing practices that should be followed and negative recommendations describing practices that should be avoided. Purely descriptive statements that mentioned logging without providing identifiable guidance were not classified as logging recommendations.

An AI-assisted (GPT-5.6 Sol) procedure was used to identify relevant logging content within the collected repository files. Instead of asking the model to rewrite or summarise the source material, the model was instructed to preserve the original content and add XML tags around passages containing identifiable logging guidance. The following XML categories were used:

- `<what_to_log>`,
- `<where_to_log>`,
- `<how_to_log>`,
- `<log_verbosity>`.

The original wording and structure of the source files were preserved as far as possible. The model was instructed to add classification tags only around source content containing identifiable logging guidance and not to generate, complement, summarise or infer recommendations that were not explicitly supported by the underlying material.

Where a passage addressed more than one logging aspect, multiple corresponding classifications could be retained. For example, a recommendation could specify both which information should be recorded and how that information should be represented. In such cases, the passage was not forced into only one of the four high-level dimensions.

Files for which no identifiable logging guidance was detected were retained without additional XML classification tags. This prevented the procedure from requiring every retrieved file to produce a guideline and reduced the risk of forcing unrelated or purely descriptive content into one of the predefined categories.

The tagging procedure was applied automatically to the collected files using OpenAI GPT-5.6 Sol through the API. A predefined prompt was used consistently across the processed dataset. The prompt defined what was considered relevant logging guidance, specified the four classification categories and instructed the model to preserve the original file content while adding XML tags only around passages containing actionable logging recommendations or practices.

The prompt used for this procedure was:

I am providing you with a file that may contain logging guidelines. Analyse the entire file and return the complete file with XML classification tags added around passages that contain actual logging guidelines. A logging guideline is a prescriptive or recommended statement that tells developers what they should, should not, must, may, or are expected to do when implementing or configuring logging. This can also include clearly stated best practices or recommendations expressed through examples. Do not classify text merely because it discusses logging. Only tag passages that contain actionable guidance, recommendations, requirements, or explicitly demonstrated logging practices. Classify each identified guideline using one or more of the following categories:

- * What to log <what_to_log>...</what_to_log>
- * Where to log <where_to_log>...</where_to_log>
- * How to log <how_to_log>...</how_to_log>
- * Log verbosity <log_verbosity>...</log_verbosity>

Important:

- * Keep the original wording exactly and dont change it at all.
- * Only add tags if possible, else return file without any tags

Prompt 1: Prompt for logging content identification and XML classification

After the AI-assisted (GPT-5.6 Sol) tagging procedure, the generated files were checked for the expected XML structure before their content was extracted. This technical integrity check examined whether the expected opening and closing tags were present and whether the generated output could be processed correctly by the subsequent extraction script. Outputs containing malformed, incomplete or inconsistent XML structures were identified and reprocessed or manually reviewed before being included in the extraction stage.

The technical XML check was distinguished from the validation of the content classification itself. The accuracy of the AI-assisted (GPT-5.6 Sol) identification of relevant logging passages was evaluated separately through manual validation as described in Section 3.5.8. The validation included both files containing identified logging guidance and files for which no guideline had been identified. Including files without assigned tags made it possible to examine whether relevant recommendations had potentially been missed during the automated identification process.

After the tagging procedure had been completed, the XML-marked passages were extracted automatically using a Python script and transferred into a structured open-source logging-content corpus. The extraction procedure identified the content contained within the four predefined XML categories and stored the corresponding passages in a structured representation.

Multiple distinct passages originating from the same source file were retained as separate entries where applicable. Similarly, comparable recommendations occurring independently in different repositories were retained separately rather than being combined during this stage.

Each extracted passage remained linked to its original repository and source file. The assigned high-level logging category was also retained to support the subsequent guideline formulation and classification.

The resulting dataset therefore preserved the relationship between the extracted logging content, its classification and its original repository context. This traceability enabled the extracted passages to be compared with their source material during subsequent normalisation and validation.

At this stage, the extracted passages represented source-level logging content and were not yet treated as normalised logging guidelines. No semantic consolidation of similar recommendations was performed. Consequently, similar recommendations documented independently in different repositories remained separate source-level observations.

The subsequent transformation of the extracted source content into concise normalised guideline statements, the manual review of these statements, their comparison with the academic reference set, detailed classification and semantic consolidation are described in Section 3.5.

3.5 Logging Guideline Catalogue Construction and Classification

The preceding sections describe the collection of logging guidelines from academic literature and the identification of relevant logging content within open-source repositories. While the academic literature already provided explicitly formulated logging recommendations, the open-source material differed considerably in wording, structure and level of detail. A separate catalogue construction process was therefore applied to transform the collected material into a consistent and analysable set of logging guidelines.

The objective of this stage was to create a traceable open-source guideline catalogue while preserving the relationship between each guideline and its source. The process consisted of contextual content tagging, guideline formulation and normalisation, manual review, classification using an initial literature-based codebook, inductive extension of the codebook, semantic consolidation and final validation. The retained academic recommendations were used to derive the deductive starting structure and were compared with the consolidated catalogue after catalogue construction.

The resulting catalogue forms the basis for the thematic analysis described in Section 3.6.

3.5.1 Contextual Content Tagging

Following extraction of the XML-classified source passages, an additional contextual tagging step was applied before the passages were transformed into normalised logging guidelines. The purpose of this step was to retain a short description of the main topic or topics represented by each extracted passage.

For each extracted entry, the source passage was retained in the *Content* field. An AI-assisted procedure was then used to create a corresponding *Content Tag*. The Content Tag was used as contextual metadata to support later guideline formulation and validation rather than as the final thematic classification of the guideline.

The prompt used for this procedure was:

I am providing an Excel file containing extracted logging content.
Please analyse the Content column and add a new column called Content Tag.
For each row, create a short summary of the main topic
or topics discussed in the content.

Prompt 2: Prompt for contextual content tagging

The original Content remained available alongside the Content Tag so that the technical context could be checked when a short tag was ambiguous. During manual in-

vestigation, contextual expressions that could lead to misleading interpretations were checked against the source passage and corrected where necessary.

3.5.2 Guideline Formulation and Normalization

The logging content extracted from the open-source repository files was initially retained in its original wording. Since the extracted passages differed in length, writing style and level of detail, they were transformed into guideline statements before further analysis.

An AI-assisted (GPT-5.6 Sol) procedure supported this normalisation step. The original Content and its related Content Tag were analysed together to identify the logging recommendation contained in the source, and turn it into a clear and independently understandable logging guideline. The aim was to express recommendations that consistently appeared in different forms so that they could be compared within the catalogue. No additional logging advice was added, and the final guideline was based only on the information available in the original context.

Guidelines could also be derived from clearly demonstrated logging practices in code, configuration, examples or tables where the source material sufficiently supported the recommendation. Information that was not supported by the source was not added. The normalised guideline was formulated so that it could be understood independently without referring back to the Content field.

Where no meaningful logging guideline could be derived from the extracted material, the entry was assigned NO_GUIDELINE. Such entries were retained separately and were not assigned themes during the subsequent classification. These entries were also manually reviewed to confirm that no relevant logging guideline had been overlooked.

The original Content and Content Tag were retained alongside the normalised guideline. Consequently, every transformed guideline could subsequently be compared with the source material from which it had been derived.

The prompt used for the procedure was:

Analyse the Content column and content tag, and add a new Guideline column. Transform each row into a clear, concise, self-contained and stand alone as logging guideline, where content is not reference in the guideline. Guidelines may also be derived from clearly demonstrated practices in code, configuration, examples, or tables. Preserve the source meaning and do not add unsupported information. Use NO_GUIDELINE only when no meaningful guideline can be derived or content tag include no guideline. Add all NO_GUIDELINE rows to a separate worksheet.

Prompt 3: Prompt for guideline formulation and normalization

3.5.3 Manual Review of Normalised Guidelines

Following normalisation, the generated guideline statements remained linked to their corresponding source passages and Content Tags so that the transformation could be manually validated.

A random sample of approximately 10% of the normalised entries was manually compared with the corresponding Content and Content Tag. The review considered whether the normalised guideline represented the recommendation contained in the source, whether relevant restrictions or conditions had been preserved, whether the guideline was sufficiently self-contained and whether additional unsupported meaning had been introduced during normalisation.

Additional manual investigation was performed for entries identified during the processing procedure as unclear or inconsistent. Where recurring problems were identified, related entries were checked and corrected rather than changing only the individual example in which the problem had first been observed.

Entries classified as `NO_GUIDELINE` were also checked to ensure that no meaningful recommendation was excluded only because of the automated transformation.

After this review, the resulting entries formed the source-level open-source guideline catalogue used to address **RQ3**. Each guideline remained linked to its original passage, repository and source file during the repository collection process.

3.5.4 Relationship Between Academic and Open-Source Guidelines

The academic and open-source guideline sets were retained as analytically distinguishable source sets. The open-source recommendations formed the source-level catalogue that was normalised, thematically classified and semantically consolidated. The academic recommendations were retained with their publication references as the literature-based reference set, while descriptive literature remained available as supporting evidence.

The academic literature contributed to the catalogue analysis in two ways. First, classifications identified in previous logging research were used to construct the initial deductive codebook described in the following subsection. Second, after semantic consolidation of the open-source recommendations, the retained academic recommendations were manually compared with the resulting canonical guidelines to determine whether each academic recommendation was represented in the final consolidated open-source catalogue.

The comparison was performed at the consolidated guideline level rather than separately for every repeated source occurrence. This prevented a recommendation documented in several open-source sources from being counted repeatedly when evaluating its relationship to the academic recommendations.

The academic and open-source provenance remained retained throughout this comparison. The methodological procedure for the comparison is described later in the

thematic analysis, while the actual academic coverage identified for the final consolidated guidelines is reported in the results chapter.

3.5.5 Initial Literature-Based Codebook

An initial deductive codebook was constructed from classifications and concepts identified in previous software logging research. The purpose of this codebook was to provide an academically grounded starting structure for the classification of the collected guidelines rather than to define an exhaustive set of possible themes. The four high-level categories used during the earlier extraction process were retained:

- *what to log,*
- *where to log,*
- *how to log,*
- *log levels and verbosity.*

For *what to log*, the initial codes were based on the classification reported in *Industry Practices and Event Logging: Assessment of a Critical Software Development Process* [26]. The following codes were used:

- *state dump,*
- *execution tracing,*
- *event reporting.*

For *where to log*, the initial codes were derived from *Where Do Developers Log? An Empirical Study on Logging Practices in Industry* [13]. The following classifications were included:

- *assertion-check logging,*
- *return-value-check logging,*
- *exception logging,*
- *logic-branch logging,*
- *observing-point logging.*

For *how to log*, the initial classification was based on *A Survey of Software Log Instrumentation* [5]. The survey distinguishes three main stages of log instrumentation:

- *logging approach*,
- *logging utility integration*,
- *logging code composition*.

For *log levels and verbosity*, the initial codes were based on the verbosity levels discussed by Gholamian and Ward [14]. The paper refers to the six default Log4j verbosity levels:

- *FATAL*,
- *ERROR*,
- *WARN*,
- *INFO*,
- *DEBUG*,
- *TRACE*.

The resulting initial codebook therefore consisted of literature-derived codes within each of the four predefined logging categories. Guidelines that could not be meaningfully represented by one of these codes were not forced into the existing classification. Instead, they were marked for subsequent investigation.

The codebook was based on classifications identified in the academic literature. For each code, the corresponding literature was used to determine its meaning and to guide its application during classification.

3.5.6 Inductive Codebook Extension and Final Classification

The literature-derived codes provided the initial classification structure for the guideline catalogue. However, not all guidelines could be represented adequately by these initial codes. Additional themes were therefore introduced based on concepts identified within the collected guideline content.

The normalised Guideline column was analysed using the literature-derived codebook first. The four high-level logging categories remained *what to log*, *where to log*, *how to log* and *log verbosity*. Where a guideline could be represented by one of the existing deductive themes, the corresponding theme was assigned. If both a broad and a more specific deductive theme were applicable, the most specific applicable theme was selected.

Where none of the existing deductive themes adequately represented the guideline, a concise additional theme was derived inductively from the guideline content. For each assignment, the origin of the theme was retained in the field *Theme Basis* as either *Deductive* or *Inductive*. Guidelines assigned an inductive theme were additionally collected in a separate review worksheet so that the newly derived themes could be inspected and later harmonised.

Entries classified as NO_GUIDELINE were not assigned a Theme or Theme Basis. The prompt used for this classification was:

```
Analyse the Guideline column and add Theme and Theme Basis and Leave
out for NO_GUIDELINES
Use this codebook first:
What to log: State dump, Execution tracing, Event reporting
Where to log: Assertion-check logging, Return-value-check logging,
Exception logging, Logic-branch logging, Observing-point logging
How to log: Logging approach, Logging utility integration,
Logging code composition
Log verbosity: FATAL, ERROR, WARN, INFO, DEBUG, TRACE
Check all deductive themes first, then if a guideline fits both a broad
and a more specific deductive theme, assign the most specific applicable theme,
else when no existing theme fits, create a concise
new inductive theme from the guideline.
Set Theme Basis to Deductive or Inductive.
Put all inductively themed rows in a separate worksheet called Inductive Review.
Leave the NO_GUIDELINES empty for Theme and Theme Basis
```

Prompt 4: Prompt for deductive-inductive theme classification

The theme assignment was based on the semantic meaning of the normalised guideline. Each guideline was assigned to one final high-level logging dimension and one corresponding theme. Where a guideline or its underlying source content had previously been associated with more than one dimension, the original Content was consulted, and the guideline was assigned to the dimension that most specifically represented its primary recommendation. The original Content, Content Tag and source-level classifications remained available for contextual interpretation and traceability. The Content Tag itself was not automatically treated as a Theme. After the initial classification, inductively generated themes were reviewed across the catalogue. Overlapping inductive themes representing the same concept were merged, and their names standardised, while distinct concepts were retained separately. The existing literature-derived themes retained their deductive basis. The resulting classification therefore combined literature-derived deductive themes with additional themes derived from the collected guideline content. The analytical structure consisted of the high-level logging dimension, the assigned theme and the individual guideline. Guidelines assigned to the same theme were not assumed to be semantically equivalent, since the guideline itself retained the specific recommendation and context.

3.5.7 Guideline Consolidation and Traceability

Following classification, the source-level catalogue was examined for semantically equivalent guideline statements. This step was separate from the removal of dupli-

cate repository material performed during data cleaning. Recommendations independently documented in different sources were treated as separate source occurrences even when their meaning was equivalent.

Guidelines were considered candidates for consolidation when they described the same underlying normative recommendation. Equivalence was assessed by considering the recommended action, the logging concern to which the action referred and any conditions or restrictions attached to the recommendation. Guidelines addressing a similar topic but containing a distinct recommendation, condition or restriction were retained separately. Uncertain cases were also kept separate.

When semantically equivalent source-level guidelines occurred under different dimensions or themes, their original Content was compared before consolidation to determine whether the apparent equivalence resulted from normalisation or represented the same underlying recommendation. Where consolidation was appropriate, the resulting canonical guideline was assigned to the single dimension and theme that most specifically represented its primary recommendation. The original source-level dimension and theme assignments were retained in the consolidation mapping for traceability. Where the underlying recommendations were meaningfully different, or the equivalence remained uncertain, the guidelines were retained separately.

Due to the number of guidelines contained in the catalogue, AI-assisted semantic comparison using GPT-5.6 Sol was used to support the identification and consolidation of semantically equivalent guidelines. The model was instructed not to remove unique recommendations or conditions and to preserve the mapping from every original guideline to the corresponding consolidated guideline.

A separate code-review indicator was used to identify source Content containing technical material that could later be displayed as an illustrative example for a guideline. This included source code, configuration, use-case or implementation examples, example input and program output. In the working catalogue, this information was retained through the *Code Involved* field. The term *Code Review* in the consolidation prompt referred to this example-identification step and did not indicate a methodological validation status.

For entries identified in this way, the associated source Content was preserved as a separate *Example Reason* linked to the consolidated guideline. These examples were not merged, removed or generalised during consolidation, even when their underlying guideline was consolidated.

The prompt used for the consolidation procedure was:

Merge semantically equivalent guidelines without losing any unique recommendation or condition. Keep uncertain cases separate.
 For every row with Code Review = Yes, preserve its Content as a separate Example Reason linked to the consolidated guideline and do not merge, remove, or generalise these examples during consolidation.
 Create consolidated Guidelines and consolidation mapping
 Then update the codebook: keep existing deductive themes as Deductive, merge overlapping inductive themes, standardise names, and keep distinct concepts separate.
 create updated codebook and theme mapping
 Ensure every original guideline is represented exactly once and summarise identical guidelines together. Provide a sheet with all summarized guidelines, their content, type, theme for manual investigation later

Prompt 5: Prompt for guideline consolidation and codebook refinement

Guidelines confirmed as semantically equivalent were mapped to a common canonical guideline. The original source-level guideline records were retained to preserve traceability. Each original guideline was required to be represented exactly once in the consolidation mapping.

The consolidation stage also updated the codebook and theme mapping. Existing deductive themes remained deductive. Overlapping inductive themes were merged and standardised where they represented the same underlying concept, while distinct concepts were kept separate.

This approach reduced repetition in the final presentation of the catalogue while preserving provenance, contextual examples and the independent source occurrences associated with each consolidated recommendation. Exact duplicates originating from repeated retrieval of the same underlying source remained handled by the earlier data-cleaning stage and were not counted as independent source occurrences.

3.5.8 Validation of the Guideline Catalogue

The AI-assisted (GPT-5.6 Sol) catalogue construction involved several processing stages at which identification, transformation or classification errors could occur. Validation was therefore performed at multiple stages of the catalogue construction process.

First, the XML-tagged repository files were checked for technical integrity. The check examined whether the expected opening and closing tags were present and whether the output could be processed correctly by the extraction script. Malformed, incomplete or inconsistent XML outputs were identified and reprocessed or manually investigated.

The semantic content of the extracted material was additionally reviewed during the subsequent preparation steps. Manual investigation was used to identify cases

in which a contextual expression had been interpreted incorrectly, or the high-level XML category did not adequately represent the source content. Examples included log-verbosity cases, unstructured log-message content, formatting-related recommendations and HTTP-related content. In particular, HTTP-related entries were investigated because occurrences of HTTP could represent hyperlinks or references rather than an HTTP-specific logging recommendation. Where recurring issues were identified, corresponding entries were reviewed and corrected throughout the affected part of the dataset.

Second, the transformation from Content and Content Tag to normalised guidelines was validated manually. A random sample of 234 entries, corresponding to approximately 10% of the dataset at this stage, was compared with the associated source Content and Content Tag. The review examined whether the guideline preserved the source meaning, retained relevant conditions and restrictions, remained self-contained and did not introduce unsupported information. Cases requiring additional context were corrected using the original Content as the reference.

Entries assigned `NO_GUIDELINE` were additionally reviewed to ensure that the classification did not exclude meaningful logging recommendations. Content that only contained references, links or descriptive material without a supported logging recommendation remained excluded from thematic classification.

Third, the final theme classification was validated using the deductive-inductive codebook. Validation was performed across the individual themes rather than relying on one undifferentiated sample from the complete dataset. Larger themes were checked using manual samples, while themes with comparatively few assigned guidelines were inspected more closely because individual classification errors would have a larger proportional influence on their reported distribution.

The guidelines initially assigned to *Logging approach* were re-examined against the complete final codebook and, where necessary, their original Content to determine whether a more specific existing theme represented the recommendation more accurately. This review aimed to determine whether the guidelines fell within the theme's conceptual scope. Guidelines within such a broad theme did not need to be semantically equivalent, since each guideline retained its specific scenario and recommendation. In contrast, the theme served as a higher-level classification.

During this validation, incorrectly assigned log levels, event-reporting cases and logging-approach cases were corrected where identified. When a repeated classification pattern was detected, related entries were investigated rather than correcting only the initial sampled row. The final theme distribution was calculated after these corrections.

Finally, the semantic consolidation and traceability of the catalogue were checked. Consolidated guidelines were reviewed to determine whether semantically equivalent recommendations had been combined without removing a unique condition, restriction or recommendation. Where equivalence remained uncertain, the guidelines were retained separately.

Where manual validation showed that a consolidated group contained source occurrences representing different recommendations, the affected occurrences were sepa-

rated and, where an equivalent canonical guideline already existed elsewhere in the catalogue, reassigned to that guideline rather than creating a duplicate recommendation. A new canonical guideline was retained only when the recommendation was not adequately represented by an existing guideline.

An integrity check was also performed on the consolidation mapping. Every retained source-level guideline was required to be represented exactly once by a consolidated canonical guideline, and every canonical guideline remained linked to the source-level guideline or guidelines from which it had been derived. Example Reasons originating from code, configuration, usage, implementation or output examples remained separately linked to the corresponding consolidated guideline.

The final validation additionally checked that each canonical guideline was assigned to one final high-level logging dimension and corresponding theme. Where consolidation had combined equivalent source occurrences with different earlier classifications, the most specific classification supported by the underlying recommendation was retained as the final assignment, while the original classifications remained traceable in the mapping.

The resulting catalogue therefore preserves the relationship between the source content, contextual Content Tag, normalised source-level guideline, assigned theme and consolidated canonical guideline. This structure supports later manual investigation while preserving the provenance of the final catalogue.

3.6 Thematic Analysis

The final classified and consolidated logging guideline catalogue was used as the basis for answering **RQ4**: *What themes can be identified within the logging guideline catalogue?*

A hybrid deductive–inductive thematic classification was applied. The deductive component originated from the literature-derived initial codebook, while the inductive component allowed additional themes to be introduced for guideline concepts that could not be represented adequately by the initial classification.

The thematic analysis considered both the final themes and the individual guidelines assigned to them. The four predefined logging aspects *what to log*, *where to log*, *how to log*, and *log levels and verbosity* were retained as the high-level analytical dimensions. Within these dimensions, the final deductive and inductive themes represented recurring areas of logging guidance.

The individual normalised, or consolidated guideline remained the more specific unit of actionable knowledge. Consequently, guidelines assigned to one broad theme were not interpreted as identical recommendations. The themes provided an organisational and analytical structure through which recurring logging concerns could be examined across the catalogue.

3.6.1 Theme Identification

Theme identification followed the hybrid deductive–inductive classification procedure described in Section 3.5. Each guideline was first compared with the literature-derived deductive themes. Where an existing deductive theme adequately represented the recommendation, that theme was assigned. Where no existing theme was sufficient, an inductive theme was created based on the semantic meaning of the guideline.

The themes were reviewed iteratively across the catalogue. Overlapping inductive themes representing the same underlying concept were merged, and their names standardised, while themes representing distinct logging concepts remained separate. Existing deductive themes retained their literature-derived basis.

Theme development was based on the meaning of the underlying guidelines rather than only on similarity in terminology. The source passages, Content Tags and normalised guidelines were consulted where necessary to preserve the context of a recommendation during classification.

The resulting structure consisted of the four high-level logging dimensions, the final deductive or inductive Theme assigned within the corresponding dimension, and the specific guideline. This structure allowed the themes to be analysed descriptively while retaining the detailed recommendations contained in the catalogue.

3.6.2 Theme Distribution

After the themes had been identified, their distribution within the guideline catalogue was analysed descriptively. This analysis considered both the number of canonical guidelines associated with each theme and the number of source-level occurrences contributing to those guidelines.

The distinction between canonical guidelines and source occurrences was maintained throughout this analysis. A canonical guideline represents a semantically consolidated recommendation, whereas a source occurrence represents an independent appearance of that recommendation within an open-source repository.

For each identified theme, the following information was considered where applicable:

- number of canonical guidelines,
- number of source-level guideline occurrences,
- number of contributing open-source repositories,
- distribution across the four high-level logging categories.

The number of occurrences was used as a descriptive indication of how frequently a theme appeared within the collected corpus. Frequency was not interpreted as a measure of guideline quality, correctness or importance. A frequently occurring recommendation may indicate that different sources repeatedly address a concern, while a less frequent recommendation may still represent a relevant or specialised logging practice.

Where appropriate, repository metadata such as programming language, project type or application field was used as additional descriptive context. Such comparisons were used to explore whether particular themes occurred across different types of projects or were concentrated within specific parts of the collected dataset.

3.6.3 Coverage of Academic Recommendations

After semantic consolidation, the retained actionable academic recommendation targets were manually compared with the canonical open-source guidelines. The purpose of this comparison was to determine the extent to which recommendations identified in academic literature were represented in the final consolidated open-source catalogue.

The unit of comparison was the individual actionable recommendation rather than the publication as a whole. A publication could therefore contribute several distinct recommendation targets, while publications retained only for empirical, conceptual or tool-oriented context did not contribute a coverage target.

The comparison was performed against the consolidated catalogue rather than separately against each repeated source occurrence. Where equivalent guidance appeared independently in several repositories, the corresponding canonical guideline was used for the coverage assessment, while the source-level occurrences remained available for the separate frequency analysis.

The comparison considered the semantic meaning of the academic recommendation and the corresponding canonical guideline, including its recommended action, relevant conditions, scope and restrictions.

An academic recommendation was classified as fully covered when one or more canonical guidelines expressed the same underlying recommendation while preserving the relevant meaning and conditions.

A recommendation was classified as partially covered when the principal logging concern was represented in the catalogue but one or more relevant aspects, conditions or sub-recommendations were not represented sufficiently.

A recommendation was classified as not identified when no canonical guideline in the collected open-source catalogue sufficiently represented the underlying recommendation. Thematic similarity alone was not considered sufficient for guideline-level coverage.

The retained provenance of both datasets allowed each academic recommendation to remain linked to its publication and each matching recommendation to remain

linked to the corresponding canonical guideline or guidelines. The resulting coverage classification is reported in the Results chapter.

The comparison is limited to the selected academic recommendation set and the collected open-source corpus. A partial or missing match therefore does not demonstrate that a recommendation is absent from software logging research or open-source software in general.

4 Results

This chapter presents the results of the developer study, the academic literature analysis, the open-source repository analysis and the thematic classification, validation and consolidation of the logging guideline catalogue. The results are presented according to the four research questions and are discussed in more detail in Chapter 5.

4.1 Developer Study Results

This section addresses RQ1: *What logging practices and challenges do software developers encounter in software development and maintenance?* The results cover developers' experiences with logging during implementation, log analysis and fault localisation. Quantitative survey responses are presented together with the practices and challenges reported in the free-text responses.

4.1.1 Survey Response Overview

The survey dataset contained 26 recorded responses, including both complete and partial questionnaires. Ten participants reached the final section of the questionnaire. For the analyses presented in this chapter, missing responses were excluded question by question rather than interpreted as negative answers. Therefore, the number of responses included in the analysis differs by question and is specified where necessary.

The three parts of the developer study examined logging implementation, log analysis and fault localisation. Eight respondents provided usable answers for the logging implementation section, seven respondents answered the log-analysis section and six respondents answered the fault-localisation section.

Selected results are presented in the following subsections. Additional survey figures, detailed response distributions and the paraphrased open-ended responses are provided in Appendix A.0.1, Appendix A.0.2 and Appendix A.0.3.

4.1.2 Logging Implementation Practices

Eight respondents provided usable answers for the logging implementation section. The reported share of overall development time spent implementing logging ranged from 3% to 90%, with a median of 10% (Figure 4.1).

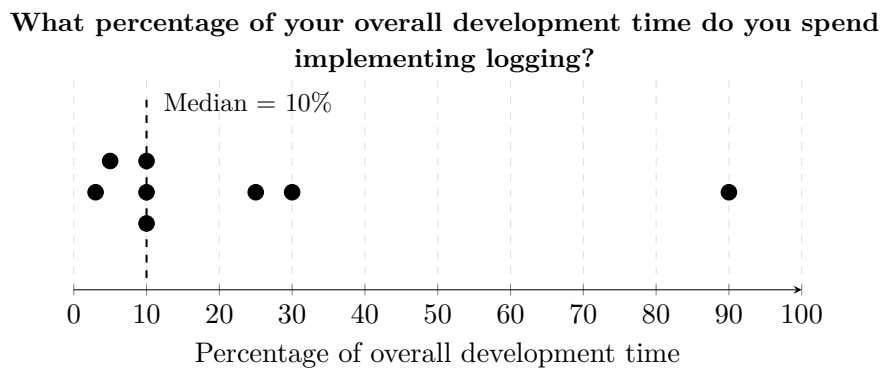


Figure 4.1: Percentage of overall development time spent implementing logging among respondents who implement logging.

The implementation questions examined what developers log, which methods and tools they use, how they determine the criticality of information and which logging decisions they perceive as difficult. Because multiple selections were possible for several questions, the percentages in the following figures do not sum to 100%.

Regarding the type of information recorded, event reporting was the most frequently represented category, selected by 75% of respondents (6/8). State-dump information was reported by 62.5% (5/8), while execution-tracing information was reported by 50% (4/8), as shown in Figure 4.2.

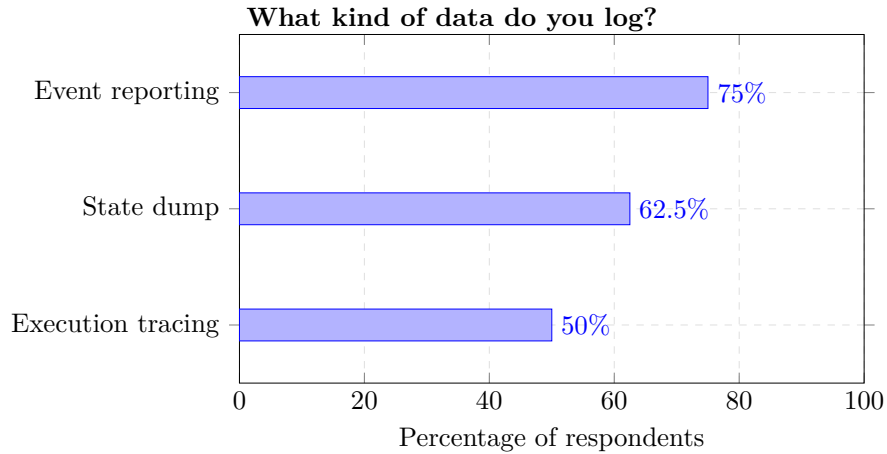


Figure 4.2: Types of information logged according to the literature-derived *What to log* codebook. Multiple categories could be assigned to one response.

Print statements were the most frequently reported logging method, used by 75% of respondents (6/8), followed by off-the-shelf logging frameworks at 62.5% (5/8). In-house logging tools, external logging services and self-developed logging solutions were each reported by 37.5% (3/8) (Figure 4.3).

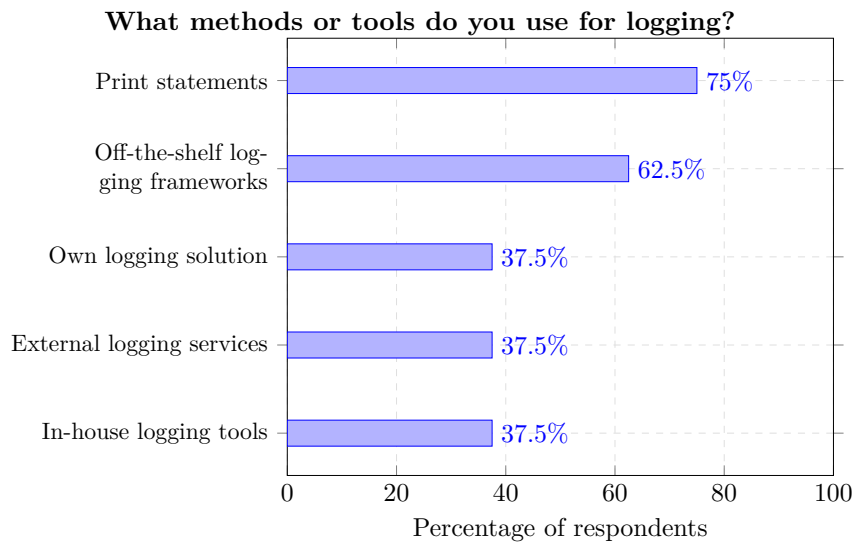


Figure 4.3: Methods and tools used for logging. Multiple selections were possible.

The most frequently selected criteria for determining the criticality of information to be logged were potential performance impact and user feedback or incident reports,

both selected by 62.5% of respondents. The affected system part, the frequency of similar events, and potential security impact were each selected by 50%. Compliance or regulatory requirements were selected by 37.5%. Historical trends, team-lead or architect recommendations, current system conditions and predefined guidelines or standards were each selected by 25%. In contrast, automated tools or algorithms were selected by 12.5% (Figure 4.4).

Based on which criteria do you determine how critical the information to log is?



Figure 4.4: Criteria used to determine the criticality of information to be logged. Multiple selections were possible.

The difficulty statements show that deciding *which data to log* was the most prominent implementation difficulty among the three logging-decision questions. Half of the respondents agreed that it was difficult to decide which data should be logged, compared with 12.5% agreeing that deciding how to log was difficult. For deciding when to log, 12.5% strongly agreed that this was difficult, while 50% disagreed and 37.5% were neutral. Similarly, 62.5% disagreed that deciding how to log was difficult. Half of the respondents disagreed or strongly disagreed that their effort for logging data was high. The responses therefore show a greater reported difficulty around log-content selection than around the timing or technical mechanism

of logging (Figure 4.5).

To which extent do you agree or disagree with the following statements?

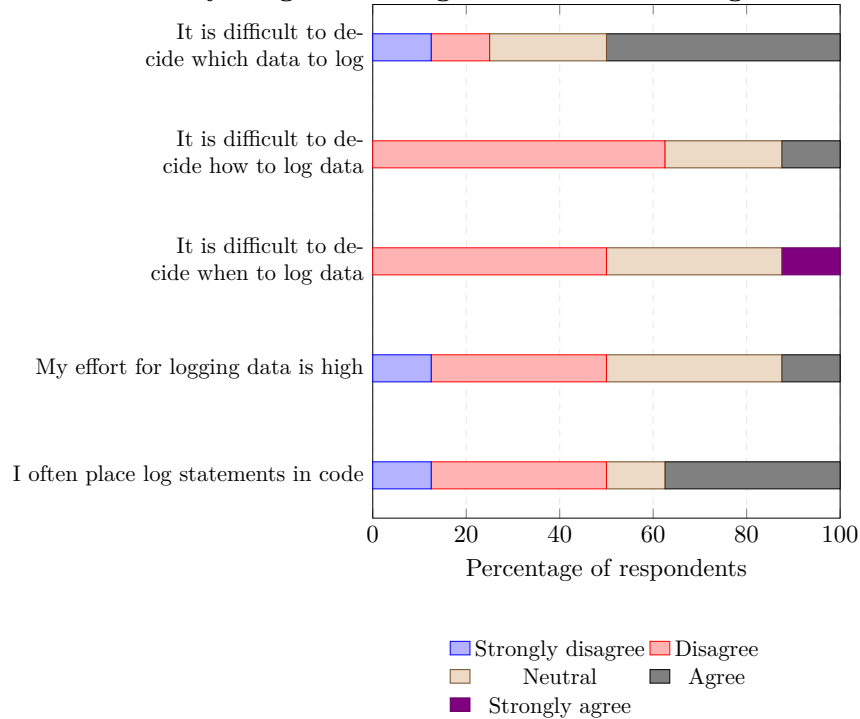


Figure 4.5: Respondents' perceptions of logging implementation practices and difficulties.

The open-ended responses provided additional examples of how developers approach logging implementation in practice. Reported practices included creating clear, readable and structured log output, placing logs at module or component level to support execution tracing and using third-party applications for error logging. Some respondents described considering logging during architecture and design, using the same logging framework where possible and avoiding both excessive and insufficient logging.

Other reported practices included recording incoming and outgoing data, writing meaningful log entries, sampling logs, considering the performance cost of logging and discussing logging implementation within the development team. The individual paraphrased responses are provided in Appendix A.0.1.1.

4.1.3 Log Analysis Practices

Seven respondents answered the log-analysis section. Respondents reported spending between 5% and 70% of their overall time analysing logged data, with a median of 10% (Figure 4.6).

What percentage of your overall time do you spend analysing logged data?

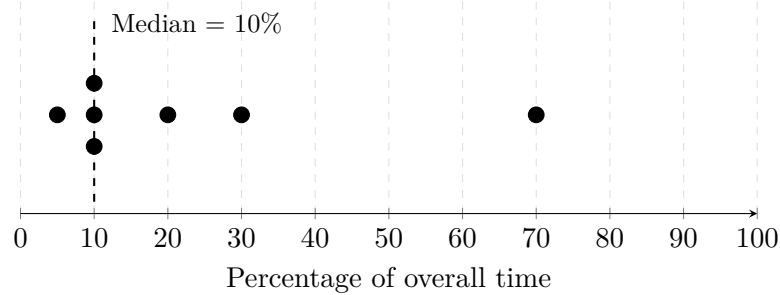


Figure 4.6: Percentage of overall time spent analysing logged data among respondents who completed the log-analysis section ($n = 7$).

All seven respondents reported using logged data for debugging or resolving failures. Identifying failures and locating faults were each reported by 85.7% (6/7), customer-reported errors by 57.1% (4/7), and performance monitoring by 28.6% (2/7) (Figure 4.7).

Why do you analyse logged data?

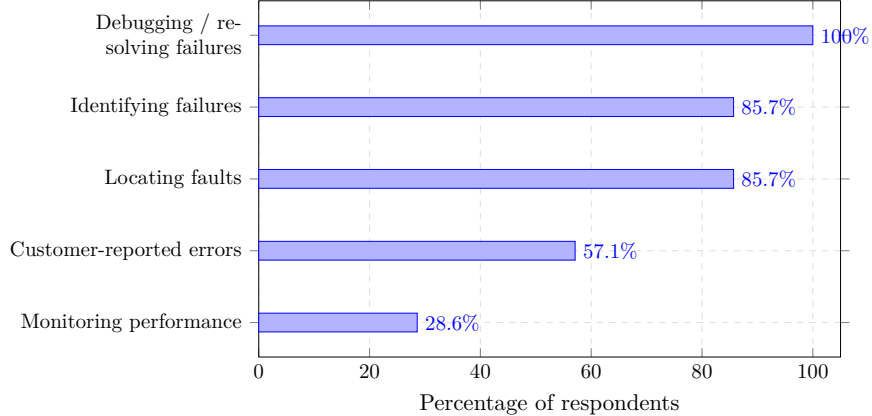


Figure 4.7: Reported reasons for analysing logged data ($n = 7$). Multiple selections were possible.

A majority of respondents considered structured log messages easier to analyse:

42.9% agreed, and 14.3% strongly agreed. The same distribution was observed for the preference for analysing information logged by oneself. At the same time, 57.1% agreed that logs are generally overwhelming. For failure-related logs, 50% agreed or strongly agreed that the available information can be overwhelming, while 33.3% were neutral (Figure 4.8).

To which extent do you agree or disagree with the following statements?

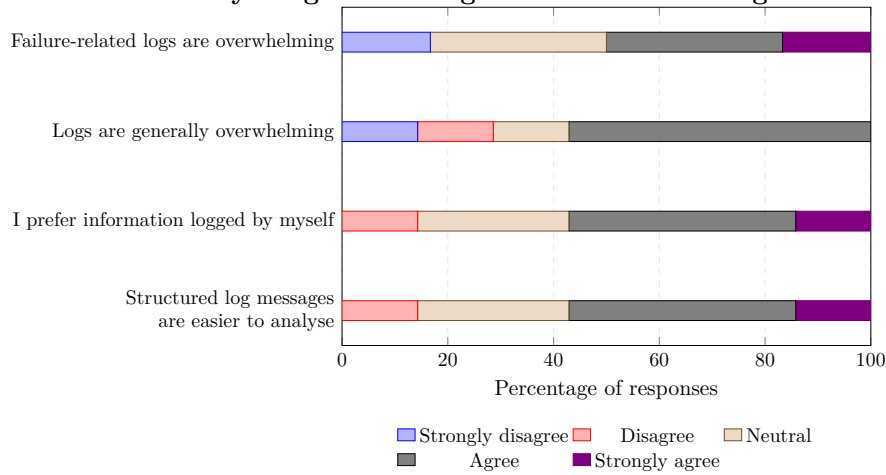


Figure 4.8: Respondents' perceptions of log analysis. The failure-related-log statement is based on six responses; the remaining statements are based on seven responses.

Analysis of unfamiliar logs was relatively infrequent. Five of the seven respondents (71.4%) selected “less frequently”, while one respondent reported analysing unfamiliar logs daily and one weekly. Collaborative log review was also generally infrequent: 42.9% selected “less frequently”, 28.6% monthly, 14.3% daily, and 14.3% never (Figure 4.9).

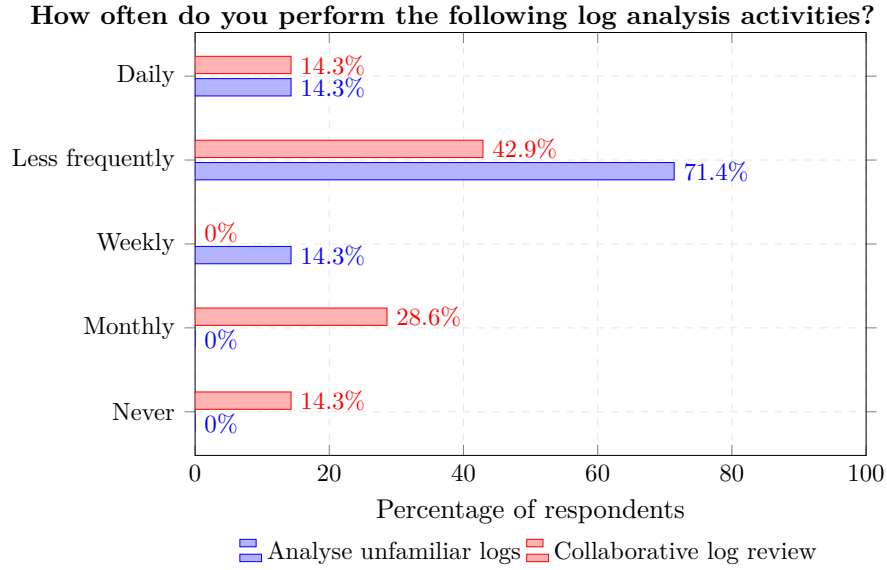


Figure 4.9: Reported frequency of analysing unfamiliar logs and reviewing logs collaboratively ($n = 7$).

Semi-structured logs were the most commonly analysed format (5/7 respondents), followed by structured logs (2/7). No respondent reported unstructured logs as the primary format analysed.

The open-ended responses provided additional information about how developers perform log analysis in practice. Respondents described selecting relevant logs based on the affected component, recent builds or operations, previous experience, log naming conventions and the reported source of a problem. Several responses described linking log information back to the corresponding source code to understand the execution flow or identify the location of an error.

Reported analysis techniques included searching for error strings, timestamps, and other relevant keywords, locating recent error or warning messages, tracing the corresponding source code and examining changes in system input, output or state. Some respondents additionally described consulting documentation, other developers or AI-supported tools when log information was difficult to interpret.

Reported recommendations included using clear and consistent log structures, understanding the relevant component and execution flow, using text search to locate relevant information and formatting logs so that important information can be identified more easily. The individual paraphrased responses are provided in Appendix A.0.2.1.

4.1.4 Fault Localization Practices

The fault-localisation section contained six valid responses. Respondents reported spending between 3% and 25% of their overall time localising faults, with a median of 12.5%, as shown in Figure 4.10.

What percentage of your overall time do you spend localising faults?

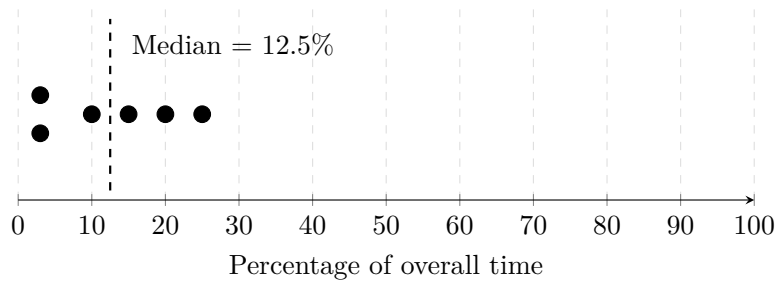


Figure 4.10: Percentage of overall time spent localising faults among respondents in the fault-localisation section ($n = 6$).

The time required to localise a single fault varied between respondents. Five of the six respondents (83.3%) reported that fault localisation took hours on average, while one respondent (16.7%) reported an average duration of days. The maximum reported localisation time was generally longer: 66.7% reported several days, while 16.7% reported hours and 16.7% reported weeks (Figure 4.11).

How long does it take to localise the fault of a single failure?

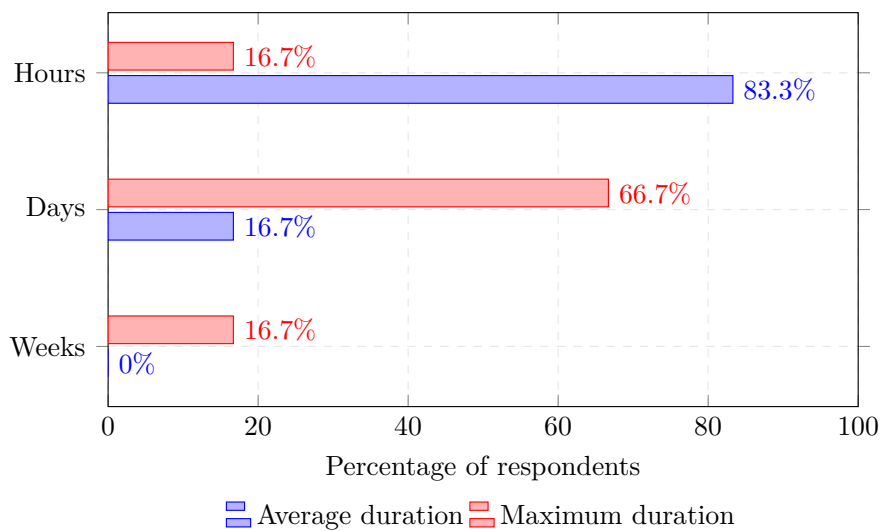


Figure 4.11: Reported average and maximum duration required to localise the fault of a single failure ($n = 6$).

The knowledge requirements varied by area. Domain or project knowledge received the highest proportion of “advanced” responses at 50%. Architecture knowledge was most frequently rated “moderate” (66.7%), while development-process or evolution history and responsibilities or group organisation were each rated “moderate” by 50%. Knowledge of the language or technology stack was most commonly rated “limited” (50%). Logging knowledge and log-analysis knowledge were also most frequently rated “limited” (50%), although a third of respondents considered advanced logging knowledge necessary (Figure 4.12).

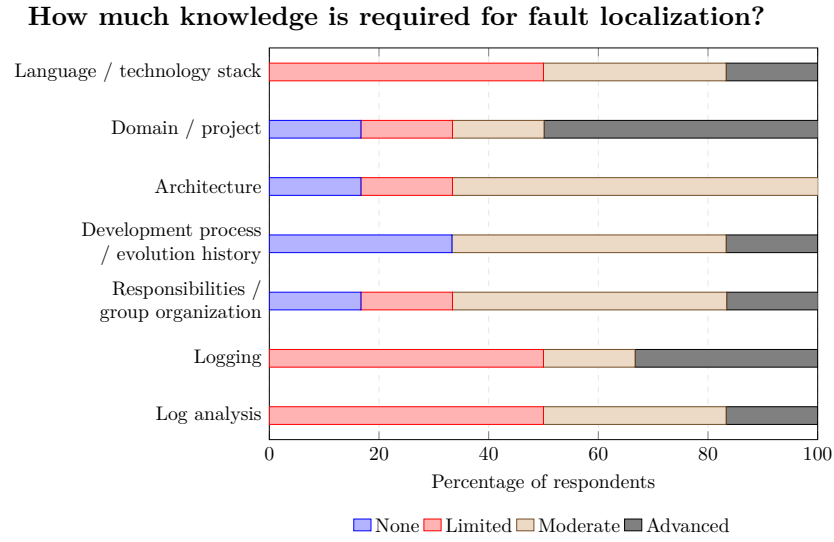


Figure 4.12: Reported level of knowledge required for fault localisation across different areas ($n = 6$).

Cross-component behaviour was a prominent difficulty. All respondents agreed or strongly agreed that faults involving interactions between system parts are harder to identify. Furthermore, 83.3% agreed or strongly agreed that logs from different system parts are analysed, and 83.3% agreed that logs are analysed multiple times during fault localisation. Two thirds of respondents agreed or strongly agreed that the available logged data is suitable for fault localisation. In contrast, only 33.3% agreed that logs lack required information, while 50% disagreed with this statement. Responses concerning the analysis of non-useful logs were evenly divided: 50% agreed or strongly agreed and 50% disagreed or strongly disagreed (Figure 4.13).

To which extent do you agree or disagree with the following statements?

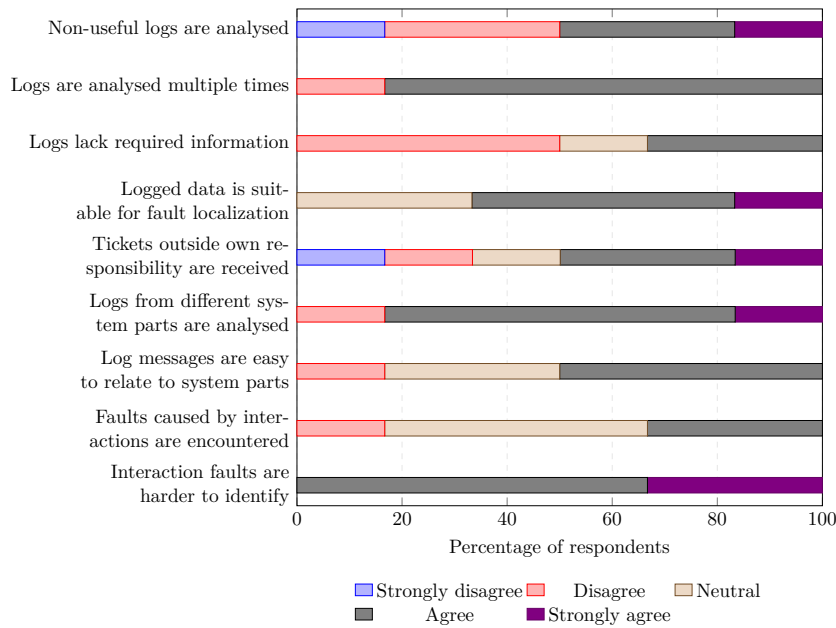


Figure 4.13: Respondents' reported experiences and challenges during fault localisation ($n = 6$). Statement labels were shortened for readability.

The open-ended responses provided further examples of how developers perform fault localisation. Reported practices included tracing an error backwards towards its underlying cause, identifying the affected functionality through log metadata or source code and examining the behaviour of the corresponding component.

Respondents also described combining logs with additional information. Examples included debugger output, variable values, configuration exports, generated files, database entries and information provided by users or customers. Logs were therefore not always used as the only source of information during fault localisation.

For failures involving several components, respondents described isolating the individual functionalities before examining their interaction, contacting developers responsible for other components and comparing expected system behaviour with the observed sequence of events. Historical project knowledge was also reported as a source for understanding interactions between system components. One respondent described constructing a flowchart of the involved systems and comparing the expected interaction sequence with the observed behaviour.

The individual paraphrased responses are provided in Appendix A.0.3.1.

4.1.5 Logging Challenges

Across the three survey parts, several recurring challenges can be observed in the structured responses. During implementation, the clearest difficulty concerned the selection of log content: 50% of respondents agreed that deciding which data to log is difficult, whereas fewer respondents reported comparable difficulty in deciding when or how to log.

During log analysis, the volume and interpretability of available information emerged as relevant concerns. Although a majority considered structured logs easier to analyse, 57.1% agreed that logs are generally overwhelming. Respondents also encountered unfamiliar logs and collaborative analysis, although these activities were generally reported less frequently than routine analysis of familiar logs.

During fault localisation, the strongest difficulty concerned failures caused by interactions across system components. All six respondents considered such faults harder to identify. The repeated analysis of logs and the need to inspect logs from different system parts further indicate that fault localisation frequently requires context beyond a single log statement or component.

4.1.6 Answer to RQ1

RQ1 investigated the logging practices and challenges encountered by software developers during implementation, log analysis and fault localisation.

For logging implementation, developers reported recording events, system state, and execution information using mechanisms ranging from print statements to established logging frameworks and project-specific solutions. The open-ended responses further described practices such as creating structured and readable logs, considering logging during architecture and design, using consistent logging frameworks, recording relevant input and output data and considering logging performance costs. The most frequently reported implementation difficulty was deciding which information should be logged.

For log analysis, logs were primarily used for debugging, failure identification and fault localisation. Reported practices included selecting logs based on the affected component or previous experience, searching for errors and timestamps, relating log entries to source code and execution flow and examining changes in system state or data. Structured logs were generally perceived as easier to analyse, while respondents also reported that log information can become overwhelming.

For fault localisation, respondents described tracing failures towards their underlying cause, connecting log entries with affected system components and combining logs with additional technical and contextual information. Fault localisation also involved collaboration with developers responsible for other components when several parts of a system interacted. Failures involving such interactions were consistently reported as more difficult to identify.

Overall, the survey shows that logging practice extends beyond the creation of log

statements. Developers use logging throughout implementation, analysis and fault localisation and combine log information with source code, system knowledge, additional runtime information and collaboration with other developers. At the same time, the responses show difficulties concerning the selection of useful log content, the interpretability and volume of log information and the localisation of faults across interacting system components.

4.2 Logging Guidelines from Academic Literature

This section addresses RQ2: *What logging guidelines are documented in academic literature?* The results summarise the selected academic literature, the identified logging recommendations across the four logging dimensions and the coverage of the retained actionable recommendations in the final consolidated open-source catalogue.

4.2.1 Literature Search Results

The literature collection resulted in 31 academic publications relevant to software logging. In addition, seven logging tools or utilities identified during the literature analysis were recorded separately as supporting information. These tool entries were not counted as independent academic publications.

The literature search combined publications identified directly through the defined Google Scholar search terms with earlier publications identified through backward snowballing. The resulting literature set therefore contained both recent studies and earlier work addressing logging practices, logging decisions, logging instrumentation and the subsequent use of generated log data.

The selected publications contained different forms of evidence. Some publications provided explicit or clearly identifiable logging recommendations. In contrast, others primarily reported empirical observations, conceptual classifications, logging tools, automated instrumentation approaches, evaluation results or techniques for processing and analysing logs. Consequently, inclusion in the literature set did not automatically mean that a publication contributed an actionable recommendation to the later guideline-level coverage comparison.

For example, *Improving Software Diagnosability via Log Enhancement* was retained because it contributes relevant research on logging and software diagnosability. However, the extracted material primarily described the LogEnhancer approach and its method for improving log information rather than providing a sufficiently explicit standalone logging recommendation. It was therefore retained as supporting academic evidence but was not treated as an independent recommendation target. The same distinction was applied to other publications whose relevant contribution was primarily empirical, conceptual, tool-oriented, descriptive or evaluation-based.

A publication was treated as a source of actionable logging guidance only when an

identifiable recommendation prescribed, recommended or clearly demonstrated an action concerning the creation, configuration or implementation of logging. A single publication could therefore contribute no actionable recommendation, one recommendation or several conceptually distinct recommendation targets.

This distinction between the complete literature set and the actionable academic recommendation set was maintained throughout the subsequent analysis. The complete set of 31 publications provides the academic context for RQ2, while only the identified actionable recommendations are used for the later guideline-level comparison with the consolidated open-source catalogue.

4.2.2 Selected Literature

The selected publications addressed different aspects of software logging. Four publications were particularly important for the deductive classification used later in the open-source analysis because they provided the initial structure for the four high-level logging dimensions.

Dimension	Academic basis	Literature-derived themes
What to log	<i>Industry Practices and Event Logging: Assessment of a Critical Software Development Process</i> [26]	State dump, Execution tracing, Event reporting
Where to log	<i>Where Do Developers Log? An Empirical Study on Logging Practices in Industry</i> [13]	Assertion-check, Return-value-check, Exception, Logic-branch, Observing-point logging
How to log	<i>A Survey of Software Log Instrumentation</i> [5]	Logging approach, Logging utility integration, Logging code composition
Log verbosity	Gholamian and Ward [14]	FATAL, ERROR, WARN, INFO, DEBUG, TRACE

Table 4.1: Academic sources used to ground the initial deductive classification.

These publications provided the deductive starting point for the thematic classification. The literature-derived themes were not treated as a complete classification of all logging guidance. Instead, they provided an initial structure that was later applied to the open-source guideline corpus and extended inductively where the deductive themes did not sufficiently represent recurring concepts.

4.2.3 Extracted Logging Guidelines

The actionable recommendations identified in the academic literature addressed all four major logging decisions considered in this thesis: *what to log*, *where to log*, *how to log*, and *log levels and verbosity*.

Recommendations concerning *what to log* included recording system state, dynamic values, events, failures, errors and contextual information.

Recommendations concerning *where to log* addressed locations such as assertion checks, exception handlers, return-value checks, logic branches, observing points and method or function boundaries.

Within *how to log*, the literature contained guidance concerning structured and consistent log representations, logging approaches, the integration of logging utilities, and composing log messages.

The *log levels and verbosity* dimension contained recommendations concerning the selection and use of severity levels such as FATAL, ERROR, WARN, INFO, DEBUG and TRACE, as well as the balance between logging detail, usefulness, and operational cost.

Not every publication contributed a standalone actionable recommendation. Some publications instead provided empirical findings, classifications, tool descriptions, evaluation results or broader observations concerning logging. These publications were retained in the academic literature set but were not forced into the coverage analysis.

The literature review identified a set of academic logging recommendations that was kept separate from publications whose contributions were primarily empirical, conceptual, descriptive or tool-oriented. Depending on its content, a publication could contribute none, one or several distinct recommendation targets. These targets were subsequently used as the academic reference set for comparison with the consolidated open-source catalogue.

4.2.4 Coverage in the Open-Source Catalogue

The retained actionable academic recommendations were subsequently compared with the final consolidated open-source catalogue to assess the extent to which the recommendations identified in academic literature were represented in the collected practical guidance.

The comparison was performed at the level of individual actionable recommendations rather than at publication level. A publication could therefore contribute several distinct recommendation targets, while publications retained primarily as empirical, conceptual, descriptive or tool-oriented evidence did not contribute an independent coverage target.

A recommendation was classified as *fully covered* when one or more canonical guidelines expressed the same underlying recommendation while preserving its relevant

meaning, scope and conditions. A recommendation was classified as *partially covered* when the principal logging concern was represented in the catalogue but one or more relevant aspects, conditions or sub-recommendations were not represented sufficiently. A recommendation was classified as *not identified* when no canonical guideline sufficiently represented the underlying recommendation.

The comparison showed that a substantial part of the actionable academic guidance was also represented in the collected open-source guidance. Several recommendations were only partially represented, however, because individual aspects of the academic guidance were not identified completely in the final catalogue.

For example, the literature-derived recommendation concerning logging at different program locations was largely represented through guidelines addressing return-value checks, exception handling, logic branches and observing points. However, no open-source guideline in the collected corpus was assigned to *Assertion-check logging*. Other partial cases concerned recommendations whose main logging concern was represented in the catalogue but whose complete combination of conditions or sub-recommendations was not identified.

The comparison therefore indicates substantial overlap between actionable recommendations documented in academic literature and logging guidance identified in the collected open-source sources, while also revealing specific aspects of academic guidance that were only partially represented.

4.2.5 Answer to RQ2

RQ2 investigated which logging guidelines are documented in academic literature. The literature collection contained 31 academic publications, together with seven logging tools or utilities recorded separately as supporting information.

The literature contained both actionable logging recommendations and publications whose contributions were primarily empirical, conceptual, descriptive, tool-oriented or evaluation-based. Publications without a sufficiently identifiable actionable logging recommendation were retained as supporting academic evidence but were not treated as independent recommendation targets.

The actionable recommendations addressed all four logging dimensions considered in this thesis: *what to log*, *where to log*, *how to log*, and *log levels and verbosity*. Recommendations included guidance concerning the information that should be recorded, suitable locations for logging, the construction and integration of logging mechanisms, and the selection of appropriate log levels and verbosity.

The academic literature also provided classifications that formed the deductive starting point for the later thematic analysis of the open-source guideline corpus. These classifications included State dump, Execution tracing and Event reporting for *what to log*, five program-location themes for *where to log*, three themes concerning logging implementation for *how to log*, and the six conventional severity levels from FATAL to TRACE.

Comparison with the consolidated open-source catalogue showed substantial semantic overlap between the actionable academic recommendations and the practical guidance identified in open-source sources. Some recommendations were only partially represented because individual aspects or conditions of the academic guidance were not identified completely in the collected open-source corpus. In particular, *Assertion-check logging* remained part of the literature-derived codebook but was not observed in the collected open-source guidance.

4.3 Logging Guidelines from Open-Source Repositories

This section addresses RQ3: *What logging guidelines are documented in open-source repositories?* The results present the GitHub search and collection results, the characteristics of the collected sources, the extracted logging guidelines and the construction, validation and refinement of the final consolidated guideline catalogue.

4.3.1 Repository Collection Results

The GitHub collection was performed using ten search queries defined in the methodology. The individual query-result files contained 966 search-result rows in total. Because the same repository or file could be returned by more than one query, the combined results contained 939 unique file URLs belonging to 895 unique repositories.

Table 4.2 shows the number of results obtained for each query.

Query	Search focus	Results
Query 1	General logging guidelines, standards, conventions, and policies	87
Query 2	What to log, log levels, structured logging, and sensitive information	102
Query 3	Logging in relation to observability, tracing, telemetry, and engineering practices	86
Query 4	Structured logging, correlation IDs, and trace IDs	102
Query 5	Sensitive data, PII, GDPR, and credentials in logs	109
Query 6	Log levels and severity conventions	103
Query 7	Logging guidance with additional Java and Go focus	94
Query 8	Logging and observability guidance in Markdown documentation	104
Query 9	Observability, telemetry, diagnostics, tracing, and logging practices	96
Query 10	Logging smells, bad practices, and anti-patterns	83
Total		966

Table 4.2: Focus and number of results obtained for the ten GitHub search queries.

The query results formed the initial candidate set for the open-source collection. Repeated results across queries were removed, and sources that were no longer available or had been removed were excluded during collection. File-level duplicate checking was additionally supported by cosine-similarity analysis. Highly similar files were reviewed to identify duplicate content, while files that remained meaningfully different were retained. This similarity analysis was used only for source-file cleaning and was separate from the later semantic consolidation of individual logging guidelines. Following the cleaning process, manual relevance review, and the extended source collection, 229 source files were retained for content extraction. The files represented 11 file types, with Markdown files forming the majority of the corpus.

File type	Files
.md	188
.go	22
.txt	9
.adoc	2
.rst	2
.yaml	1
.mdx	1
.pod	1
.toml	1
.py	1
.ts	1
Total	229

Table 4.3: File types in the collected logging corpus before content extraction.

After extraction and contextual processing, 203 source files contributed at least one extracted row to the structured dataset. Of these, 202 sources contributed at least one valid logging guideline. The structured dataset contained 2,347 source rows, consisting of 2,289 valid source-level guideline occurrences and 58 rows classified as `NO_GUIDELINE`.

Among the valid guideline occurrences, 1,885 (82.4%) originated from repository sources and 404 (17.6%) from external or third-party logging sources included in the catalogue. At source level, 185 repository sources and 17 external or third-party sources contributed at least one valid guideline.

4.3.2 Repository Characteristics

The collected sources represented a broad range of project contexts, programming languages and application fields. Metadata were retained for project type, programming language, application field, repository activity and source provenance.

The metadata labels were kept in their descriptive form rather than being assigned to a small set of predefined categories. Across the 203 sources contributing extracted content, the structured corpus contained 200 distinct project-type descriptions, 187 distinct language or technology descriptions and 200 distinct application-field descriptions.

For the 185 GitHub repository sources that contributed at least one valid guideline, repository size and activity were additionally summarised using stars, commits, and the number of developers or contributors. Because these distributions were not symmetrically distributed, both mean and median values are reported together with the observed minimum and maximum.

Repository metric	N	Mean	Median	Min.	Max.
Stars	185	188.4	2	0	10.644
Commits	185	991.3	88	1	52.217
Developers / contributors	185	12.2	2	1	390

Table 4.4: Descriptive characteristics of the GitHub repositories contributing valid logging guidelines.

The large differences between the mean and median values show that the repository characteristics were unevenly distributed. The median repository had two stars, 88 commits and two developers or contributors, whereas the observed maxima reached 10.644 stars, 52.217 commits and 390 developers or contributors. The collected guidance therefore originated from both comparatively small projects and substantially larger open-source projects. These characteristics are treated as contextual information about the sources rather than as indicators of guideline quality.

Repository creation and update dates were also retained as descriptive indicators of project age and activity. The contributing repositories included projects created between 2014 and 2026. The available update metadata also showed a large number of repositories with recent activity, particularly in 2025 and 2026.

4.3.2.1 Project Types

The project types ranged from reusable logging libraries and application frameworks to educational repositories, cloud-platform guidance, database automation, command-line tools, workflow engines, desktop utilities and domain-specific applications. The diversity of project descriptions shows that the collected logging guidance was not restricted to a single software architecture or application type.

4.3.2.2 Programming Languages

The language metadata included language-specific repositories as well as multi-language guideline documents. Frequently represented technologies included Python, JavaScript/TypeScript, Node.js, Go, Java/Kotlin, Rust and framework-specific logging stacks. Because several guidance documents addressed multiple languages, the language information was retained as descriptive metadata rather than reducing every source to one programming language.

4.3.2.3 Application Fields

The application-field metadata similarly represented heterogeneous contexts, including cloud and platform engineering, security, education, developer tooling, workflow orchestration, data platforms, database administration, desktop and systems software and domain-specific applications.

These metadata were retained in the catalogue to preserve the project context of individual recommendations and to allow guidelines to be examined according to their source environment.

4.3.3 Extracted Logging Guidelines

The extraction process produced 2,347 structured source rows. The source content and its contextual information were retained during the subsequent normalisation process.

For each extracted item, the source content and its Content Tag were analysed together to formulate a clear, concise and self-contained logging guideline. Where the extracted material did not support a meaningful logging recommendation, the row was classified as `NO_GUIDELINE` rather than being forced into the catalogue.

This resulted in 2,289 valid source-level guideline occurrences and 58 `NO_GUIDELINE` rows. The valid occurrences formed the source-level guideline corpus used for the subsequent thematic classification and consolidation.

In addition, 440 source rows contained code, configuration, input, usage or program-output material that was identified as relevant example information. These examples were retained separately as Example Reasons and linked to the corresponding guideline where applicable.

After thematic classification, semantically equivalent guidelines were consolidated while preserving differences in recommended action, conditions, scope and restrictions. Guidelines with uncertain equivalence were retained separately.

Semantic consolidation reduced the 2,289 source-level guideline occurrences to 2,127 canonical guidelines, corresponding to a reduction of approximately 7.1%. Most canonical guidelines (2,034) were unique or deliberately retained separately. Fifty-four consolidation groups resulted from normalised-identical guideline wording, while

39 groups represented conservative semantic matches. In total, 93 canonical guidelines represented more than one source-level occurrence.

Metric	Count
Collected source files	229
Source files contributing extracted rows	203
Source files contributing valid guidelines	202
Structured source rows	2.347
Valid source-level guideline occurrences	2.289
NO_GUIDELINE rows	58
Example Reason rows	440
Canonical guidelines after consolidation	2.127
Unique / kept separate canonical guidelines	2.034
Normalized-identical consolidation groups	54
Conservative semantic-match groups	39
Multi-row consolidation groups	93

Table 4.5: Open-source guideline collection, extraction, and consolidation results.

4.3.4 Validation and Refinement of the Guideline Catalogue

The initial thematic classification was subsequently validated against the complete deductive-inductive codebook. Particular attention was given to broad themes that contained a comparatively large number of guidelines. The most prominent case was *Logging approach*, to which 794 source-level guidelines had initially been assigned. The guidelines assigned to this theme were re-examined together with their source Content and compared against all themes in the final codebook. Following this validation, 378 guidelines remained classified as *Logging approach*, while 416 were reassigned to more specific themes. Of these reassigned guidelines, 149 remained within the *How to log* dimension but were represented more specifically by themes such as *Logging utility integration*, *Logging code composition*, *Log lifecycle management*, *Logging verification*, or *Sensitive-data protection*. A further 267 guidelines were reassigned to themes belonging to another high-level logging dimension where the source Content supported a more appropriate interpretation.

Subsequent manual validation of the consolidated mapping resulted in four further assignment changes, leaving 374 source-level occurrences assigned to *Logging approach* in the final catalogue.

The same classification logic was subsequently applied across the remaining themes to identify comparable inconsistencies. Theme assignments were corrected where the original Content supported a more specific existing theme. The deductive or inductive origin of a theme itself was not changed during this procedure, it was only the assignment of individual guidelines that was reconsidered.

Following thematic validation, the semantic consolidation was also manually inspected. Consolidated groups containing more than one source occurrence were compared with their original Content to determine whether all occurrences represented the same underlying recommendation. Where different recommendations had been generalised into the same normalised wording, the affected source occurrences were separated. If another canonical guideline already represented the recommendation, the occurrence was reassigned to that guideline rather than creating a duplicate canonical recommendation. A new canonical guideline was retained only where no sufficiently equivalent existing guideline was identified.

As part of the final validation, consolidated guidelines whose source-level occurrences had previously been associated with different dimensions or themes were re-examined against their underlying Content. Each canonical guideline was assigned to the single high-level logging dimension and theme that most specifically represented its primary recommendation. Earlier source-level classifications were retained in the consolidation mapping for traceability.

For example, one initially consolidated error-related guideline contained source occurrences related to several distinct concepts, including error logging, stack traces, correlation identifiers, structured error representation, and diagnostic configuration. During manual validation, these occurrences were redistributed to existing canonical guidelines representing the corresponding recommendations. Only a recommendation concerning pointer dereferencing in structured error representations remained as a distinct guideline because no sufficiently equivalent canonical guideline was available.

The final catalogue therefore reflects both semantic consolidation and subsequent manual validation of the merged groups while preserving the mapping from each source-level occurrence to exactly one canonical guideline.

4.3.5 Distribution Across Logging Categories

Following the final thematic validation, each of the 2,289 valid source-level guideline occurrences was associated with exactly one final high-level logging dimension. These source-level occurrences represent the individual instances of logging guidance identified across the collected sources. The final classification therefore contained 2,289 source-level dimension assignments.

Semantic consolidation grouped source-level occurrences representing the same underlying recommendation while preserving the mapping to their sources. As a result, the 2,289 source-level occurrences were represented by 2,127 final canonical guidelines. Each canonical guideline was assigned to exactly one high-level logging dimension and one corresponding theme. The source-level counts therefore describe how frequently guidance belonging to each dimension occurred in the collected corpus. In contrast, the consolidated counts describe the number of distinct canonical recommendations retained in the final catalogue.

At source level, *How to log* formed the largest dimension, with 921 occurrences

(40.2%). *What to log* contained 644 occurrences (28.1%), *Log levels and verbosity* contained 391 occurrences (17.1%), and *Where to log* contained 333 occurrences (14.5%).

At the consolidated level, *How to log* contained 865 canonical guidelines, followed by *What to log* with 603, *Log levels and verbosity* with 337, and *Where to log* with 322. The differences between the source-level and consolidated counts reflect the consolidation of semantically equivalent recommendations that occurred independently in different sources.

The detailed distribution of source-level occurrences and consolidated guidelines across dimensions and themes is presented in Section 4.4.

4.3.6 Answer to RQ3

RQ3 investigated which logging guidelines are documented in open-source software repositories. The ten GitHub searches produced 966 search-result rows, representing 939 unique file URLs across 895 repositories before the subsequent cleaning and source-selection stages.

After removing duplicates, reviewing relevance, and expanding the source collection, 229 source files were retained for content extraction. The retained material included documentation, source code, configuration files and other repository artefacts containing potentially relevant logging guidance.

The extraction and contextual processing stages produced 2,347 structured source rows. Of these, 2,289 contained valid source-level logging guideline occurrences, while 58 rows were classified as `NO_GUIDELINE` because no sufficiently supported logging recommendation could be derived from the corresponding source content.

The identified guidance covered a broad range of logging decisions and originated from heterogeneous project types, programming languages and application fields. The source material also contained practical code, configuration, input, usage and program-output examples. A total of 440 such source rows were retained as *Example Reasons* and linked to the corresponding guidelines where applicable.

Following guideline normalisation and thematic classification, semantically equivalent recommendations were consolidated while preserving differences in recommended action, conditions, scope and restrictions. The consolidation was subsequently manually validated against the source Content. Where a consolidated group contained different underlying recommendations, the affected source occurrences were separated and reassigned to an existing canonical guideline where an appropriate equivalent was already available. A new canonical guideline was retained only when no existing guideline adequately represented the recommendation.

After semantic consolidation and subsequent manual validation, 2,289 valid source-level guideline occurrences were represented by 2,127 canonical guidelines. This corresponds to a reduction of 162 repeated occurrences, or approximately 7.1%. Of the final canonical guidelines, 2,034 represented a single source occurrence, while 93 represented more than one source occurrence.

The source-to-canonical mapping retained all 2,289 valid occurrences, with every source-level guideline mapped to exactly one canonical guideline. The resulting catalogue therefore preserves both the original occurrence of logging guidance in the collected sources and a less repetitive canonical representation of semantically equivalent recommendations. The detailed thematic structure of these guidelines is examined separately in RQ4.

4.4 Themes in the Logging Guideline Catalogue

This section addresses RQ4: *What themes can be identified within the logging guideline catalogue?* The results present the validated thematic structure of the catalogue across the four logging dimensions and show how the initial literature-derived themes were extended by inductive themes identified in the collected guideline corpus.

4.4.1 Thematic Catalogue Overview

RQ4 investigated which themes can be identified within the logging guideline catalogue. The thematic analysis was based on the 2,289 valid source-level guideline occurrences identified in the open-source corpus.

The initial classification used 17 deductive themes derived from the academic literature. Where a guideline could not be adequately represented by an existing deductive theme, an additional inductive theme was introduced. After reviewing and consolidating overlapping inductive concepts, the final codebook contained 28 themes: 17 deductive themes and 11 inductive themes.

Before calculating the final theme distribution, the theme assignments were validated against the complete deductive-inductive codebook and, where necessary, against the source Content. This validation was particularly relevant for broad themes containing a comparatively large number of guidelines.

The most prominent case was *Logging approach*, to which 794 source-level guideline occurrences had initially been assigned. After re-examination of this theme, 378 occurrences initially remained assigned to *Logging approach*, while 416 were reassigned to more specific themes. Of these reassigned occurrences, 149 remained within the *How to log* dimension, whereas 267 were reassigned to themes belonging to another high-level logging dimension.

Subsequent manual validation of the consolidated mapping resulted in four further assignment changes, leaving 374 source-level occurrences assigned to *Logging approach* in the final catalogue.

Comparable classification inconsistencies identified during this validation were also corrected across the remaining themes. The validation changed the assignment of individual guidelines where the guideline and its original Content supported a more appropriate existing theme. It did not change the deductive or inductive origin of the themes themselves. Consequently, the distributions reported in the following subsections represent the final validated classification rather than the initial AI-assisted assignments.

The semantic consolidation of the catalogue was subsequently reviewed against the source Content. Consolidated groups containing several source occurrences were checked to determine whether they represented the same underlying recommendation without losing a distinct action, condition, restriction or scope. Where a consolidated group contained semantically different recommendations, the affected

occurrences were separated. If an equivalent canonical guideline already existed elsewhere in the catalogue, the occurrence was reassigned to that guideline instead of introducing a duplicate recommendation. New canonical guidelines were retained only where an existing guideline did not adequately represent the corresponding recommendation.

After final validation, each of the 2,289 valid source-level occurrences remained mapped to exactly one canonical guideline. The final catalogue contained 2,127 canonical guidelines.

Each canonical guideline was assigned to one final high-level logging dimension and corresponding theme. Consequently, the 2,289 valid source-level occurrences resulted in 2,289 final source-level dimension assignments. Earlier source-level classifications that associated content with more than one logging dimension were retained in the consolidation mapping for traceability but were not counted as multiple final thematic assignments.

Dimension	Source-level	Source-level %	Consolidated	Consolidated %
What to log	644	28.1%	603	28.4%
Where to log	333	14.5%	322	15.1%
How to log	921	40.2%	865	40.7%
Log levels and verbosity	391	17.1%	337	15.8%
Total	2,289	100.0%	2,127	100.0%

Table 4.6: Distribution of source-level guideline occurrences and final consolidated guidelines across the four logging dimensions.

Percentages may not sum to exactly 100% due to rounding.

Source-level assignments ($n = 2,289$) represent the individual occurrences identified in the collected sources, while consolidated guidelines ($n = 2,127$) represent the final canonical recommendations after semantic consolidation and validation.

At the dimension level, *How to log* remained the largest category after final validation, accounting for 921 source-level assignments. The remaining assignments were distributed across *What to log*, *Log levels and verbosity*, and *Where to log*. The following subsections present the detailed distribution of the validated themes within these four dimensions.

4.4.2 What to Log

Four themes were identified within the *What to log* dimension. Three themes originated from the deductive literature-based codebook, while *Sensitive-data exclusion* was introduced inductively.

Theme	Basis	Source-level	Consolidated
State dump	Deductive	254	245
Execution tracing	Deductive	24	20
Event reporting	Deductive	337	309
Sensitive-data exclusion	Inductive	29	29
Total		644	603

Table 4.7: Themes within the *What to log* dimension.

Event reporting was the largest theme within this dimension, with 337 source-level occurrences and 309 consolidated assignments. These guidelines concerned the recording of discrete operational, business, security, lifecycle or user-related events. *State dump* was the second largest theme and represented recommendations concerning the recording of program, request, object, variable or system state so that the state surrounding an event or failure can later be reconstructed.

Execution tracing occurred less frequently and concerned calls, transitions, execution steps or timing information along an execution path.

The inductive theme *Sensitive-data exclusion* represented explicit recommendations describing information that should not be recorded, including credentials, secrets, personal information or other sensitive data. This extended the *What to log* dimension by including explicit restrictions on log content in addition to recommendations concerning information that should be recorded.

4.4.3 Where to Log

The final *Where to log* dimension contained ten themes. Five originated from the deductive literature-based classification and five were introduced inductively from the open-source guideline corpus.

Theme	Basis	Source Assignment	Consolidated Assignment
Assertion-check logging	Deductive	0	0
Return-value-check logging	Deductive	1	1
Exception logging	Deductive	16	15
Logic-branch logging	Deductive	12	12
Observing-point logging	Deductive	6	6
Log destination	Inductive	150	144
Log storage and retention	Inductive	16	15
Logger placement and reuse	Inductive	9	10
Logging configuration location	Inductive	4	4
Logging scope	Inductive	119	115
Total		333	322

Table 4.8: Themes within the *Where to log* dimension.

The largest themes within this dimension were *Logging scope* and *Log destination*. *Logging scope* describes the component, subsystem, operation, logger or audit boundary to which logging applies. *Log destination* describes where generated log records should be sent, including files, consoles, syslog facilities and other logging sinks.

Log storage and retention covered requirements concerning the persistence, storage, rotation or retention of generated logging information. *Logger placement and reuse* concerned the placement and reuse of logger instances, while *Logging configuration location* described where logging configuration should be defined or maintained.

For *Logger placement and reuse*, the consolidated count is one higher than the source-level count. This difference results from a canonical guideline being reassigned to this theme during final validation, while the corresponding source-level occurrence retained its earlier classification in the source-level mapping used for the occurrence-based distribution. The source-level and consolidated counts therefore reflect different units of analysis.

The literature-derived program-location themes were represented much less frequently. *Logic-branch logging* contained 12 source-level occurrences, *Observing-point logging* six, *Exception logging* 16 and *Return-value-check logging* one.

No source-level guideline was assigned to *Assertion-check logging*. The theme was nevertheless retained in the final codebook because it formed part of the literature-derived deductive classification.

The inductive themes therefore extended the meaning of *Where to log* beyond source-code control-flow locations. Within the collected practical guidance, “where” also referred to the scope in which logging applies, the destination of generated records, and the location at which logging information is stored or configured.

4.4.4 How to Log

The *How to log* dimension formed the largest part of the final catalogue. It contained six themes: three deductive and three inductive.

Theme	Basis	Source-level	Consolidated
Logging approach	Deductive	374	360
Logging utility integration	Deductive	215	201
Logging code composition	Deductive	242	218
Log lifecycle management	Inductive	53	52
Logging verification	Inductive	16	16
Sensitive-data protection	Inductive	21	18
Total		921	865

Table 4.9: Themes within the *How to log* dimension.

Logging approach was the largest individual theme in the complete catalogue, with 374 source-level occurrences and 360 consolidated assignments. It covered recommendations concerning the overall strategy, representation, mode or general practice used for logging.

The large number of guidelines within this theme does not mean that the guidelines are semantically equivalent. *Logging approach* functions as a broad organisational theme, while each guideline retains its specific recommended action, technical context, conditions and restrictions.

Logging code composition contained 242 source-level occurrences and covered recommendations concerning how log statements, messages, fields, and structured records should be constructed.

Logging utility integration contained 215 occurrences and represented guidance concerning the integration and configuration of logging frameworks, handlers, appenders, middleware and related utilities.

Three additional themes emerged inductively. *Log lifecycle management* covered the management of logging across its operational lifecycle. *Logging verification* represented recommendations concerning the verification or testing of logging behaviour, while *Sensitive-data protection* covered practices such as masking, filtering, or redacting sensitive information before writing it to logs.

4.4.5 Log Levels and Verbosity

The final *Log levels and verbosity* dimension contained the six literature-derived severity levels together with two inductively derived themes.

Theme	Basis	Source-level	Consolidated
FATAL	Deductive	3	3
ERROR	Deductive	19	19
WARN	Deductive	18	15
INFO	Deductive	48	33
DEBUG	Deductive	48	39
TRACE	Deductive	7	7
Log level selection	Inductive	242	215
Custom verbosity scale	Inductive	6	6
Total		391	337

Table 4.10: Themes within the *Log levels and verbosity* dimension.

The most frequently represented theme was *Log level selection*, with 242 source-level occurrences and 215 consolidated assignments. These guidelines concerned the selection, configuration or adjustment of an appropriate logging level rather than prescribing one individual severity level.

Among the six deductive severity levels, DEBUG and INFO were the most frequently represented, with 48 and 48 source-level occurrences respectively. FATAL and TRACE were comparatively uncommon.

The inductive *Custom verbosity scale* theme represented recommendations associated with project-specific verbosity systems that did not directly follow the conventional FATAL–TRACE hierarchy.

4.4.6 Final Thematic Codebook

The final codebook contained 28 themes across the four logging dimensions. Seventeen themes originated from the literature-derived deductive codebook, while 11 themes were introduced inductively from concepts identified within the collected open-source guidelines.

Dimension	Basis	Themes
What to log	Deductive	State dump Execution tracing Event reporting
What to log	Inductive	Sensitive-data exclusion
Where to log	Deductive	Assertion-check logging Return-value-check logging Exception logging Logic-branch logging Observing-point logging
Where to log	Inductive	Logging scope Log destination Log storage and retention Logger placement and reuse Logging configuration location
How to log	Deductive	Logging approach Logging utility integration Logging code composition
How to log	Inductive	Log lifecycle management Logging verification Sensitive-data protection
Log levels and verbosity	Deductive	FATAL ERROR WARN INFO DEBUG TRACE
Log levels and verbosity	Inductive	Log level selection Custom verbosity scale

Table 4.11: Final thematic codebook by logging dimension and theme origin.

Of the 28 themes retained in the final codebook, 27 were represented by at least one source-level guideline. *Assertion-check logging* was the only theme without an observed source-level occurrence.

The final themes function as organisational and analytical categories rather than as statements that all guidelines assigned to the same theme are semantically equivalent. This distinction is particularly important for broad themes such as *Logging approach*. The individual guideline remains the specific actionable unit, while the theme provides a higher-level structure for organising and analysing the catalogue.

4.4.7 Answer to RQ4

RQ4 investigated which themes can be identified within the logging guideline catalogue. The final thematic codebook contained 28 themes across the four high-level logging dimensions. Seventeen themes were derived deductively from the academic literature, while 11 additional themes were identified inductively from recurring concepts in the collected open-source guideline corpus.

Of the 28 themes, 27 appeared in at least one canonical guideline. The deductive theme *Assertion-check logging* was retained in the literature-derived codebook but was not observed in the collected open-source corpus.

Following final validation and consolidation, the catalogue contained 2,127 canonical guidelines, each assigned to one final dimension and one final theme. *How to log* remained the largest dimension, with 865 canonical guidelines (40.7%), followed by *What to log* with 603 guidelines (28.3%), *Log levels and verbosity* with 337 guidelines (15.8%), and *Where to log* with 322 guidelines (15.1%).

Within the final validated classification, *Logging approach* remained the largest individual theme, containing 360 canonical guidelines. Its final size was substantially lower than in the initial classification because guidelines that were better represented by more specific themes were reassigned during manual validation.

The inductive analysis extended the initial literature-derived structure in several areas. Additional themes captured sensitive-data exclusion and protection, logging scope and destination, log storage and retention, logger placement and logging configuration location, log lifecycle management, logging verification, general log-level selection and custom verbosity systems.

The resulting thematic structure therefore combines established literature-derived logging concepts with additional concerns identified in practical open-source guidance. The themes provide a higher-level organisation of the catalogue, while the individual canonical guidelines retain the specific recommendations, conditions and contextual distinctions required for practical use.

5 Discussion and Conclusion

This chapter discusses the main findings of the thesis in relation to the four research questions, reflects on the contribution of the resulting logging guideline catalogue and considers the study's limitations. Finally, it outlines directions for future research and further development of the catalogue.

5.1 Discussion of Findings

The results of this thesis show that software logging is not a single uniform practice or decision. Instead, logging involves decisions concerning the information that should be recorded, the locations and scopes in which logging should occur, the way logging should be implemented and the level of detail at which information should be emitted. The developer study, academic literature and open-source repository analysis provide complementary perspectives on these decisions.

A central observation across the study is that logging guidance is highly contextual. Individual recommendations may apply to particular frameworks, programming languages, architectural environments, security requirements or operational scenarios. At the same time, the collected recommendations can be organised according to recurring higher-level logging concerns. The resulting catalogue therefore distinguishes between the specific guideline as the primary unit of actionable knowledge and the theme as an organisational abstraction used to structure and analyse the catalogue.

5.1.1 Developer Practices and Challenges

The developer study addressed **RQ1** by examining practical logging behaviour, log analysis and fault localisation.

The responses indicate that logging forms a regular but varying part of software development work. Among respondents who reported implementing logging, the median estimated proportion of development time spent on logging was 10%, although the individual estimates varied considerably. This variation suggests that the amount of effort associated with logging depends strongly on the development context and individual developers' responsibilities.

Regarding the information being logged, event reporting was the most frequently

represented category among the respondents, followed by state information and execution tracing. This corresponds to the practical role of logs as records of relevant events and application behaviour. However, the responses also demonstrate that developers do not rely on a single form of logging. Different types of information are combined according to the purpose for which the logs are created.

The reported logging mechanisms further illustrate the heterogeneity of practical logging. Print statements and established logging frameworks were both commonly reported, while project-specific, external and internally developed logging solutions were also present. Logging practice is therefore not limited to dedicated logging frameworks but can involve multiple implementation mechanisms depending on the development environment.

The responses concerning the selection of logging information show that developers consider several contextual criteria, including system impact, security implications, the affected part of the system, event frequency, compliance requirements and previous incidents. This supports the view that logging decisions cannot always be reduced to universal rules. The usefulness of a logging recommendation may depend on the operational and technical context in which it is applied.

An important finding is that deciding *what* information to log appeared more problematic than deciding *how* to implement logging technically for part of the surveyed group. This distinction is relevant for the guideline catalogue. Frameworks and tools may well document technical logging mechanisms, while determining which information is useful in a particular situation remains more dependent on developer judgement and system context.

The log-analysis results additionally show that logs are used primarily for debugging, failure identification and fault localisation. All respondents to the corresponding question reported debugging or resolving failures as a reason for analysing logs, while identifying failures and locating faults were also frequently reported purposes. Logs therefore serve not merely as operational records but as an important source of diagnostic information during maintenance.

At the same time, the survey indicates that analysing logs can itself create difficulties. Some respondents perceived logs as overwhelming, and analysing unfamiliar logs was commonly reported as an activity performed only occasionally. Respondents also tended to perceive structured log information as easier to analyse. These observations support logging recommendations focused on structure, consistency and contextual information: producing additional log output is not necessarily useful if the resulting information is difficult to interpret.

The fault-localisation responses reinforce this point. Respondents reported that fault localisation can require substantial time and knowledge about the application, architecture, technologies and project domain. Faults involving interactions between system components were perceived as particularly difficult. Logs were generally regarded as useful for fault localisation. Still, the responses indicate that log information alone does not necessarily provide all of the contextual knowledge required to understand a failure.

Overall, the developer study suggests that logging should be considered part of a

broader diagnostic process rather than simply inserting log statements. Useful logging requires decisions about relevant content, sufficient context and appropriate representation, while subsequent analysis also depends on architectural and domain knowledge.

Several practices reported in the developer study also appear in the final guideline catalogue. Practices on selecting log content correspond to guidelines within the *What to log* dimension, including event reporting, state information and execution tracing. The reported use of logging frameworks and different technical logging mechanisms is reflected in themes such as *Logging utility integration* and *Logging approach*, while recommendations concerning structured and readable log messages are represented within *Logging code composition* and related implementation guidance. The catalogue therefore contains recommendations corresponding to several of the practical logging activities reported by the surveyed developers.

The reported challenges are not represented as challenges themselves, but the catalogue contains guidance relevant to several areas in which developers reported difficulty. The difficulty of deciding which information to record relates directly to the *What to log* dimension. Concerns about overwhelming or difficult-to-interpret log information are addressed by recommendations concerning message composition, structured representations and log-level selection. Difficulties in understanding behaviour across system components may also relate to recommendations on execution tracing, logging scope, and contextual logging information.

The comparison therefore shows that the final catalogue covers several areas corresponding to both the practices and the difficulties reported in the developer study. However, the existence of related guidance does not demonstrate that a practical challenge is fully resolved. The survey findings indicate that applying logging guidance remains dependent on the technical, architectural and operational context in which developers work.

5.1.2 Logging Guidance in Academic Literature

RQ2 investigated recommendations documented in academic software-logging research.

The literature analysis identified recommendations addressing the four central dimensions used throughout this thesis: *what to log*, *where to log*, *how to log*, and *log levels and verbosity*. In addition to providing individual recommendations, previous research provided conceptual classifications that could be used as the deductive foundation of the catalogue.

The literature-derived categories demonstrate that previous logging research has already investigated several important aspects of instrumentation. The *what to log* dimension distinguishes between state information, execution tracing and event reporting. Research on logging locations identifies assertion checks, return-value checks, exception handling, logic branches and observing points as locations at which logging may occur. Existing work on log instrumentation additionally distinguishes

between logging approaches, utility integration and logging-code composition. Finally, commonly used logging levels provide a standard starting point for analysing verbosity recommendations.

These classifications proved useful as an initial analytical framework. However, the open-source material showed that they were not sufficient to represent the complete range of collected recommendations. The need to introduce additional inductive themes is therefore an important result of the study. The literature-derived classifications remain useful as conceptual foundations, but practical open-source guidance extends beyond these initial categories.

The retained academic recommendations were manually compared with the final consolidated open-source catalogue. The comparison showed substantial semantic overlap between the actionable academic recommendations and the practical guidance identified in the collected open-source sources. Some recommendations were only partially represented because individual aspects, conditions or sub-recommendations of the academic guidance were not identified completely in the catalogue.

The partially represented recommendations indicate that similar logging concerns appear in academic and practical guidance, while the full scope or specific conditions of an academic recommendation are not always expressed in the corresponding open-source guidelines. In particular, assertion-check logging remained part of the literature-derived classification but was not observed in the collected open-source guidance.

5.1.3 Open-Source Logging Guidelines

RQ3 investigated logging guidance documented in open-source repositories and related practical sources.

The repository analysis resulted in a substantially larger and more heterogeneous body of logging guidance than the academic recommendation set. After extraction, normalisation and validation, 2.289 source-level guideline occurrences were retained. A further 58 extracted entries were classified as `NO_GUIDELINE` because no sufficiently supported actionable recommendation could be derived from the corresponding content.

The 2.289 source-level occurrences represent individual appearances of logging guidance within the collected corpus. They therefore include recommendations that occur independently in more than one repository or source. Semantic consolidation reduced these occurrences to 2.127 canonical guidelines.

The comparatively small reduction from 2.289 source-level occurrences to 2.127 canonical guidelines is noteworthy. It indicates that, although equivalent recommendations occur repeatedly, the collected corpus also contains a large amount of context-specific and distinct guidance. Logging documentation in open-source projects therefore cannot be reduced to a small number of universally repeated rules.

This result also supports the conservative consolidation strategy used in this thesis.

Recommendations concerning the same general topic were retained separately when they differed in their required action, scope, condition, or restriction. This preserves distinctions that may be important when the catalogue is used in a practical development context.

The semantic consolidation was subsequently subjected to final manual validation against the source Content. This review showed that similar or identical normalised wording did not always represent the same underlying recommendation. Where a consolidated group contained distinct actions, conditions, scopes or technical concerns, the affected source occurrences were separated. If an equivalent canonical guideline already existed elsewhere in the catalogue, the occurrence was reassigned to that guideline rather than creating an additional duplicate recommendation. A new canonical guideline was retained only where no sufficiently equivalent guideline was already represented in the catalogue.

The repository material also demonstrated that logging recommendations are not restricted to dedicated guideline documents. Actionable guidance was found in documentation, configuration descriptions, examples and source-code-related material. Practical examples containing code, configuration, input, usage or program output were consequently retained separately as Example Reasons where applicable. This allows the catalogue to preserve both the normalised recommendation and the practical context in which the behaviour was demonstrated.

The open-source findings therefore provide a complementary perspective to academic literature. Academic work generally formulates logging concepts at a broader level, while repository guidance frequently describes concrete actions required within particular tools, frameworks and technical environments. The catalogue preserves these specific recommendations rather than replacing them with only generalised rules.

5.1.4 Themes within the Logging Guideline Catalogue

RQ4 examined the themes represented within the logging guideline catalogue.

The final codebook contained 28 themes across the four dimensions *what to log*, *where to log*, *how to log*, and *log levels and verbosity*. Seventeen themes were derived from previous literature, and eleven additional themes emerged inductively from the collected guideline content. Of the 28 retained themes, 27 were represented by at least one guideline in the collected corpus, while *Assertion-check logging* remained unrepresented.

An important result of the thematic analysis was that the initial classification required further refinement before the final distribution could be determined. This was particularly visible for broad themes such as *Logging approach*. In the initial classification, 794 source-level guideline occurrences had been assigned to this theme. During validation, these occurrences were re-examined against the complete deductive-inductive codebook and, where necessary, against their source Content. After this review, 378 occurrences remained assigned to *Logging approach*, while

other existing themes better represented 416. Of the reassigned occurrences, 149 remained within the *How to log* dimension but were moved to more specific themes. In contrast, 267 were reassigned to themes belonging to another high-level logging dimension.

Subsequent manual validation of the consolidated mapping resulted in four further assignment changes, leaving 374 source-level occurrences assigned to *Logging approach* in the final catalogue.

This refinement demonstrates an important distinction between broad thematic similarity and specific guideline meaning. A recommendation may initially appear to concern a general logging approach. At the same time, its original Content more precisely describes, for example, a logging utility, message composition, destination, logging scope or verbosity decision. The final theme distribution therefore reflects the validated assignments rather than the initial AI-assisted classification.

The validation of *Logging approach* also illustrates why the size of a theme should not by itself be interpreted as evidence that it represents one homogeneous body of recommendations. Themes provide an organisational and analytical abstraction, whereas the individual guideline remains the more specific actionable unit. The original Content was therefore consulted where the normalised guideline alone did not provide sufficient information to distinguish between possible themes.

Comparable classification inconsistencies identified in other parts of the catalogue were corrected using the same principle. Where another theme already present in the final codebook represented the original recommendation more precisely, the source-level occurrence was reassigned accordingly. This procedure did not alter the deductive or inductive origin of the themes. It only reconsidered the assignment of individual guidelines.

Within *What to log*, event reporting remained an important category alongside state information and execution tracing. The additional inductive theme *Sensitive-data exclusion* demonstrates that practical guidance also defines information that should not be recorded. This extends the content-focused view of logging beyond deciding what information to capture to defining boundaries on permissible log content.

The *Where to log* dimension showed an important difference between literature-derived and repository-derived concepts. The literature-based codebook contains specific program locations such as assertion checks, return-value checks, exception handling, logic branches and observing points. Several of these themes were represented by comparatively few guidelines in the collected corpus, and assertion-check logging was not represented by a source-level guideline in the final classification.

In contrast, inductively derived themes such as *Logging scope* and *Log destination* occurred more prominently in the repository guidance. This indicates that practical documentation can interpret where not only as a location in source-code control flow, but also as an architectural or operational concern involving the subsystem to which logging applies or the destination to which log records are emitted.

The *How to log* dimension remained an important part of the catalogue after validation. Its deductive themes distinguish between the overall logging approach, the integration of logging utilities and the composition of logging code. Additional

inductive themes capture lifecycle management, verification and the protection of sensitive information. Reassigning many initially broad *Logging approach* occurrences to these and other themes increased the specificity of the final classification. The *Log levels and verbosity* dimension similarly extends beyond the six conventional severity levels used as deductive categories. The inductive theme *Log level selection* represents recommendations concerned with choosing, configuring or changing an appropriate logging level rather than prescribing one particular severity for a specific event. The additional *Custom verbosity scale* theme captures project-specific verbosity systems that do not map directly to the conventional INFO, DEBUG, WARN, ERROR, FATAL, TRACE hierarchy.

The emergence of eleven inductive themes therefore represents a central finding of RQ4. The literature-derived categories provide a meaningful analytical foundation. Still, practical logging documentation includes additional concerns related to sensitive-data handling, logging scope and destination, storage and retention, logger placement, configuration location, lifecycle management, verification and project-specific verbosity concepts.

The thematic distribution should nevertheless be interpreted descriptively. A frequently represented theme is not necessarily more important, effective or correct than a less frequent theme. Some logging practices are inherently specialised and apply only to particular technical environments. The reported counts therefore indicate which concerns were represented within the collected corpus rather than ranking the quality or importance of individual logging practices.

Overall, the thematic analysis shows that the initial literature-derived structure provides a useful basis for organising logging guidance but requires extension and validation when applied to heterogeneous practical repository material. The combination of deductive themes, inductively identified themes and source-level validation provides a thematic structure that remains connected to the specific recommendations represented by the individual canonical guidelines.

5.2 Contribution and Implications

This thesis makes contributions at both the empirical and artefact level.

First, the developer study provides practical observations about logging implementation, log analysis and fault localisation. Although the sample is limited, the responses show how developers use logging in practice and which contextual factors influence logging decisions. These observations complement the guideline analysis by demonstrating the activities for which logging information is eventually consumed. Second, the thesis provides a structured synthesis of logging recommendations identified in academic literature. The literature analysis not only identifies individual recommendations but also provides the deductive foundation for classifying the larger guideline corpus.

Third, the study contributes a large catalogue of logging guidance derived from

open-source repository material. The catalogue contains both source-level occurrences and semantically consolidated canonical guidelines. Maintaining these two analytical levels is important because they support different uses of the data. Source-level occurrences preserve information about how recommendations appear across projects, while canonical guidelines provide a less repetitive representation suitable for practical exploration.

Fourth, the hybrid deductive–inductive codebook extends existing logging classifications. Rather than forcing every recommendation into the literature-derived categories, additional themes were introduced where recurring practical concepts were underrepresented. The final structure therefore connects established research terminology with additional concerns emerging from repository documentation.

Fifth, the catalogue preserves traceability. Normalised guidelines remain linked to their source content, source metadata and consolidation mapping. Where technically relevant code or configuration material was identified, the corresponding example can additionally be preserved as an Example Reason. This traceability matters because normalisation and consolidation necessarily abstract from the original material’s exact wording.

The catalogue’s practical contribution therefore extends beyond the final list of recommendations. Its structure enables exploration by logging dimension, theme, and contextual metadata. A developer investigating a particular logging problem can use these properties to narrow the catalogue while retaining access to the specific recommendation and its source context.

The findings also have implications for logging research. The large representation of implementation-oriented recommendations suggests that practical logging guidance contains substantial operational knowledge that is not necessarily visible when logging is studied only through high-level instrumentation decisions. At the same time, the small representation of some literature-derived themes suggests that concepts prominent in empirical logging research may not always be explicitly formulated as recommendations in open-source guideline documents.

The relationship between the developer study and the catalogue additionally highlights the importance of contextual retrieval. Developers require guidance for specific decisions and scenarios rather than merely a list of abstract logging principles. This suggests that the catalogue should be viewed as a structured knowledge base whose individual entries can later support more context-sensitive retrieval mechanisms.

5.3 Threats to Validity

Several threats to validity need to be considered when interpreting the findings of this thesis. The threats are discussed in terms of internal validity, external validity, construct validity, and conclusion validity.

5.3.1 Internal Validity

Internal validity concerns whether the procedures used during data collection and analysis may have introduced systematic errors into the results. In this study, potential threats arise particularly from the identification, transformation, classification and consolidation of logging guidelines.

AI-assisted processing was used at several stages of open-source catalogue construction, including contextual tagging, guideline normalisation, thematic classification, and the identification of semantically equivalent recommendations. These processing stages may introduce interpretation, classification or transformation errors.

Several measures were applied to reduce this risk. The original repository content was retained throughout the processing pipeline so that transformed guidelines could be compared with their source material. A random sample of normalised guidelines was manually reviewed, `NO_GUIDELINE` entries were investigated, theme assignments were checked, and recurring classification problems identified during validation were examined across related entries. The consolidation mapping was additionally checked to ensure that every retained source-level guideline remained represented in the final catalogue.

Manual summarisation of recommendations extracted from academic literature also involves researcher interpretation. This risk was reduced by retaining the original publication references and by preserving the meaning, direction, conditions and restrictions of the source recommendation during summarisation.

Semantic consolidation introduces a further potential source of error. Overly aggressive consolidation could combine guidelines that express different recommendations or apply under different conditions, whereas overly conservative consolidation may retain guidelines that could reasonably be represented by one canonical guideline. The study therefore followed a conservative strategy: guidelines were retained separately where their recommended actions, conditions or restrictions differed or where semantic equivalence remained uncertain. The original source-level entries were retained through the consolidation mapping so that these decisions remained traceable. This risk was further reduced by manually reviewing the final multi-occurrence consolidation groups against their source Content and by reassigning source occurrences where another existing canonical guideline represented the underlying recommendation more accurately.

Despite these measures, individual processing or classification errors cannot be completely excluded.

5.3.2 External Validity

External validity concerns the extent to which the findings can be generalised beyond the datasets investigated in this thesis.

The developer study was conducted within a single industry organisation operating in the aviation domain. The responses therefore reflect a particular organisational

and technical environment and cannot be assumed to represent software developers or industrial software development in general. Additional studies involving developers from other organisations, application domains and technical environments would be required to assess whether the observed practices and challenges occur more broadly.

The defined GitHub collection similarly limits the open-source dataset procedure. The selected search terminology influenced repository selection, GitHub search ranking and the fixed search-result boundary. The resulting catalogue therefore represents the logging guidance identified within the collected repository corpus rather than all logging guidance available in open-source software.

The searches were also performed at a specific point in time. Repository content, project activity, availability and GitHub search results may change over time. Consequently, the open-source dataset should be interpreted as a snapshot of the guidance available through the defined search procedure at the time of collection.

The academic literature collection does not constitute an exhaustive systematic review of all software logging research. Google Scholar was used as the main search platform, together with a defined publication period and backward snowballing from relevant publications. Publications using different terminology or not surfaced by the search procedure may therefore have been missed.

Accordingly, the coverage comparison should be interpreted only with respect to the selected academic recommendation set and the collected open-source corpus. The observed overlap does not demonstrate that all logging recommendations documented in the wider academic literature are represented in the catalogue, particularly because the literature collection was not exhaustive and some retained recommendations were not fully represented.

5.3.3 Construct Validity

Construct validity concerns whether the concepts investigated in the thesis are represented adequately by the data and classifications used in the analysis.

A central limitation is that logging practices are not always documented explicitly. The repository search and guideline-identification process primarily captures practices that are expressed as identifiable recommendations, requirements, examples, or demonstrated practices. Logging decisions that remain implicit in source code or that are described using terminology not covered by the search strategy may therefore be underrepresented.

The classification of guidelines into themes also requires interpretation. The literature-derived themes provided the initial structure, while additional themes were introduced when the existing classification did not adequately represent the collected guideline content. Different researchers could potentially choose different theme names, boundaries or levels of abstraction. This risk was reduced by applying the literature-derived classification first, reviewing newly introduced themes and merging overlapping inductive themes where they represented the same underlying con-

cept.

The broad *Logging approach* theme represents a particular construct-validity consideration. It contains different implementation-oriented recommendations that share a common higher-level logging concern. Guidelines assigned to this theme are therefore not assumed to be semantically equivalent. The individual guideline remains the specific actionable unit, while the theme provides a broader analytical classification.

The developer study introduces an additional construct-validity limitation because the survey relies on self-reported behaviour. Estimates concerning development effort, logging practices, log-analysis activities, and fault-localisation effort may differ from actual behaviour because of recall bias or different interpretations of the survey questions.

The resulting thematic structure should therefore be understood as an empirically derived organisation of the collected logging guidance rather than as the only possible taxonomy of software logging.

5.3.4 Conclusion Validity

Conclusion validity concerns whether the available evidence supports the conclusions drawn from the collected data.

The number of valid responses in the developer study decreased across the questionnaire because partial responses were retained. Although 26 responses were recorded overall, considerably fewer participants reached the later sections of the questionnaire. Findings based on small numbers of responses should therefore be interpreted as descriptive and exploratory observations rather than as estimates that can be generalised to a wider developer population.

The thematic results are also primarily descriptive. The number of guidelines or source occurrences assigned to a theme indicates how frequently that concern was represented within the collected corpus, but it does not measure the importance, effectiveness or correctness of the corresponding logging practice. Some logging concerns may naturally generate more explicit documentation than others.

Themes containing only a small number of guidelines are particularly sensitive to individual classification errors. Although manual validation was used to investigate such cases, exact counts for small categories should therefore be interpreted with greater caution.

Similarly, the academic coverage analysis is limited to the literature and open-source corpora collected in this study. The substantial overlap observed between the retained academic recommendations and the consolidated catalogue should not be interpreted as evidence that the catalogue covers all logging recommendations documented in academic research. The literature collection was not exhaustive, and some recommendations were only partially represented in the collected open-source guidance.

The conclusions of the thesis are therefore restricted to patterns observed within the collected developer responses, academic literature, and open-source repository

corpus. Frequency and coverage results are used descriptively and are not intended to establish universal rankings or causal relationships between logging practices.

5.4 Future Work

The limitations identified in this study, together with the resulting logging guideline catalogue, provide several opportunities for future research and further development. First, the developer study could be extended to a larger and more diverse sample. Including developers from several organisations, application domains, project sizes, and programming environments would make it possible to examine whether the logging practices and challenges observed in this study also occur in other development contexts. Such an extension could provide a broader view of industrial logging practices beyond the organisation investigated in this thesis.

Second, the academic literature collection could be expanded into a broader systematic literature review. Additional research databases could be included alongside a more extensive search and selection procedure. This would allow the academic coverage analysis to be repeated against a larger literature corpus and could reveal additional logging recommendations not identified in the literature set used in this study.

Third, the open-source repository collection could be extended beyond the current GitHub snapshot. Repeating the repository search at later points in time could support an analysis of how logging guidance develops and changes across software projects. Future studies could also include additional software-development platforms or publicly available technical documentation to broaden the range of open-source guidance considered.

Fourth, the thematic classification could be refined as additional data becomes available. In particular, broad themes such as *Logging approach* could be examined for stable recurring subthemes. Such refinement should remain data-driven and should only introduce additional themes where conceptually distinct and recurring forms of guidance can be identified, rather than splitting a theme because it contains a large number of guidelines.

A further extension would be semantic retrieval. Since the individual guidelines remain specific while themes provide broader organisational categories, vector-based retrieval or a retrieval-augmented language model could be used to identify guidelines that are relevant to a developer's concrete logging problem. For example, a developer could describe a logging scenario and retrieve semantically related recommendations together with their associated examples and source context.

Such a retrieval system would represent an application built on top of the catalogue rather than a replacement for the catalogue itself. The catalogue would continue to provide the structured and traceable knowledge base, while semantic retrieval could improve access to contextually relevant recommendations.

Future work could also evaluate the practical usefulness of the catalogue with software practitioners. Developers could, for example, perform logging-design, maintenance or debugging tasks with and without access to the catalogue. Such an evaluation could provide evidence regarding the usability and discoverability of the catalogue and whether access to the collected guidelines supports practical logging decisions.

Finally, future research could investigate conflicting or context-dependent logging recommendations in more detail. Two guidelines may address the same logging concern while remaining appropriate under different technical or operational conditions. Explicitly representing applicability conditions, limitations, and relationships between recommendations could therefore increase the catalogue's practical value and support more context-sensitive use of the collected logging guidance.

5.5 Conclusion

This thesis investigated software logging from three complementary perspectives: developer practices, academic research and guidance documented in open-source repositories and related practical sources.

The developer study showed that logging supports debugging, failure identification, and fault localisation, but also requires contextual decisions about which information to record and how to interpret it later. The responses further demonstrate that effective log analysis depends not only on the existence of log data but also on its structure, context and the developer's knowledge of the underlying system.

The academic literature provided both explicit logging recommendations and an established conceptual foundation for organising logging decisions. The literature collection contained 31 academic publications, while tool-focused and other non-prescriptive material was retained as supporting evidence where relevant. The literature-derived concepts formed the deductive starting point for analysing the larger open-source corpus.

The actionable academic recommendations were manually compared with the final consolidated open-source catalogue. The comparison showed substantial semantic overlap between academic recommendations and the practical guidance identified in the collected open-source sources. Some recommendations were only partially represented because individual aspects or conditions of the academic guidance were not identified completely in the collected corpus.

The repository analysis demonstrated that practical logging guidance is extensive and heterogeneous. From the validated source-level catalogue, 2.289 guideline occurrences were identified and subsequently consolidated into 2.127 canonical guidelines while preserving provenance and source-level traceability.

The thematic analysis resulted in 28 themes across the four dimensions *what to log*, *where to log*, *how to log*, and *log levels and verbosity*. Seventeen themes were derived

from previous literature, and eleven additional themes emerged from the collected guideline content.

The catalogue distribution shows that implementation-oriented guidance makes up the largest part of the collected corpus. At the same time, the inductive themes demonstrate that practical logging guidance extends beyond the initial literature-derived classification, particularly regarding logging scope and destination, storage and lifecycle management, sensitive-data handling, verification and general log-level selection.

The central contribution of the thesis is therefore not a claim that one universal logging strategy can be applied to every software system. Instead, the study provides a structured and traceable catalogue of concrete logging recommendations and an analytical framework for organising those recommendations.

The distinction between themes and individual guidelines is important. Themes provide an abstraction for understanding and navigating the catalogue, while the individual guidelines retain the contextual specificity required for practical application. This structure allows the catalogue to remain useful even where logging recommendations depend strongly on framework, project or operational context.

Overall, the findings demonstrate that software logging is a multidimensional and context-dependent engineering activity. Combining academically documented logging knowledge with guidance identified in practical open-source development provides a broader view of the decisions involved in software logging. The resulting catalogue establishes a foundation that can support developers in exploring existing logging recommendations and can provide a basis for future research and context-sensitive guideline retrieval.

List of Figures

3.1	Overview of the research design.	22
4.1	Percentage of overall development time spent implementing logging among respondents who implement logging.	50
4.2	Types of information logged according to the literature-derived <i>What to log</i> codebook. Multiple categories could be assigned to one response.	51
4.3	Methods and tools used for logging. Multiple selections were possible.	51
4.4	Criteria used to determine the criticality of information to be logged. Multiple selections were possible.	52
4.5	Respondents' perceptions of logging implementation practices and difficulties.	53
4.6	Percentage of overall time spent analysing logged data among respondents who completed the log-analysis section ($n = 7$).	54
4.7	Reported reasons for analysing logged data ($n = 7$). Multiple selections were possible.	54
4.8	Respondents' perceptions of log analysis. The failure-related-log statement is based on six responses; the remaining statements are based on seven responses.	55
4.9	Reported frequency of analysing unfamiliar logs and reviewing logs collaboratively ($n = 7$).	56
4.10	Percentage of overall time spent localising faults among respondents in the fault-localisation section ($n = 6$).	57
4.11	Reported average and maximum duration required to localise the fault of a single failure ($n = 6$).	57
4.12	Reported level of knowledge required for fault localisation across different areas ($n = 6$).	58
4.13	Respondents' reported experiences and challenges during fault localisation ($n = 6$). Statement labels were shortened for readability.	59
A.1	Selected descriptive results for logging implementation. Multiple selections were possible where applicable.	104
A.2	Criteria used to determine the criticality of information to be logged. Multiple selections were possible.	105
A.3	Effects of criticality and respondents' perceptions of logging implementation.	105
A.4	Selected descriptive results for log analysis.	107

A.5	Respondents' perceptions of log analysis. The statement concerning failure-related logs is based on six responses due to one missing answer; the remaining statements are based on seven responses.	108
A.6	Reported effort and duration of fault localization activities ($n = 6$). .	111
A.7	Reported level of knowledge required for fault localization across different areas ($n = 6$).	111
A.8	Respondents' perceptions and experiences concerning fault localization ($n = 6$). Statement labels were shortened for readability.	112

List of Tables

1.1	Research Questions	4
3.1	GitHub search queries used for repository collection.	29
4.1	Academic sources used to ground the initial deductive classification.	62
4.2	Focus and number of results obtained for the ten GitHub search queries.	65
4.3	File types in the collected logging corpus before content extraction.	66
4.4	Descriptive characteristics of the GitHub repositories contributing valid logging guidelines.	67
4.5	Open-source guideline collection, extraction, and consolidation results.	69
4.6	Distribution of source-level guideline occurrences and final consoli- dated guidelines across the four logging dimensions.	74
4.7	Themes within the <i>What to log</i> dimension.	75
4.8	Themes within the <i>Where to log</i> dimension.	76
4.9	Themes within the <i>How to log</i> dimension.	77
4.10	Themes within the <i>Log levels and verbosity</i> dimension.	78
4.11	Final thematic codebook by logging dimension and theme origin.	79
A.1	Paraphrased open-ended responses concerning logging implementation.	106
A.2	Paraphrased responses concerning log selection and the analytical pro- cess.	108
A.3	Paraphrased open-ended responses concerning collaboration, difficul- ties, and recommended log analysis practices.	109
A.4	Paraphrased responses concerning identification of affected system functionality and additional data used for fault localization.	113
A.5	Paraphrased responses concerning failures involving interactions be- tween multiple functionalities or system parts.	114
A.6	Paraphrased responses concerning collaboration, challenges, and rec- ommended practices for fault localization.	115

List of Prompts

1	Prompt for logging content identification and XML classification . .	34
2	Prompt for contextual content tagging	36
3	Prompt for guideline formulation and normalization	37
4	Prompt for deductive-inductive theme classification	41
5	Prompt for guideline consolidation and codebook refinement	43

Bibliography

- [1] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. “Logram: Efficient Log Parsing Using nn -Gram Dictionaries”. In: *IEEE Transactions on Software Engineering* 48.3 (2022), pp. 879–892. DOI: 10.1109/TSE.2020.3007554.
- [2] Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R. Lyu. “Tools and Benchmarks for Automated Log Parsing”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 2019, pp. 121–130. DOI: 10.1109/ICSE-SEIP.2019.00021.
- [3] Zanis Ali Khan, Donghwan Shin, Domenico Bianculli, and Lionel Briand. “Guidelines for Assessing the Accuracy of Log Message Template Identification Techniques”. In: *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. 2022, pp. 1095–1106. DOI: 10.1145/3510003.3510101.
- [4] Sina Gholamian and Paul A. S. Ward. *A Comprehensive Survey of Logging in Software: From Logging Statements Automation to Log Mining and Analysis*. 2022. arXiv: 2110.12489v2 [cs.SE]. URL: <https://arxiv.org/abs/2110.12489>.
- [5] Boyuan Chen and Zhen Ming (Jack) Jiang. “A Survey of Software Log Instrumentation”. In: *ACM Comput. Surv.* 54.4 (May 2021). ISSN: 0360-0300. DOI: 10.1145/3448976. URL: <https://doi.org/10.1145/3448976>.
- [6] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. “A Survey on Automated Log Analysis for Reliability Engineering”. In: *ACM Comput. Surv.* 54.6 (July 2021). ISSN: 0360-0300. DOI: 10.1145/3460345. URL: <https://doi.org/10.1145/3460345>.
- [7] Boyuan Chen and Zhen Ming Jiang. “Characterizing and Detecting Anti-Patterns in the Logging Code”. In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 2017, pp. 71–81. DOI: 10.1109/ICSE.2017.15.

- [8] Shilin He, Xu Zhang, Pinjia He, Yong Xu, Liqun Li, Yu Kang, Minghua Ma, Yining Wei, Yingnong Dang, Saravanakumar Rajmohan, and Qingwei Lin. “An Empirical Study of Log Analysis at Microsoft”. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2022. Singapore, Singapore: Association for Computing Machinery, 2022, pp. 1465–1476. ISBN: 9781450394130. DOI: 10.1145/3540250.3558963. URL: <https://doi.org/10.1145/3540250.3558963>.
- [9] Ding Yuan, Soyeon Park, and Yuanyuan Zhou. “Characterizing Logging Practices in Open-Source Software”. In: *2012 34th International Conference on Software Engineering (ICSE)*. 2012, pp. 102–112. DOI: 10.1109/ICSE.2012.6227202.
- [10] Medium. *13 Best Practices to Improve Your Logging*. <https://medium.com/cogito-engineering/13-best-practices-to-improve-your-logging-75665183f05b>. Accessed: 2022-05-18. 2022.
- [11] Loggly. *30 Best Practices for Logging at Scale*. <https://www.loggly.com/blog/30-best-practices-logging-scale/>. Accessed: 2022-05-18. 2022.
- [12] Shenghui Gu, Guoping Rong, He Zhang, and Haifeng Shen. “Logging Practices in Software Engineering: A Systematic Mapping Study”. In: *IEEE Transactions on Software Engineering* 49.2 (2023), pp. 902–923. DOI: 10.1109/TSE.2022.3166924.
- [13] Qiang Fu, Jieming Zhu, Wenlu Hu, Jian-Guang Lou, Rui Ding, Qingwei Lin, Dongmei Zhang, and Tao Xie. “Where Do Developers Log? An Empirical Study on Logging Practices in Industry”. In: *Companion Proceedings of the 36th International Conference on Software Engineering*. ICSE Companion 2014. Hyderabad, India: Association for Computing Machinery, 2014, pp. 24–33. ISBN: 9781450327688. DOI: 10.1145/2591062.2591175. URL: <https://doi.org/10.1145/2591062.2591175>.
- [14] Sina Gholamian and Paul A. S. Ward. “What Distributed Systems Say: A Study of Seven Spark Application Logs”. In: *2021 40th International Symposium on Reliable Distributed Systems (SRDS)*. 2021, pp. 222–232. DOI: 10.1109/SRDS53918.2021.00030.
- [15] Boyuan Chen and Zhen Ming (Jack) Jiang. “Studying the use of Java logging utilities in the wild”. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. ICSE ’20. Seoul, South Korea: Association for Computing Machinery, 2020, pp. 397–408. ISBN: 9781450371216. DOI: 10.1145/3377811.3380408. URL: <https://doi.org/10.1145/3377811.3380408>.

- [16] Jieming Zhu, Pinjia He, Qiang Fu, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. “Learning to Log: Helping Developers Make Informed Logging Decisions”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. 2015, pp. 415–425. DOI: 10.1109/ICSE.2015.60.
- [17] Tsuyoshi Mizouchi, Kazumasa Shimari, Takashi Ishio, and Katsuro Inoue. “PADLA: A Dynamic Log Level Adapter Using Online Phase Detection”. In: *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. 2019, pp. 135–138. DOI: 10.1109/ICPC.2019.00029.
- [18] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. “Using deep learning to generate complete log statements”. In: *Proceedings of the 44th International Conference on Software Engineering*. ICSE ’22. Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 2279–2290. ISBN: 9781450392211. DOI: 10.1145/3510003.3511561. URL: <https://doi.org/10.1145/3510003.3511561>.
- [19] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liqun Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. “UniParser: A Unified Log Parser for Heterogeneous Log Data”. In: *Proceedings of the ACM Web Conference 2022*. WWW ’22. Virtual Event, Lyon, France: Association for Computing Machinery, 2022, pp. 1893–1901. ISBN: 9781450390965. DOI: 10.1145/3485447.3511993. URL: <https://doi.org/10.1145/3485447.3511993>.
- [20] Boyuan Chen and Zhen Ming (Jack) Jiang. “Characterizing logging practices in Java-based open source software projects – a replication study in Apache Software Foundation”. In: *Empirical Software Engineering* 22.1 (Feb. 2017), pp. 330–374. ISSN: 1573-7616. DOI: 10.1007/s10664-016-9429-5. URL: <https://doi.org/10.1007/s10664-016-9429-5>.
- [21] Ding Yuan, Jing Zheng, Soyeon Park, Yuanyuan Zhou, and Stefan Savage. “Improving Software Diagnosability via Log Enhancement”. In: *ACM Trans. Comput. Syst.* 30.1 (Feb. 2012). ISSN: 0734-2071. DOI: 10.1145/2110356.2110360. URL: <https://doi.org/10.1145/2110356.2110360>.
- [22] Weiyi Shang, Meiyappan Nagappan, Ahmed E. Hassan, and Zhen Ming Jiang. “Understanding Log Lines Using Development Knowledge”. In: *2014 IEEE International Conference on Software Maintenance and Evolution*. 2014, pp. 21–30. DOI: 10.1109/ICSME.2014.24.
- [23] Mohamed Amine Batoun, Mohammed Sayagh, Roozbeh Aghili, Ali Ouni, and Heng Li. “A literature review and existing challenges on software logging practices”. In: *Empirical Software Engineering* 29.4 (June 2024), p. 103. ISSN: 1573-7616. DOI: 10.1007/s10664-024-10452-w. URL: <https://doi.org/10.1007/s10664-024-10452-w>.

- [24] Guoping Rong, Yangchen Xu, Shenghui Gu, He Zhang, and Dong Shao. “Can You Capture Information As You Intend To? A Case Study on Logging Practice in Industry”. In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2020, pp. 12–22. DOI: 10.1109/ICSME46990.2020.00012.
- [25] Zhenhao Li, Tse-Hsun (Peter) Chen, and Weiyi Shang. “Where shall we log? studying and suggesting logging locations in code blocks”. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. ASE ’20. Virtual Event, Australia: Association for Computing Machinery, 2021, pp. 361–372. ISBN: 9781450367684. DOI: 10.1145/3324884.3416636. URL: <https://doi.org/10.1145/3324884.3416636>.
- [26] Antonio Pecchia, Marcello Cinque, Gabriella Carrozza, and Domenico Cotroneo. “Industry Practices and Event Logging: Assessment of a Critical Software Development Process”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 2. 2015, pp. 169–178. DOI: 10.1109/ICSE.2015.145.
- [27] Ying Fu, Meng Yan, Zhou Xu, Xin Xia, Xiaohong Zhang, and Dan Yang. “An empirical study of the impact of log parsers on the performance of log-based anomaly detection”. In: *Empirical Software Engineering* 28.1 (Nov. 2022), p. 6. ISSN: 1573-7616. DOI: 10.1007/s10664-022-10214-6. URL: <https://doi.org/10.1007/s10664-022-10214-6>.
- [28] Guoping Rong, Shenghui Gu, Haifeng Shen, He Zhang, and Hongyu Kuang. “How Do Developers’ Profiles and Experiences Influence their Logging Practices? An Empirical Study of Industrial Practitioners”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2023, pp. 855–867. DOI: 10.1109/ICSE48619.2023.00080.
- [29] Julian Harty, Haonan Zhang, Lili Wei, Luca Pascarella, Maurício Aniche, and Weiyi Shang. “Logging Practices with Mobile Analytics: An Empirical Study on Firebase”. In: *2021 IEEE/ACM 8th International Conference on Mobile Software Engineering and Systems (MobileSoft)*. 2021, pp. 56–60. DOI: 10.1109/MobileSoft52590.2021.00013.
- [30] Haonan Zhang, Yiming Tang, Maxime Lamothe, Heng Li, and Weiyi Shang. “Studying logging practice in test code”. In: *Empirical Software Engineering* 27.4 (Apr. 2022), p. 83. ISSN: 1573-7616. DOI: 10.1007/s10664-022-10139-0. URL: <https://doi.org/10.1007/s10664-022-10139-0>.
- [31] Ying Fu, Meng Yan, Pinjia He, Chao Liu, Xiaohong Zhang, and Dan Yang. “End-to-end log statement generation at block-level”. In: *Journal of Systems and Software* 216 (2024), p. 112146. ISSN: 0164-1212. DOI: 10.1016/j.jss.2024.112146. URL: <https://www.sciencedirect.com/science/article/pii/S0164121224001912>.

- [32] Yichen Li, Yintong Huo, Renyi Zhong, Zhihan Jiang, Jinyang Liu, Junjie Huang, Jiazhen Gu, Pinjia He, and Michael R. Lyu. “Go Static: Contextualized Logging Statement Generation”. In: *Proc. ACM Softw. Eng.* 1.FSE (July 2024). DOI: 10.1145/3643754. URL: <https://doi.org/10.1145/3643754>.

A Survey Results

A.0.1 Logging Implementation

This section presents the descriptive results of the logging implementation part of the developer survey. Eight respondents completed the logging implementation questions ($n = 8$). Two additional survey participants did not implement logging and therefore did not answer the remaining questions in this section. Unless stated otherwise, all results in this subsection refer to these eight respondents.

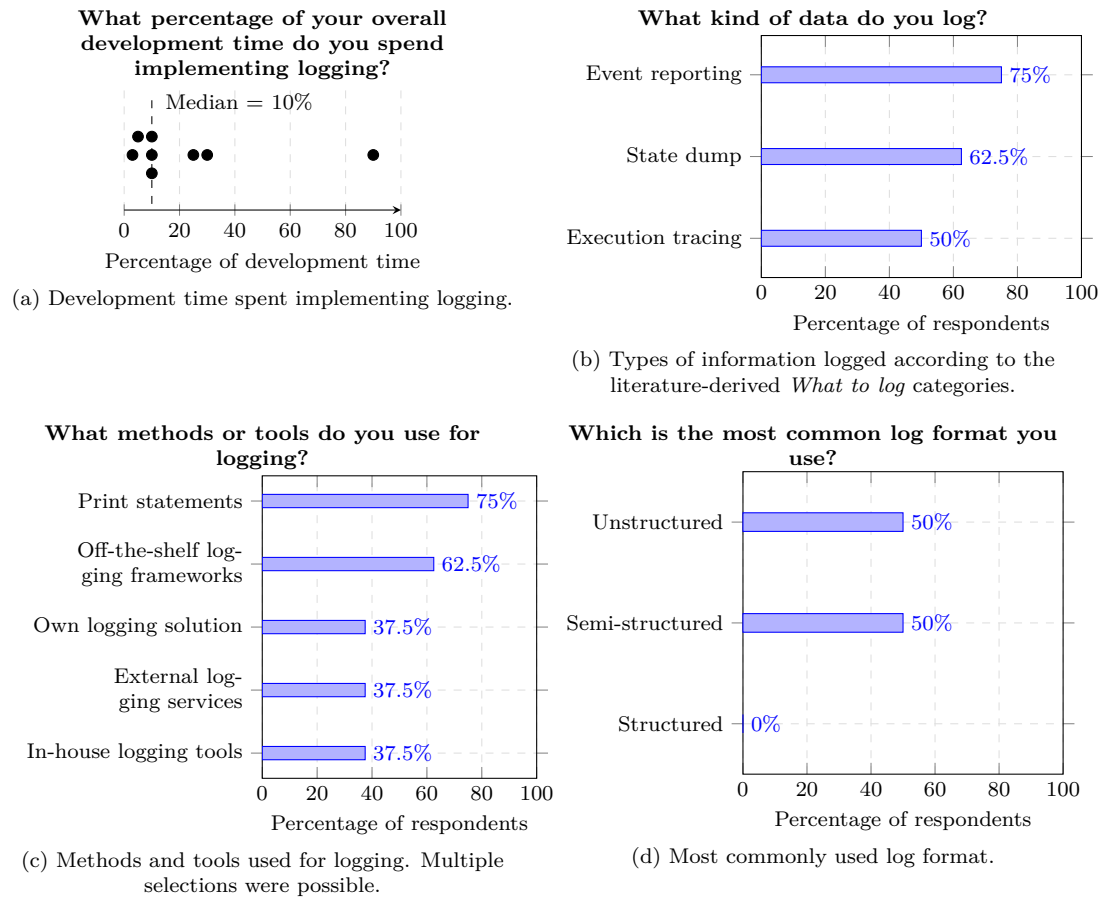


Figure A.1: Selected descriptive results for logging implementation. Multiple selections were possible where applicable.

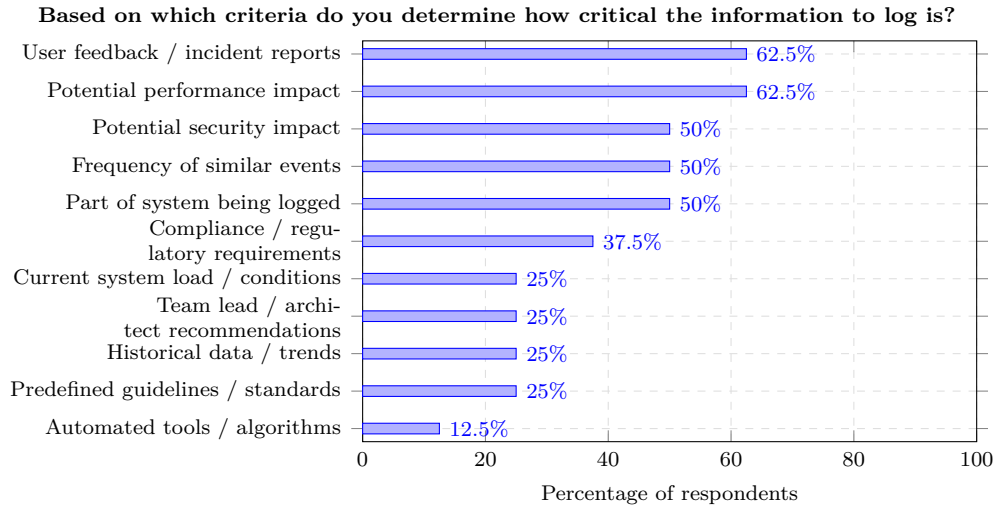


Figure A.2: Criteria used to determine the criticality of information to be logged. Multiple selections were possible.

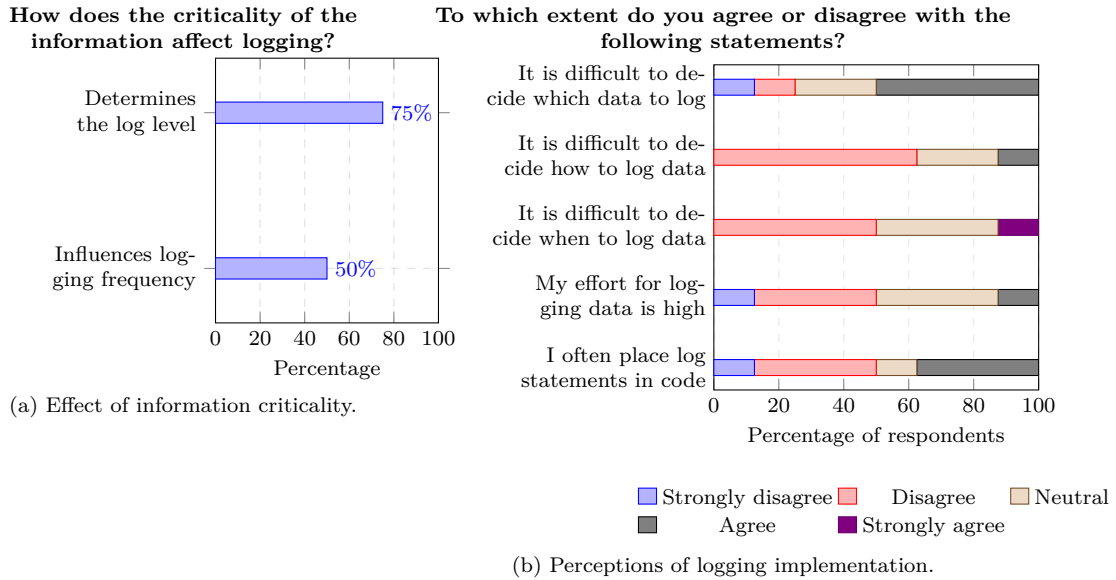


Figure A.3: Effects of criticality and respondents' perceptions of logging implementation.

A.0.1.1 Open-Ended Logging Implementation Responses

The following table presents lightly paraphrased versions of the open-ended survey responses. Responses were rephrased for readability and conciseness while preserving their original meaning. No additional thematic coding was applied.

Table A.1: Paraphrased open-ended responses concerning logging implementation.

ID	Colleague involvement	Challenges	Effective strategies
P1	The response focused on tracing errors back to the specific method or class rather than direct colleague involvement.	Outside classes with established logging, it can be unclear where and what to log.	Use a clear, readable, and well-structured log file.
P2	Colleagues are not involved.	No specific difficulties were reported.	No particular strategy was reported.
P3	Colleagues are not involved, mainly because the respondent has not worked on larger collaborative projects.	Performance-related logging statements can be difficult to place; therefore, a larger amount of information may be logged.	Use third-party applications for error logging.
P4	The response focused on using logs to identify errors rather than direct colleague involvement.	The purpose and subsequent use of collected logging metrics can be unclear.	Add logs at module or component level to trace execution flow.
P5	Colleagues are not involved.	No specific difficulties were reported.	No relevant experience with logging strategies was reported.
P6	Colleagues may be consulted about relevant data and logging requirements affecting architecture or component design.	Logging itself is usually not difficult, but suitable logging options should be considered during architecture and design.	Use the same framework where possible and avoid both excessive and insufficient logging.
P7	Colleagues are not directly involved, but logs are written so they can analyse them later.	Finding a concise and readable log format can be challenging.	Log incoming and outgoing data.
P8	The team discusses and defines how logging should be implemented.	Designing how data should be displayed in a custom log format can be difficult.	Write meaningful entries, sample logs, and consider logging performance costs.

A.0.2 Log Analysis

This section presents the descriptive results of the log analysis part of the developer survey. Seven respondents completed the log analysis questions ($n = 7$). Three additional survey participants did not answer this section. Unless stated otherwise, all results in this subsection refer to these seven respondents.



Figure A.4: Selected descriptive results for log analysis.

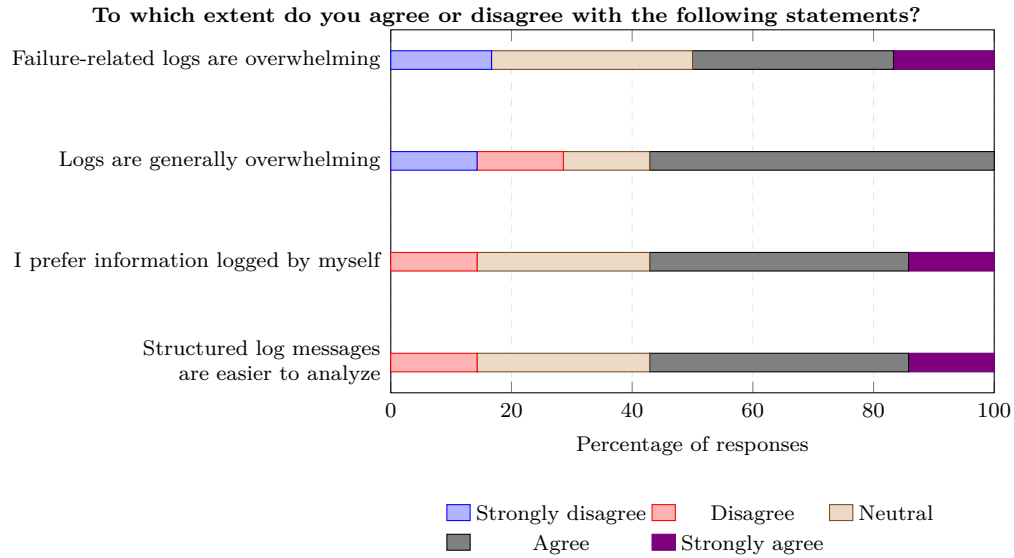


Figure A.5: Respondents' perceptions of log analysis. The statement concerning failure-related logs is based on six responses due to one missing answer; the remaining statements are based on seven responses.

How do you typically access logs? Rank the methods below from most often to least often. None of the respondents provided a ranking for the listed log access methods. Therefore, no quantitative ranking is presented for this question.

A.0.2.1 Open-Ended Log Analysis Responses

The following tables present lightly paraphrased versions of the open-ended survey responses. The responses were rephrased for readability and conciseness while preserving their original meaning. No additional thematic coding was applied.

Table A.2: Paraphrased responses concerning log selection and the analytical process.

ID	How do you know which logs to look at?	Analytical process
P1	Recent logs associated with a build or operation that resulted in an error are examined.	Find the relevant log, search for the error, inspect the corresponding source code, understand the cause, apply a fix, and rerun the software.
P2	Relevant logs are selected based on previous experience.	Understand the customer's reported problem and inspect the corresponding logs for relevant details.

Continued on next page

Table continued from previous page

ID	How do you know which logs to look at?	Analytical process
P3	The affected component is first understood and the corresponding log output path is identified in the source code.	Read the log, trace the corresponding code, understand the code flow, and identify the issue or error.
P4	Logs from the most recent experiment are usually examined. For software developed by others, the developer or documentation may be consulted.	Locate the most recent error or warning message and follow it through the code in a manner similar to following a stack trace.
P5	Relevant logs are identified using their naming syntax.	Use the timestamp and fault location and perform a text search for the relevant information.
P6	Logs are searched specifically for strings containing error .	Copilot is primarily used for analysis. Previously, documentation and interpretation of error codes were used.
P7	The relevant log is selected based on the source of the problem; in many cases the respondent already knows which log is most important.	Examine system behaviour and how data changes over time to identify unexpected or missing input and output data.

Table A.3: Paraphrased open-ended responses concerning collaboration, difficulties, and recommended log analysis practices.

ID	Colleague involvement	Issues or difficulties	Recommended practices
P1	Colleagues become involved when important components fail, particularly where the respondent has previously worked.	Different log structures can make logs difficult to understand.	Use clear and well-structured logs and, where possible, a consistent standard format.
P2	Colleagues are involved only when additional developer-level support is required.	No particular difficulties were reported.	No particular strategy was reported.
P3	Team members use the logs to identify issues and support debugging.	Understanding the code flow may remain difficult even after tracing the corresponding log.	Understand the relevant component and execution flow and analyse the logs to identify abnormal behaviour.
P4	Colleagues are rarely involved; when collaboration occurs, it is similar to pair programming.	Log messages are particularly difficult to use when they do not indicate where the message originated.	Perform log analysis diligently and in a structured manner.
P5	Colleagues participate within the scope of their programming responsibilities.	Cryptic descriptions and missing information, such as timestamps, make analysis difficult.	Use text search and note frequently occurring or relevant keywords.

Continued on next page

Table continued from previous page

ID	Colleague involvement	Issues or difficulties	Recommended practices
P6	Other colleagues are involved when the respondent does not understand a log entry.	The main difficulty is identifying the root cause of a failure.	No particular strategy was reported.
P7	The issue is discussed collaboratively to determine the expected or correct behaviour of the system.	Identifying the correct timestamp can be difficult.	Format logs so that important information is clearly highlighted and easy to identify.

A.0.3 Fault Localization

This section presents the descriptive results of the fault localization part of the developer survey. Six respondents completed the fault localization questions ($n = 6$). Four additional survey participants did not provide responses to this section. Unless stated otherwise, all results in this subsection refer to these six respondents.

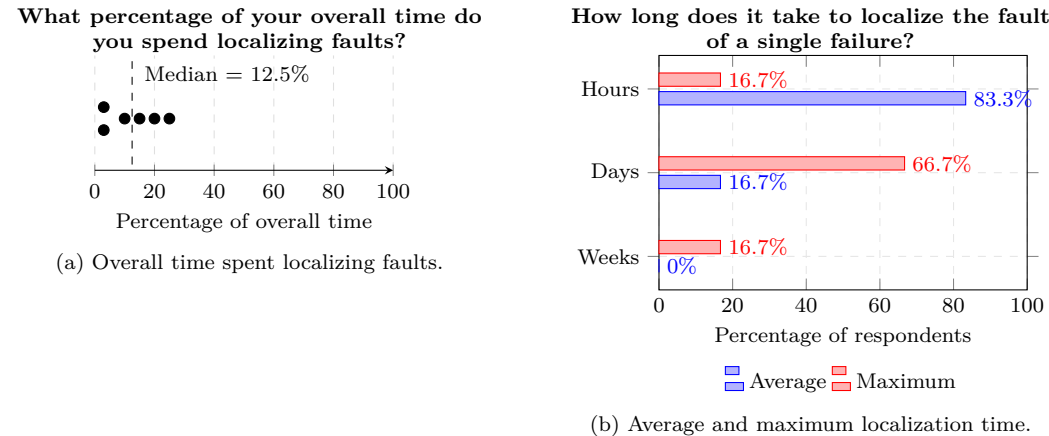


Figure A.6: Reported effort and duration of fault localization activities ($n = 6$).

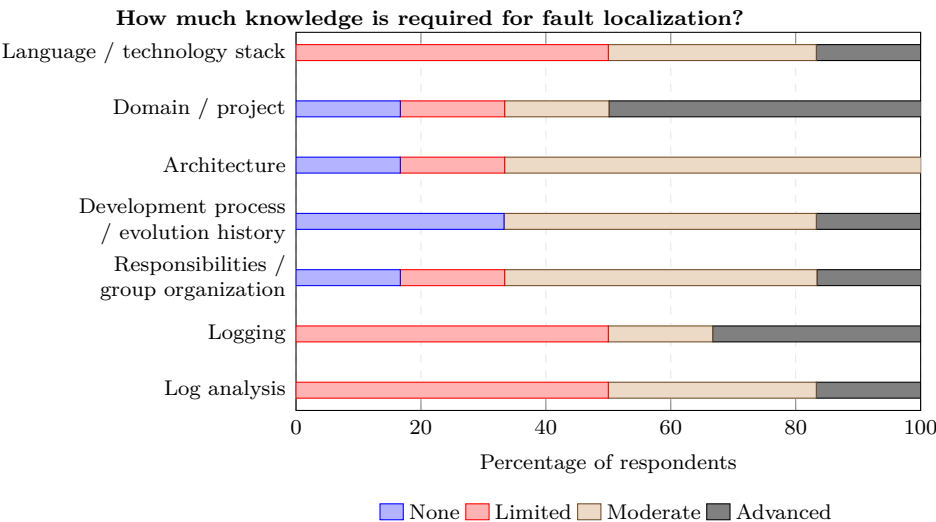


Figure A.7: Reported level of knowledge required for fault localization across different areas ($n = 6$).

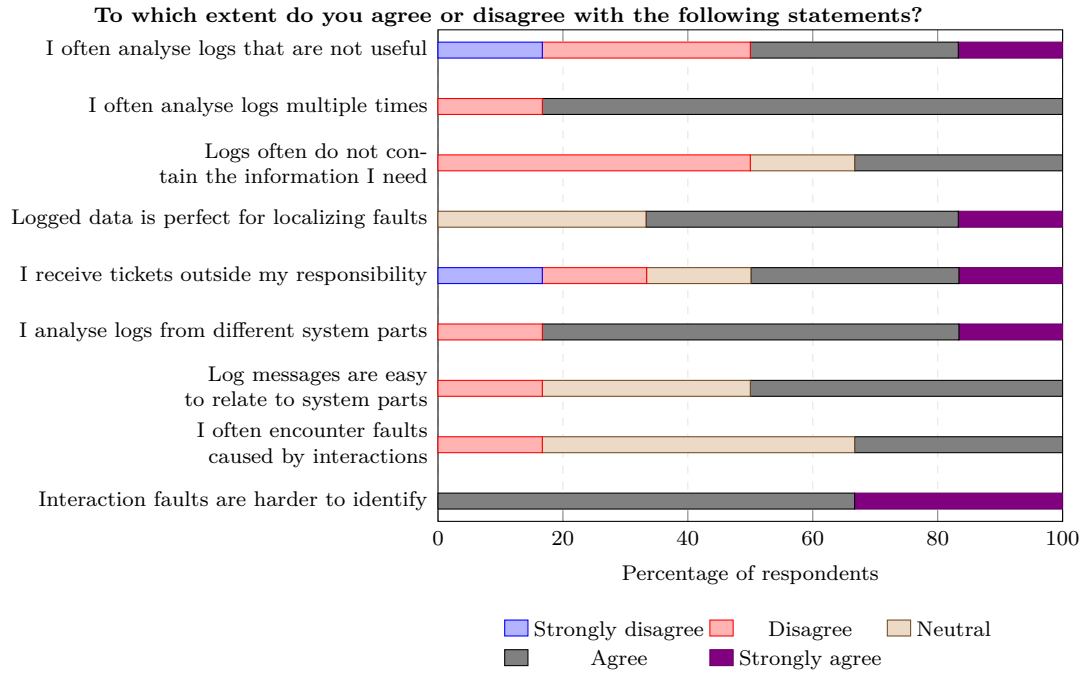


Figure A.8: Respondents' perceptions and experiences concerning fault localization ($n = 6$). Statement labels were shortened for readability.

A.0.3.1 Open-Ended Fault Localization Responses

The following tables present lightly paraphrased versions of the open-ended survey responses. The responses were rephrased for readability and conciseness while preserving their original meaning. No additional thematic coding was applied.

ID	How do you identify the functionality or part of the system affected by a log message?	What data are used in addition to logs?
P1	The cause of the problem is investigated and traced back to the main underlying problem so that similar or future errors can also be addressed.	Debugger information and variable values from different program states are used.
P2	Semi-structured log messages containing the corresponding class name are used to identify the affected part of the system.	System configuration exports are used as additional information.
P3	The affected functionality is identified directly from the log or its origin information where possible. Otherwise, the source code is searched.	Files produced by the system may be inspected when relevant.

Continued on next page

Table continued from previous page

ID	How do you identify the functionality or part of the system affected by a log message?	What data are used in addition to logs?
P4	The affected part of the system is identified using labels associated with the log entries.	Database entries are used as additional information.
P5	Installer logs containing WiX Toolset-specific messages are used to identify the affected functionality.	No additional data were reported.
P6	Unexpected or missing input or output data are used to identify the affected part of the system.	Clients are asked to describe the error and, where possible, provide a video demonstrating the problem.

Table A.4: Paraphrased responses concerning identification of affected system functionality and additional data used for fault localization.

ID	Identifying interactions	Handling interaction failures	Example
P1	The error is traced backwards until the main underlying problem is found.	Failures are listed, the developer responsible for the relevant implementation is identified, and the error is then investigated and fixed.	Rather than providing a specific failure example, the response described a process of detecting the error, contacting the relevant developer, and either collaboratively or independently fixing it.
P2	No clear response was provided.	The owners of the affected system parts are contacted.	An example involved business logic communicating with a third-party system through an adapter.
P3	A thorough understanding of how the system operates is developed.	The involved functionalities are isolated, analysed individually, and then analysed together.	No specific example was provided.
P4	Historically accumulated company knowledge is used to understand interactions between system components.	The approach depends on the particular failure.	An example involved application functionality that depended on mandatory hardware components.
P5	No response was provided.	No response was provided.	No response was provided.
P6	The normal behaviour of the individual functionalities is considered and their overall interaction is examined.	The people responsible for the other functionalities or systems are contacted and the problem is investigated collaboratively.	Two interacting systems exchanged control signals. A required signal was either not received or occurred with problematic timing. The analysis therefore had to determine whether the fault originated in another team's system or from timing and latency between the systems.

Continued on next page

Table continued from previous page

ID	Identifying interactions	Handling interaction failures	Example
-----------	---------------------------------	--------------------------------------	----------------

Table A.5: Paraphrased responses concerning failures involving interactions between multiple functionalities or system parts.

ID	Colleague involvement	Challenges	Recommended practices
P1	The extent of colleague involvement depends on how strongly colleagues were involved in the relevant project and implementation.	Finding the correct location of a fault in a large project can be difficult.	Trace the error backwards until the underlying root problem is found.
P2	Colleagues support confirmation of the issue from the logs and help define an appropriate solution or fix.	No specific challenge was reported.	No specific strategy was reported.
P3	When software belonging to colleagues is involved in the problem, fault localization becomes a collaborative process similar to pair programming.	The main difficulty arises when the respondent is unfamiliar with how a system works.	First obtain an overview of the system structure and its logging practices or policies.
P4	Developers inspect the logs associated with the parts of the system for which they are responsible.	A lack of historically accumulated company knowledge makes fault localization more difficult.	Use text search to locate relevant information.
P5	No specific colleague involvement was reported.	No specific challenge was reported.	No specific strategy was reported.
P6	The behaviour of the overall system is discussed collaboratively to identify the source of the fault.	Timing and latency across multiple systems are difficult to analyse, particularly when other systems have changed without notification.	Discuss the expected behaviour of all involved systems, construct a flowchart of their interactions, and compare the expected sequence with the observed behaviour.

Table A.6: Paraphrased responses concerning collaboration, challenges, and recommended practices for fault localization.

Titel der Arbeit / title of the paper

Mining and Structuring Logging Guidelines for Improving Log Analysis

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich keine Arbeit in gleicher oder ähnlicher Fassung bereits als Prüfungsleistung an der Ruhr-Universität Bochum oder einer anderen Hochschule eingereicht habe.

Ich versichere, dass die vorliegende Arbeit auf meiner konzeptionellen, methodischen und inhaltlichen Eigenleistung basiert und ich keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle direkten und indirekten Zitate sowie KI-generierten fachlichen Inhalte habe ich unter Angabe der Quellen kenntlich gemacht. Diese Versicherung bezieht sich auf alle Textbausteine, Tabellen, Codes, Grafiken und andere Abbildungen. Mir ist bewusst, dass die Unterlassung der angemessenen Dokumentation mit einem Täuschungsversuch aufgrund der Verwendung nicht zulässiger Hilfsmittel gleichzusetzen ist.

Im Falle der Verwendung von KI-Tools habe ich allen Output sorgfältig geprüft und übernehme die volle Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Ich erkläre mich außerdem damit einverstanden, dass die digitale Version dieser Arbeit zwecks Plagiatsprüfung verwendet wird.

Declaration of Authorship

I hereby declare that I have never before submitted this paper in this or similar form for examination at Ruhr-Universität Bochum or any other institution or university.

I profess that this paper is the result of my own conceptional, methodical and analytical work and that I have not used any sources or tools other than the ones stated. Any quotes, whether direct or indirect, and any substantial content that was generated with the help of AI have been fully disclosed by using proper marking and quotation. This declaration is valid for all textual elements, charts, codes, images and figures. I am aware that neglecting the proper documentation constitutes an attempt at deception by use of unauthorized aid.

When using AI tools, I have critically reviewed and verified all output and assume full responsibility for any misrepresented or falsified content, faulty references, plagiarism and breaches of data protection or copyright laws that may result from the use of generative AI.

I furthermore agree that the digital version of this thesis will be used to subject the paper to plagiarism examination.

Not this English translation but only the official version in German is legally binding.

Beim Verfassen dieser Arbeit habe ich die unten aufgeführten KI-Tools für folgende Zwecke verwendet:

In writing this paper, I have used the AI tools specified below for the following purposes:

I used ChatGPT (GPT-5.6 Sol) to rephrase and restructure sentences and paragraphs.

That makes the thesis easier to read and improves the transition between sections.

All AI-assisted text was reviewed and adjusted by me.

GPT-5.4 was used for the initial GitHub search queries.

I also used ChatGPT to help correct TikZ figures and diagrams.

Other AI-supported tasks are described in the thesis, and the prompts are listed in the appendix.

30.09.2026

Mahadeva, Dilsan

Datum / date

Name, Vorname / surname, name

Unterschrift / signature